

DATA STREAMING AND ANALYSIS

From Real-Time Data
to Powerful Insights



STREAM



PROCESS



ANALYZE



INSIGHTS

STREAM. PROCESS. ANALYZE. INSPIRE.
Turning Data into Decisions

P. Renukadevi

Chinmaya Kumar Swain

Archana Sasi

Shahad P



LURNEXA PUBLICATIONS

DATA STREAMING AND ANALYSIS

-Dr. P. Renukadevi
Dr. Chinmaya Kumar Swain
Dr. Archana Sasi
Mr. Shahad P

LURNEXA PUBLICATIONS

Copyright © 2026 Lurnexa Publications. All rights reserved.

No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without prior written permission of the publisher, except in the case of brief quotations embodied in critical reviews and certain other non-commercial uses permitted by copyright law.

Title: *DATA STREAMING AND ANALYSIS*

Authors: Dr. P. Renukadevi, Dr. Chinmaya Kumar Swain, Dr. Archana Sasi, Mr. Shahad P

Edition: First Edition, 2026

Published by:

Lurnexa Publications Private Limited
(An ISO 9001:2015 Certified Organization)
130–187, Ramulavari Gudi Centre,
Gorantla, Guntur – 522034, Andhra Pradesh, India

Email: lurnexapublication@gmail.com

Website: www.lurnexa.in

ISBN: 978-81-685077-8-4

Disclaimer:

The views expressed in this publication are those of the authors and do not necessarily reflect those of the publisher. While every effort has been made to ensure accuracy, the publisher shall not be liable for any loss or damage arising from the use of this material.

Cataloguing-in-Publication Data: Available on request

Printed and bound in India

PREFACE

This book provides a structured and comprehensive introduction to **Data Streaming, Streaming Data Processing, Data Transformation, Real-Time Data Processing, and Data Visualization and Analytics**. It explains how continuously generated data can be collected, processed, transformed, analyzed, and visualized to support timely and effective decision-making.

The book introduces fundamental concepts such as streaming data, batch and stream processing, streaming architectures, and the complete streaming-data lifecycle. It provides an overview of important technologies including **Apache Kafka, Apache Storm, Apache Spark, and Apache Flink**, along with their architectures, processing models, scalability, reliability, and fault-tolerance mechanisms.

The book also addresses practical aspects such as **data cleaning, aggregation, real-time analytics, dashboards, monitoring, alerting, statistical analysis, machine learning, anomaly detection, forecasting, and data storytelling**. Applications in **fraud detection, recommendation systems, IoT, healthcare, transportation, cybersecurity, predictive maintenance, and financial analytics** demonstrate the relevance of streaming technologies in real-world environments.

To encourage hands-on learning, the book includes **laboratory experiments, tool-wise assignments, industry case studies, mini projects, report guidelines, assessment rubrics, and viva questions**. These components help learners connect theoretical concepts with practical implementation and develop problem-solving skills.

Overall, the book aims to provide a clear pathway from **data generation and ingestion to stream processing, transformation, real-time analysis, visualization, and decision-making**, making it a useful resource for students, educators, and learners interested in modern data streaming and real-time analytics.

AUTHORS

Dr. P. Renukadevi is an Associate Professor in the Department of Computer Science and Engineering (AIML) at Jain (Deemed-to-be University), Karnataka, India. She holds a Ph.D. in Computer Science and Engineering from Anna University, Chennai, and has over 20 years of teaching experience. Her research interests include Data Analytics, with around 20 research publications in reputed journals and conferences. She has received research funding from ICMR and Jain University for research projects. She is an active member of IEEE and ISTE.



Dr. Chinmaya Kumar Swain is currently an Assistant Professor of IT Systems and Analytics at the Indian Institute of Management (IIM) Jammu. He holds an MCA from Utkal University and an M.Tech. in Computer Science and Engineering from IIT Bombay, and earned his Ph.D. in Cloud Computing from IIT Guwahati. He has academic experience at IMIT Cuttack, SRM University Andhra Pradesh, and SOA University. His research interests include Cloud, Fog and Edge Computing, Natural Language Processing, and Machine Learning. He has authored over 30 research papers in reputed international journals and conferences and has mentored numerous undergraduate and postgraduate students.



Dr. Archana Sasi is an Assistant Professor in the Department of Computer Science and Engineering at Jain (Deemed-to-be University), Bengaluru. She holds a Ph.D. in Computer Science and Engineering from Presidency University, Bengaluru, along with B.Tech. and M.Tech. degrees from the College of Engineering Cherthala, Kerala. With over 9 years of teaching and academic experience, she has published research papers in SCI, SCIE, and Scopus-indexed journals and conferences. She has received two Best Paper Awards at international conferences and is an AWS Certified Solutions Architect – Associate. Her research interests include Supply Chain Management, Machine Learning, and Internet of Things (IoT).



Mr. Shahad P. is an Assistant Professor in the Department of Artificial Intelligence and Machine Learning at Jain (Deemed-to-be University), Bengaluru, with over 17 years of teaching and academic experience. He holds an M.Tech. in Computer Science and Engineering from MEA Engineering College, Kerala, and has submitted his Ph.D. thesis at IIIT Kottayam. His research interests include Streaming Data Analysis, Machine Learning, and Responsible AI, with publications in reputed SCI/Scopus-indexed journals and conferences. He has also received two Best Paper Awards.



ACKNOWLEDGEMENT

We express our sincere gratitude to all those who contributed directly and indirectly to the completion of this book, **Data Streaming and Analysis**.

We are grateful to our teachers, mentors, colleagues, and academic institutions for their valuable guidance, encouragement, and continuous support throughout the preparation of this work. Their suggestions and insights have greatly helped in organizing the concepts and presenting the subject in a clear and systematic manner.

We extend our appreciation to the researchers, developers, and open-source communities whose contributions to technologies such as **Apache Kafka, Apache Storm, Apache Spark, and Apache Flink** have enriched the field of real-time data streaming and provided valuable foundations for this book.

We also thank our students, whose questions, discussions, and practical experiences inspired us to include application-oriented examples, case studies, laboratory activities, assignments, and mini projects.

Our heartfelt thanks go to our families and friends for their patience, encouragement, and unwavering support during the preparation of this book. Finally, we express our sincere appreciation to **Lurnexa Publications** for their support and cooperation in bringing this book to publication.

Above all, we are thankful to everyone who contributed to this work in any form and helped make this book a useful resource for learning and understanding **data streaming, real-time processing, analytics, and visualization**.

- AUTHOR'S

TABLE OF CONTENTS

CHAPTER 1: Data Streaming and Analysis	1
1.0 Introduction.....	2
1.1 Definition of Data Streaming and Analysis	2
1.1.1 What Is Data Streaming?.....	2
1.1.2 What Is Data Analysis in a Streaming Context?.....	2
1.1.3 Batch Processing versus Stream Processing	3
1.2 Key Concepts of Data Streaming and Analysis	4
1.2.1 Bounded and Unbounded Data.....	4
1.2.2 How Modern Systems Handle Both.....	5
1.2.3 Core Characteristics of Stream Processing.....	5
1.2.4 The Stream-Processing Workflow.....	6
1.2.5 An End-to-End View	9
1.3 Use Cases and Applications in Various Domains.....	10
1.3.1 Domain-wise Use Cases.....	10
1.4 Real-World Examples	11
1.4.1 Netflix and Amazon.....	11
1.4.2 Twitter (X) and Tesla	12
1.4.3 Hospitals — Real-Time Patient Monitoring.....	12
1.5 Potential Benefits of Data Streaming and Analysis	13
1.6 Tools and Technologies	13
1.6.1 Key Tools at a Glance.....	14
1.7 Comparison of Tools and Technologies	15
1.8 Strengths and Limitations of Tools and Technologies.....	17
1.9 Module Summary	17
1.10 Key Terms	18
1.11 Review Questions	19
Short Answer	19
Essay / Discussion.....	19
1.12 References	19

CHAPTER 2: Streaming Data Processing	20
2.1 Introduction to Streaming Data Processing	20
2.1.1 Batch Processing vs. Streaming Processing.....	20
2.1.2 Principles of Streaming Data Processing	21
2.1.3 Key Challenges in Streaming Data Processing.....	22
2.2 Streaming System Architecture and Distributed Foundations	23
2.2.1 Role of Distributed Systems in Streaming Data Processing	23
2.2.2 Stream Ingestion and Event Processing — Core Concepts	24
2.2.3 Generic Architecture of a Streaming Pipeline	24
2.3 Apache Kafka: Architecture and Working.....	25
2.3.1 Introduction to Apache Kafka.....	26
2.3.2 Kafka Architecture.....	26
2.3.3 Topics and Partitions	27
2.3.4 Producers	28
2.3.5 Consumers and Consumer Groups	28
2.3.6 Brokers and the Kafka Cluster	29
2.3.7 Offsets and Message Ordering	30
2.3.8 Kafka Data Flow.....	30
2.3.9 Reliability and Fault Tolerance in Kafka	31
2.3.10 Kafka Working Mechanism — Step-by-Step Operational Flow	31
2.4 Apache Storm: Architecture and Working.....	32
2.4.1 Introduction to Apache Storm	32
2.4.2 Storm Architecture.....	34
2.4.3 Topologies, Spouts, and Bolts	36
2.4.4 Stream Grouping Strategies	38
2.4.5 Storm Data Flow.....	40
2.4.6 Reliability and Fault Tolerance in Storm	40
2.4.7 Storm Working Mechanism — Step-by-Step Operational Flow.....	42
2.4.8 Common Design Considerations for Storm Topologies	43
2.4.9 Storm Processing Guarantees at a Glance.....	44
2.4.10 Section 2.4 — Key Terminology Recap	45
2.5 Comparative Analysis: Apache Kafka vs. Apache Storm	46

2.5.1 Core Comparison.....	46
2.5.2 Programming Model Comparison	47
2.5.3 Common Misconceptions	48
2.5.4 Complementary Roles in a Streaming Architecture.....	48
2.5.5 When to Use Kafka Alone, Storm Alone, or Both Together	49
2.5.6 Fault-Tolerance Mechanisms Compared	50
2.5.7 Operational Considerations	51
2.5.8 A Practical Decision Checklist.....	52
2.5.9 Summary of the Comparison	52
2.6 Handling and Managing Real-Time Data Streams	53
2.6.1 Real-Time vs. Micro-Batch Processing.....	53
2.6.2 Event Time vs. Processing Time Semantics.....	54
2.6.3 Introduction to Windowing in Stream Processing.....	55
2.6.4 Introduction to State Management in Streaming Systems	56
2.6.5 Ensuring Data Ordering and Consistency in Streams.....	57
2.6.6 Section 2.6 — Key Terminology Recap	59
2.7 Building Scalable and Fault-Tolerant Streaming Pipelines.....	60
2.7.1 Principles of Scalability in Streaming Systems.....	60
2.7.2 Partitioning and Load-Balancing Techniques	61
2.7.3 Replication and Fault-Tolerance Techniques.....	62
2.7.4 Checkpointing and Recovery Mechanisms	62
2.7.5 Illustrative Scenario: Designing an End-to-End Scalable Pipeline.....	63
2.7.6 Section 2.7 — Key Terminology Recap	65
2.8 Real-World Domains for Streaming Applications.....	65
2.8.1 IoT and Sensor Data Streaming	65
2.8.2 Banking and Financial Fraud Detection.....	65
2.8.3 Social Media Analytics	66
2.8.4 Transportation and Logistics.....	66
2.8.5 Healthcare Monitoring.....	66
2.8.6 Cybersecurity (Intrusion and Anomaly Detection).....	66
2.9 Case Studies on Real-World Data Streaming Applications	66
2.9.1 Case Study: Kafka-Based Log Aggregation in an E-Commerce Platform....	67

2.9.2 Case Study: Real-Time IoT Sensor Pipeline Using Kafka and Storm	67
2.9.3 Case Study: Smart City Traffic Monitoring Using Streaming Data	67
2.9.4 Summary Table of Domains and Case Studies.....	67
2.9.5 Common Patterns Across Domains and Case Studies.....	68
2.10 Unit Summary	69
2.11 Key Terms.....	69
2.12 Review Questions	70
2.13 Multiple Choice Questions	70
2.14 Application/Problem-Solving Questions	71
2.15 Practical/Laboratory Activities	72
2.16 References	72
CHAPTER 3: Data Transformation and Analysis	73
3.1 Introduction to Data Transformation and Analysis.....	73
3.1.1 Concept and Need for Data Transformation in Streaming Systems	73
3.1.2 Principles of Data Transformation and Analysis.....	73
3.1.3 From Streaming Ingestion to Real-Time Analytics	74
3.2 Apache Spark: Architecture and Working.....	75
3.2.1 Introduction to Apache Spark.....	75
3.2.2 Spark Architecture	77
3.2.3 Resilient Distributed Datasets (RDDs) and DataFrames	78
3.2.4 Spark Streaming and Structured Streaming	79
3.2.5 Spark Data Flow	80
3.2.6 Fault Tolerance in Spark	81
3.2.7 Spark Working Mechanism — Step-by-Step Operational Flow	82
3.3 Apache Flink: Architecture and Working.....	84
3.3.1 Introduction to Apache Flink.....	84
3.3.2 Flink Architecture.....	86
3.3.3 Flink Streaming Dataflow Model	87
3.3.4 Windowing in Flink.....	88
3.3.5 State Management in Flink.....	90
3.3.6 Fault Tolerance — Checkpointing and Savepoints in Flink.....	91
3.3.7 Flink Working Mechanism — Step-by-Step Operational Flow	91

Flink Performance and Scalability Considerations	93
3.4 Comparative Analysis: Apache Spark vs. Apache Flink.....	93
Common Pitfalls When Choosing Between Spark and Flink	96
A Practical Decision Checklist.....	96
3.5 Data Cleaning Techniques for Streaming Data.....	97
3.5.1 Handling Missing and Incomplete Data	97
3.5.2 Handling Duplicate and Noisy Data.....	98
3.5.3 Data Validation and Consistency Checks	99
3.6 Data Aggregation and Real-Time Analysis Techniques.....	100
3.6.1 Aggregation Concepts over Streaming Windows.....	100
3.6.2 Real-Time Analytics Techniques	101
3.6.3 Illustrative Scenario: Aggregating Streaming Sensor Data	102
3.7 Real-Time Dashboard Development and Visualization.....	103
3.7.1 Principles of Real-Time Dashboard Design	103
3.7.2 Techniques for Developing Real-Time Dashboards.....	105
3.7.3 Visualization Techniques for Streaming Data	105
3.7.4 Overview of Real-Time Visualization Tools.....	106
3.8 Challenges and Solutions in Data Transformation and Analysis.....	107
3.8.1 Latency vs. Accuracy Trade-Offs.....	107
3.8.2 Handling Late and Out-of-Order Data	107
3.8.3 Scalability Challenges in Analytics Pipelines.....	108
3.8.4 Data Quality and Governance Challenges	109
3.9 Real-World Applications and Case Studies.....	110
3.9.1 Case Study: Real-Time Fraud Detection Dashboard in Banking (Spark) ..	110
3.9.2 Case Study: Flink-Based Real-Time Recommendation Analytics in E-Commerce.....	111
3.9.3 Case Study: Industrial Monitoring Dashboard for Predictive Maintenance	111
3.9.4 Case Study: Real-Time Patient Monitoring Analytics in Healthcare.....	112
Summary Table of Unit 3 Case Studies.....	112
Common Patterns Across the Unit 3 Case Studies	112
3.10 Unit Summary	113
3.11 Key Terms.....	114

3.12 Review Questions	115
3.13 Multiple Choice Questions	116
3.14 Application/Problem-Solving Questions	117
3.15 Practical/Laboratory Activities	118
3.16 References	118
CHAPTER 4: Real-Time Data Processing	119
4.1 Introduction to Real-Time Data Processing	119
4.1.1 What is Real-Time Data Processing?	119
4.1.2 Key Characteristics.....	119
4.1.3 A Typical Real-Time Pipeline	120
4.1.4 Why Real-Time Processing Matters	120
4.2 Real-Time Data Processing vs Batch Processing.....	121
4.2.1 Batch Processing.....	121
4.2.2 Stream (Real-Time) Processing.....	121
4.2.3 Comparative Summary	121
4.2.4 Choosing Between Batch and Streaming	121
4.3 Real-Time Data Processing Tools and Frameworks	122
4.3.1 Message Brokers and Ingestion Systems	122
4.3.2 Stream-Processing Engines.....	122
4.3.3 Serving, Storage and Visualisation	122
4.4 Real-Time Data Processing with Apache Kafka	123
4.4.1 Core Concepts.....	123
4.4.2 Kafka Architecture.....	123
4.4.3 Setting Up a Topic.....	123
4.4.4 A Simple Kafka Producer (Java).....	124
4.4.5 A Simple Kafka Consumer (Java)	124
4.4.6 Delivery Guarantees	124
4.5 Processing Real-Time Data Streams with Apache Flink	125
4.5.1 Why Flink?	125
4.5.2 Core Architectural Concepts	125
4.5.3 Event Time, Processing Time, and Windows.....	126
4.5.4 A DataStream API Example (Java)	126

4.5.5 Fault Tolerance: Checkpoints and Savepoints.....	127
4.6 Implementing Real-Time Data Processing Use Cases	127
4.6.1 A General Design Process.....	127
4.6.2 Case Study 1 – Real-Time Fraud Detection in Banking.....	128
4.6.3 Case Study 2 – Real-Time Recommendation Engine	128
4.6.4 Common Implementation Pitfalls.....	128
4.7 Real-Time Analytics for Streaming Data.....	129
4.7.1 Common Analytical Patterns	129
4.7.2 Architecture for Real-Time Dashboards	129
4.8 Real-Time Decision Making with Streaming Data	129
4.8.1 Complex Event Processing (CEP)	129
4.8.2 Rule Engines and Streaming ML.....	130
4.8.3 Human-in-the-Loop vs Fully Automated Decisions	130
4.9 Real-Time Monitoring and Alerting.....	130
4.9.1 What to Monitor	130
4.9.2 Monitoring Architecture	130
4.9.3 Designing Effective Alerts	131
4.10 Unit Summary	131
4.11 Key Terms.....	132
4.12 Review Questions	132
4.13 References	133
CHAPTER 5: Data Visualization and Analytics.....	134
5.1 Introduction to Data Visualization and Analytics	134
5.1.1 What Is Data Visualization?	134
5.1.2 Why Data Visualization and Analytics Matter	134
5.1.3 The Data-to-Decision Pipeline	135
5.1.4 Descriptive, Diagnostic, Predictive, and Prescriptive Analytics	136
5.1.5 Types of Data and Their Visual Implications.....	136
5.1.6 Analytics Maturity and the Rise of Real-Time Visualization	137
5.2 Types of Data Visualization Tools	137
5.2.1 The Business Intelligence and Visualization Tool Landscape.....	137
5.2.2 Tableau.....	138

5.2.3 Microsoft Power BI.....	138
5.2.4 Other Prominent Platforms.....	139
5.2.5 Open-Source and Programmatic Visualization	139
5.2.6 Spreadsheet Tools	140
5.2.7 Choosing a Tool: A Framework.....	140
5.2.8 Chapter Summary	141
5.3 Understanding the Visualization and Analysis of Data Streaming	141
5.3.1 Defining Streaming Data	141
5.3.2 Batch versus Stream Processing.....	141
5.3.3 Core Concepts in Stream Processing	142
5.3.4 Streaming Platforms and Architecture Patterns	143
5.3.5 Two Dominant Processing Paradigms: Lambda and Kappa.....	144
5.3.6 Characteristics That Distinguish Streaming Visualization	144
5.4 Data Visualization for Real-Time Data Streaming	144
5.4.1 What Makes Real-Time Visualization Different	144
5.4.2 Latency Budgets and the Meaning of "Real Time"	145
5.4.3 Chart Types Suited to Streaming Data	146
5.4.4 Design Techniques for Continuously Updating Displays.....	146
5.4.5 Interactivity in Real-Time Dashboards	147
5.4.6 Common Pitfalls in Real-Time Visualization	148
5.4.7 Chapter Summary	148
5.5 Best Practices for Data Visualization and Analysis	148
5.5.1 Why Best Practices Matter	148
5.5.2 Know Your Audience.....	148
5.5.3 Choose the Right Chart for the Data and the Message	149
5.5.4 Minimize Chartjunk and Maximize the Data-Ink Ratio.....	149
5.5.5 Use Color with Purpose	150
5.5.6 Provide Context.....	150
5.5.7 Ensure Accuracy and Avoid Distortion	150
5.5.8 Design for the Delivery Context.....	151
5.5.9 Validate and Iterate.....	151
5.6 Implementing Real-Time Data Visualization and Analytics Use Cases	151

5.6.1 From Principles to Practice.....	151
5.6.2 Case Study: Real-Time Equipment Monitoring and Predictive Maintenance	152
5.6.3 Case Study: Financial Market Surveillance.....	153
5.6.4 Case Study: Application and Infrastructure Monitoring (Observability).....	153
5.6.5 Case Study: Logistics and Fleet Tracking.....	154
5.6.6 Cross-Cutting Implementation Lessons	154
5.7 Exploratory Data Analysis for Streaming Data	155
5.7.1 What Exploratory Data Analysis Means in a Streaming Context	155
5.7.2 Windowed Exploration	155
5.7.3 Distribution and Shape Checks.....	156
5.7.4 Outlier and Anomaly Detection in EDA.....	157
5.7.5 Correlation and Relationship Discovery Across Streams.....	157
5.7.6 Seasonality and Trend Decomposition	157
5.7.7 Visual Techniques for Streaming EDA	158
5.8 Statistical Analysis for Streaming Data	158
5.8.1 The Role of Statistics in Streaming Analytics	158
5.8.2 Statistical Process Control	159
5.8.3 Hypothesis Testing on Streaming Data.....	160
5.8.4 Change Point and Drift Detection.....	160
5.8.5 Time Series Models for Streaming Forecasting.....	161
5.8.6 Non-Stationarity: The Central Statistical Challenge of Streaming Data ...	161
5.9 Machine Learning for Streaming Data.....	162
5.9.1 Why Streaming Data Requires a Distinct Machine Learning Approach ...	162
5.9.2 Online (Incremental) Learning Algorithms	162
5.9.3 Streaming Anomaly Detection with Machine Learning.....	163
5.9.4 Concept Drift: Definition and Detection.....	163
5.9.5 The Online Machine Learning Pipeline	164
5.9.6 Visualizing Machine Learning Output in a Streaming Context	165
5.9.7 Practical Considerations and Limitations.....	165
5.10 Data Storytelling with Real-Time Data Visualizations	166
5.10.1 What Is Data Storytelling?	166

5.10.2 Why Storytelling Matters More, Not Less, for Real-Time Data	166
5.10.3 The Narrative Arc Applied to Streaming Data.....	166
5.10.4 Techniques for Real-Time Data Storytelling	167
5.10.5 Balancing Narrative and Rigor	168
5.10.6 Storytelling for Different Real-Time Audiences	168
5.10.7 Bringing It Together: Storytelling as the Capstone of Visual Analytics..	169
5.11 Unit Summary	169
5.12 Key Terms	170
5.13 Review Questions	170
Appendix: Laboratory Experiments, Assignments, Case Studies and Mini Projects	170
A.0 About This Appendix	171
A.1 The Running Scenario: CityPulse	171
A.2 Laboratory Setup.....	172
Section A – Laboratory Experiments	173
Section B – Tool-Wise Assignments	174
Section C – Industry Case Studies	176
Section D – Mini Projects	178
Section E – Report Format, Rubrics and Viva Bank.....	179
INDEX.....	181

CHAPTER I

INTRODUCTION TO DATA STREAMING AND ANALYSIS

Learning Objectives

- By the end of this module, you should be able to:
- Define data streaming and explain how it differs from traditional batch processing.
 - Describe the core techniques used to analyse data while it is still in motion.
 - Differentiate between bounded (finite) and unbounded (infinite) data, and identify the tools suited to each.
 - Explain the stages of a stream-processing pipeline, from data generation to output.
 - Identify real-world use cases and applications of streaming analytics across industries.
 - Compare popular streaming tools and technologies, and reason about their strengths and limitations.

1.0 Introduction

Imagine standing beside a river instead of staring at a photograph of it. A photograph however detailed captures a single instant that has already passed. A river, on the other hand, is alive: water is always arriving, always moving, always telling you something new about what is happening upstream right now. Traditional data processing has long resembled the photograph we collect information, store it, and then, at some later and often inconvenient moment, we analyse it. Data streaming asks a more urgent question: what if we could understand the river while it flows, rather than waiting for it to become a lake?

This shift from analysing data after the fact to analysing data as it is created is one of the most consequential changes in modern computing. It powers the fraud alert that stops a stolen credit card with milliseconds, the recommendation that appears the instant you finish a video, and the dashboard that warns a hospital nurse of a patient's deteriorating vital signs before a human eye would notice. This module lays the conceptual foundation for everything that follows in this course: what streaming means, how it differs from batch processing, the vocabulary you will use

throughout (bounded/unbounded data, windowing, event time, state), and the tools that make it all possible.

1.1 Definition of Data Streaming and Analysis

1.1.1 What Is Data Streaming?

Data streaming is the continuous, high-speed transmission and processing of data as it is generated, rather than after it has been collected and stored. It allows data to be examined, acted upon, or archived almost the instant it is produced. This stands in direct contrast to batch processing, in which data is gathered over a period of time an hour, a day, a month and then processed together as one large, static chunk.

The key distinction is motion versus rest. Batch systems analyse data at rest: complete files sitting quietly on disk. Streaming systems analyse data in motion: events flowing continuously through a pipeline, with no guarantee that the flow will ever stop. This single difference cascades into everything else you will learn in this module how systems are architected, what guarantees they can offer, and what trade-offs a designer must accept.

In Short

Data streaming is the continuous, real-time ingestion of data from one or more sources, processed as it arrives enabling insight and action with minimal delay between an event happening and a system responding to it.

1.1.2 What Is Data Analysis in a Streaming Context?

If streaming is the pipe through which data flows, analysis is what we do with the water as it passes. Streaming analysis is the application of techniques that extract meaningful insight from a continuous, never-ending flow of records not from a dataset you can scroll to the end of. Four techniques recur constantly:

- **Filtering:** discarding irrelevant, malformed, or noisy records before they consume downstream processing resources. Think of it as a bouncer at the door of your pipeline.
- **Aggregation:** summarising many events into a single meaningful number counts, sums, averages, minimums, maximums typically computed over a window of time.
- **Transformation:** reshaping, enriching, or reformatting a record: converting currencies, joining a sensor reading with the machine's metadata, or normalising a timestamp format.

- **Machine-learning inference:** applying an already-trained model to each incoming event for prediction, classification, or anomaly detection for example, scoring a transaction for fraud risk in real time.

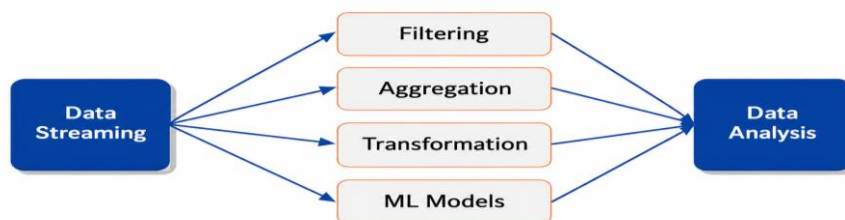


Figure 1.1: Illustration accompanying the discussion of “1.1.2 What Is Data Analysis in a Streaming Context?”

1.1.3 Batch Processing versus Stream Processing

It is tempting to think of stream processing simply as “batch processing, but faster.” That framing understates how different the two paradigms really are. Batch processing operates on a bounded, known quantity of data and can therefore make comfortable assumptions it knows when the data ends, and it can look at the whole dataset before producing an answer. Stream processing operates on an unbounded quantity of data and must produce useful answers continuously, without ever seeing the “whole” dataset, because there is no such thing. Table 1.1 lays out the practical implications of this difference.

Table 1.1: Aspect; Batch Processing; Stream Processing

Aspect	Batch Processing	Stream Processing
Data handling	Collected and processed in discrete chunks	Processed continuously, record by record, as it arrives
Latency	Higher minutes to hours	Low milliseconds to seconds (near real-time)
Data scope	Bounded / finite datasets	Unbounded / infinite streams
Best suited for	Historical analysis, reporting, large-scale training	Immediate insight and time-sensitive action
Throughput pattern	High volume, delivered per batch	Continuous, event-by-event
Implementation complexity	Comparatively simple	Higher requires windowing, state, and fault tolerance
Representative tools	Hadoop MapReduce, Apache Spark (batch mode)	Apache Kafka, Apache Flink, Spark Structured Streaming

Neither paradigm is universally "better" they answer different questions. A retailer computing last quarter's tax filings needs batch processing: the data is bounded, accuracy matters more than speed, and the deadline is measured in weeks. The same retailer detecting a fraudulent purchase needs stream processing: the data is unbounded, and a correct answer delivered five minutes too late is worthless. A mature data platform typically uses both, side by side.

1.2 Key Concepts of Data Streaming and Analysis

1.2.1 Bounded and Unbounded Data

Before choosing a tool or an architecture, every streaming system must answer one foundational question: does this data have an end? The answer places it into one of two categories.

Bounded Data (Finite Data)

Bounded data has a defined beginning and a defined end. Its size is known, even if large a CSV export, a database table, last month's complete sales log. Because the system can see the entire dataset, it can process it in batch mode and simply stop once every record has been consumed. Typical examples include a monthly sales report, a static machine-learning training dataset, or a full database backup. These are traditionally handled by tools such as Hadoop MapReduce or Apache Spark running in batch mode.

Unbounded Data (Infinite Data)

Unbounded data has no predefined end. New records keep arriving indefinitely, and a processing system can never say "I have seen it all." Live IoT sensor feeds, stock-market ticker updates, website clickstreams, and social-media post feeds are all unbounded. Because such data never "finishes," it demands real-time or incremental processing and two special techniques you will meet often in this course:

- **Windowing** — artificially slicing an infinite stream into finite, digestible chunks, such as "the last five minutes of transactions," so that familiar aggregation techniques can still be applied.
- **Event-time processing** — correctly handling data that arrives late or out of order, which is unavoidable in any distributed, real-world system.

Table 1.2: Feature; Bounded Data; Unbounded Data

Feature	Bounded Data	Unbounded Data
Size	Finite, known in advance	Infinite, continuously growing
Processing style	Batch (or a finite stream)	Real-time / continuous streaming
End condition	Processing completes and halts	Runs indefinitely until manually stopped
Typical examples	CSV file, database table, log archive	IoT sensor feed, live transaction stream

1.2.2 How Modern Systems Handle Both

A useful mental model, popularized by modern engines such as Apache Flink and Spark Structured Streaming, is to treat all data bounded or unbounded as a stream. A bounded stream is simply one that happens to end, and can be processed much like a batch job (for instance, reading a fixed file). An unbounded stream never ends and is processed continuously using time or count windows, watermarks to tolerate late-arriving events, and check pointing for fault tolerance during long-running jobs.

Worked Example

Bounded scenario: A retail analyst studies yesterday's completed sales log to understand which products sold best. The dataset is fixed and will not change.

Unbounded scenario: A bank's fraud-detection system evaluates every card swipe the instant it happens, comparing it against a customer's recent spending pattern. The stream of swipes never stops.

1.2.3 Core Characteristics of Stream Processing

Five characteristics distinguish a genuine stream-processing system from a batch system that has simply been run more frequently. Understanding these five ideas deeply is arguably the single most important outcome of this module, because every tool discussed later in the course is, at its heart, an attempt to deliver these five properties well.

characteristics of stream processing

Real-time processing — data is processed with minimal latency, typically within milliseconds to a few seconds of being generated.

- **Continuous queries** — unlike a one-off SQL query run against a static table, a streaming query runs constantly, re-evaluating itself as new data arrives.

- **Windowing** — data is analysed within specific, bounded time (or count) windows so that meaningful trends can be captured from an otherwise endless flow.
- **Fault tolerance** — the system must survive machine failures, network partitions, and restarts without silently losing or duplicating data.
- **Scalability** — the system must absorb increasing data volume and velocity from a few events per second to hundreds of thousands by adding more processing capacity.

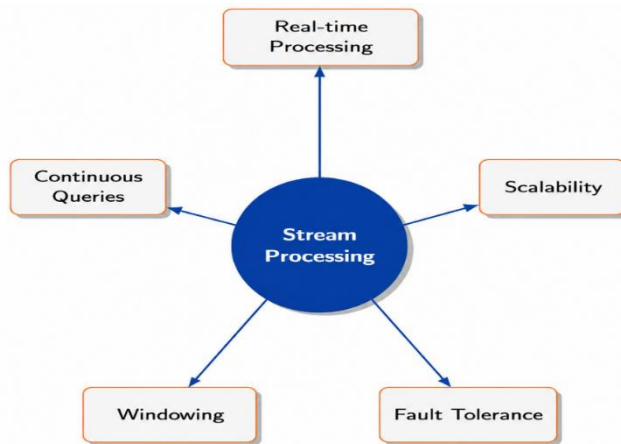


Figure 1.2: The five defining

1.2.4 The Stream-Processing Workflow

Stage 1 — Data Generation

Every stream begins with a source that produces data continuously and, often, unpredictably: sensors and IoT devices, user clicks, social-media posts, credit-card payments, or stock trades. Consider a modern connected car, whose sensors can stream road-condition data more than one hundred times per second, or the social platform X (formerly Twitter), which handles several hundred thousand posts every minute. Three properties define this stage: high velocity (data is produced around the clock), an unbounded nature (there is no predefined end, unlike a fixed CSV file), and time-sensitivity (the value of the data decays quickly a fraud signal noticed an hour late is far less useful than one noticed in a second).

Stage 2 — Data Ingestion

Ingestion is the continuous collection and import of that real-time data into a processing system. It draws from sources such as IoT devices, application logs, financial transactions, and social feeds, using ingestion tools such as Apache Kafka, Amazon Kinesis, or Azure Event Hubs, and transport protocols such as HTTP, MQTT, or Web Sockets.

A closer look at ingestion reveals six sequential steps: (1) data collection, where producers emit events; (2) protocol handling, encoding each event in a format such as JSON, Avro, or Protocol Buffers; (3) buffering, using queues to absorb sudden spikes in traffic; (4) initial processing, applying lightweight filtering or attaching metadata; (5) partitioning, splitting the stream for parallel processing while preserving ordering guarantees where required; and (6) delivery, handing the prepared stream to a downstream processing engine such as Flink or Spark.

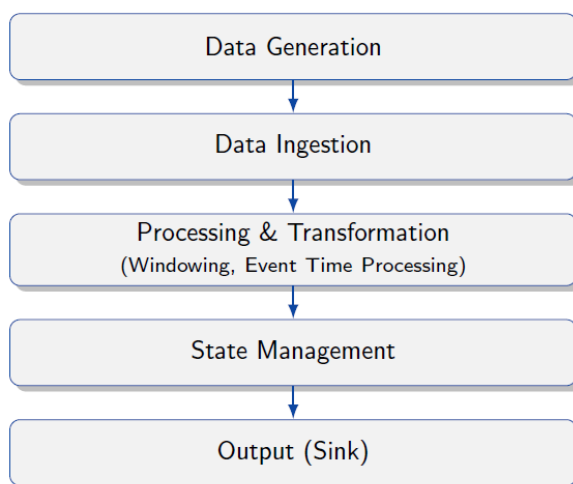


Figure 1.3: Illustration accompanying the discussion of “1.2.4 The Stream-Processing Workflow”.

Every streaming pipeline, regardless of the specific tools used to build it, passes data through the same five conceptual stages. Understanding this pipeline is essential because it is the mental map you will use to reason about any streaming architecture you encounter in industry.

Stage 3 — Processing and Transformation: Windowing

Because a stream is, by definition, infinite, no aggregation ("what is the average temperature?") can be computed over "all of it." Windowing solves this by carving the endless stream into finite, analysable chunks. Three window types are used most often:

- **Tumbling (fixed) windows** — fixed-length, non-overlapping windows for example, a fresh five-minute window that starts the instant the previous one ends. Every event belongs to exactly one window.
- **Sliding windows** — fixed-length windows that overlap, advancing by a step smaller than the window length for example, a five-minute window

recalculated every one minute. A single event can belong to several windows at once.

- **Session windows** — windows defined not by a fixed clock but by bursts of activity separated by a gap of inactivity ideal for modelling a single user "session" on a website.

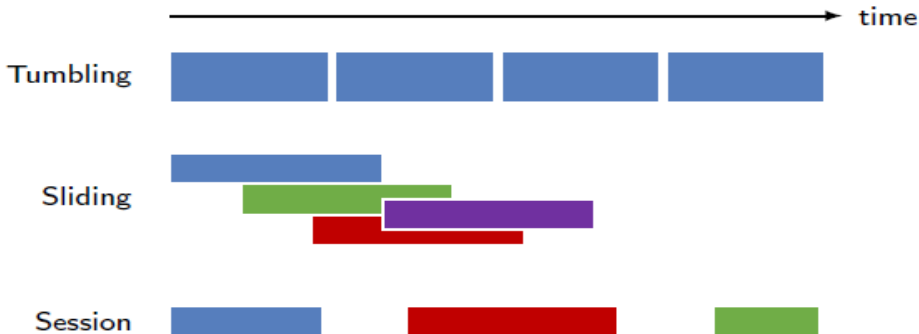


Figure 1.4: Visual comparison of tumbling, sliding, and session windows along a shared timeline.

Event-Time Processing

A subtle but critical idea in streaming is the difference between when an event happened (event time) and when the system got around to processing it (processing time). Networks are unreliable and devices go offline, so events routinely arrive late and out of order. Event-time processing uses a timestamp embedded in the data itself as an event time field rather than the system clock, so that a temperature reading generated at 9:05:44 AM is still correctly placed into the 9:05 window even if it does not reach the server until several seconds later. This guarantees correctness even in the presence of delay, at the cost of additional engineering complexity (systems must decide how long to wait for "late" data before finalising a window's result).

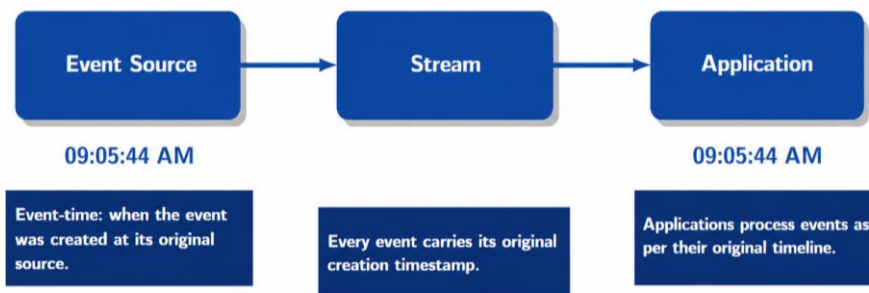


Figure 1.5: An event carries its original creation timestamp from source, through the stream, to the application, which honours that original timeline.

Stage 4 — State Management

Many useful streaming computations are not stateless. To answer "has this account made more than three purchases in the last ten minutes?" a system must remember something about the recent past it must maintain state. State is the intermediate data a streaming application keeps while processing events, and it enables sophisticated operations such as joins, running aggregations, and pattern detection across multiple events.

Challenges of Managing State

Fault tolerance — if a worker process crashes, how is its accumulated state recovered without loss?

Scalability — how is state partitioned sensibly across many machines as data volume grows?

Consistency — how are race conditions avoided when state is updated concurrently in a distributed system?

Performance — how is latency kept low even as the size of the maintained state grows large?

Stage 5 — Output (Sink)

The final stage delivers results to their destination, or sink a dashboard, an alerting system, a data lake for long-term storage, or directly into another application. A well-designed pipeline treats the sink as seriously as the source: results must be written exactly once (or with clearly understood duplicate semantics), and downstream consumers must be able to keep pace with the incoming rate of results.

1.2.5 An End-to-End View

Bringing every stage together, a typical production streaming architecture looks like this: varied data sources feed a stream message broker (most commonly Apache Kafka), which decouples producers from consumers and buffers the flow; one or more streaming engines (Kinesis, Flink, Spark Streaming, or Flume) consume from the broker to perform windowed processing, transformation, and stateful computation; results then branch in two directions into data-lake storage (such as Hadoop HDFS or Amazon S3) for later batch analysis and model training, and directly into live applications such as business-intelligence dashboards and machine-learning services for immediate action.

Figure 1.6 — End-to-End Streaming Data Architecture

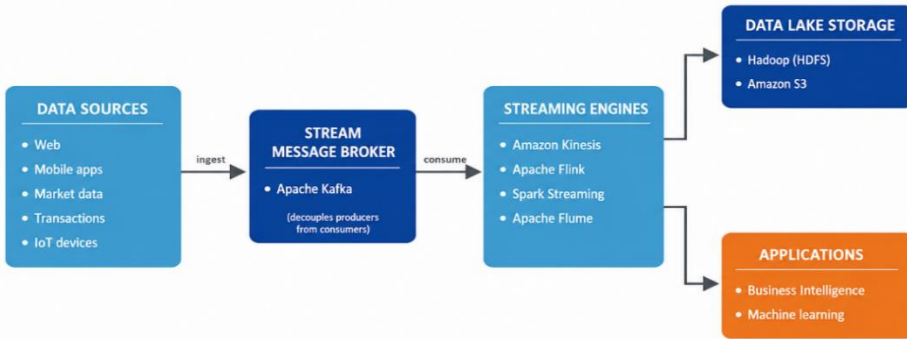


Figure 1.6: A complete streaming data flow

1.3 Use Cases and Applications in Various Domains

The abstract ideas of the previous section become concrete once we see them at work across industries. Streaming analytics has moved from a specialised technique used by a handful of technology giants to a mainstream capability expected across almost every sector.

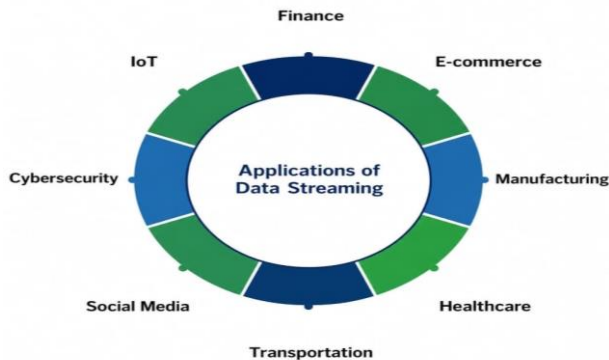


Figure 1.7: A wheel diagram summarising the domains where streaming analytics is applied.

1.3.1 Domain-wise Use Cases

- **Finance** — real-time monitoring of stock prices, fraud detection on card transactions, and algorithmic trading that reacts to market movements within microseconds.
- **E-commerce** — personalised product recommendations, live inventory management, fraud detection at checkout, and dynamic pricing that adjusts to demand.
- **Manufacturing** — predictive maintenance that flags a failing machine before it breaks down, automated quality control, and continuous process optimisation.

- **Healthcare** — remote patient monitoring, real-time clinical alerts, and early detection of disease outbreaks from aggregated health data.
- **Transportation** — live traffic monitoring, dynamic route optimisation, and the sensor fusion that underpins autonomous driving.
- **Social media** — trend and hashtag analysis, sentiment analysis, and real-time detection of breaking news events.
- **Cybersecurity** — intrusion detection, continuous threat analysis, and always-on security monitoring of network traffic.
- **Internet of Things (IoT)** — smart-home automation, industrial IoT monitoring, and coordination between connected vehicles.

1.4 Real-World Example



Figure 1.8: Real-time data analysis at work in familiar consumer platforms.

Reading about streaming in the abstract is useful, but the ideas sharpen considerably once we look at how some of the world's largest technology companies rely on them every single day.

1.4.1 Netflix and Amazon

Netflix

Netflix uses real-time data to personalise recommendations and to continuously optimise streaming video quality. It analyses viewing patterns, device information, and live network conditions to keep the viewing experience seamless, adjusting video bitrate on the fly as a user's connection quality changes.

Amazon

Amazon employs real-time data analysis for fraud detection, live inventory management, and personalised product recommendations. It continuously analyses customer behaviour, product performance, and supply-chain signals to keep its marketplace responsive at global scale.

1.4.2 Twitter (X) and Tesla

Twitter (X)

The platform analyses a torrent of real-time posts to identify trending topics, gauge public sentiment, and detect breaking news as it unfolds turning a chaotic stream of short messages into a real-time barometer of public opinion.

Tesla

Tesla streams data continuously from its vehicles to monitor performance, detect emerging mechanical issues, and improve autonomous-driving systems. It analyses sensor readings, driving behaviour, and environmental conditions from its entire fleet in near real time.

1.4.3 Hospitals Real-Time Patient Monitoring

Hospitals monitor patient vital signs continuously to detect anomalies and enable timely clinical intervention. By streaming heart rate, blood pressure, oxygen saturation, and other physiological signals, care teams can be alerted to a deteriorating patient minute sometimes hours before traditional periodic checks would have caught the change, materially improving patient outcomes.

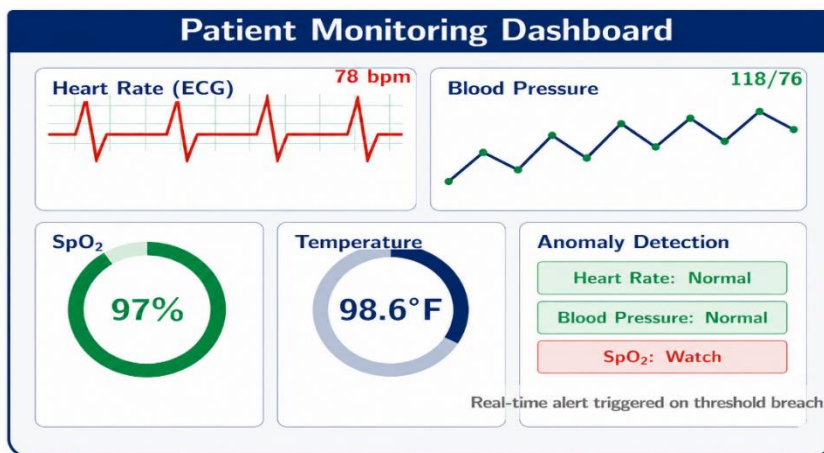


Figure 1.9: A hospital patient-monitoring dashboard displaying streaming vital-sign data with real-time alerts.

1.5 Potential Benefits of Data Streaming and Analysis

Why do organisations invest heavily in building streaming infrastructure, when batch processing is simpler and cheaper to operate? The answer lies in a cluster of benefits that compound as data volumes grow and markets move faster.

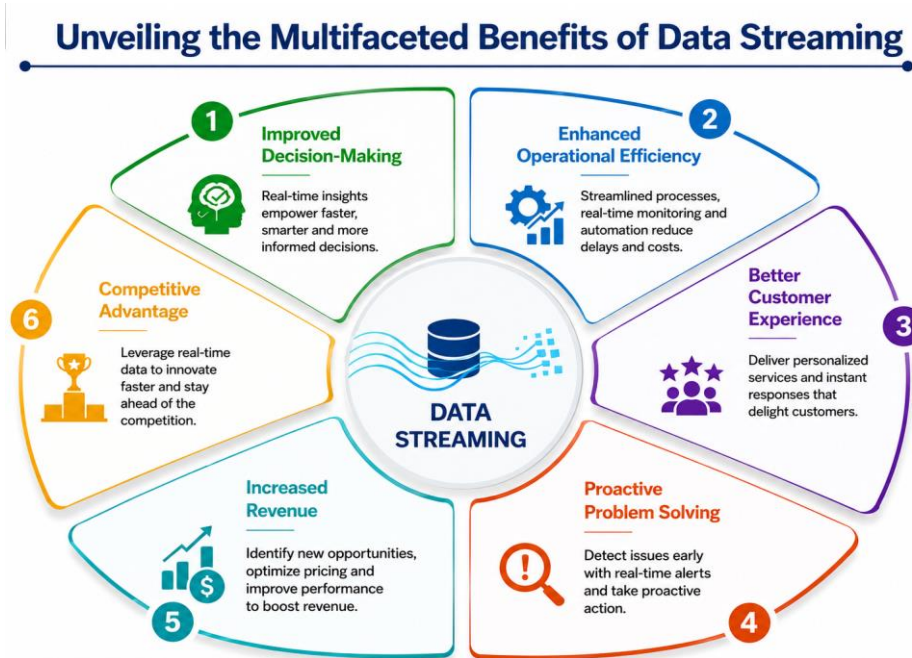


Figure 1.10: An overview of the multifaceted benefits organisations gain from adopting real-time data streaming.

- **Faster decision-making** — insight is available from data the instant it arrives, rather than after an overnight batch job completes.
- **Operational excellence** — supports real-time monitoring, automation, and continuous optimisation of resources and processes.
- **Enhanced user experience** — enables personalised services and timely, contextual responses to user behaviour.
- **Early event detection** — proactively identifies anomalies, equipment failures, and emerging trends before they escalate.
- **Business growth** — uncovers new revenue opportunities through timely, actionable analytics.
- **Strategic advantage** — allows an organisation to respond rapidly as its competitive environment changes.

1.6 Tools and Technologies

A rich ecosystem of tools has grown up around the concepts introduced in this module. Broadly, they fall into message brokers (which move data reliably from

producers to consumers), processing engines (which apply the filtering, aggregation, transformation, and windowing logic discussed in Section 1.2), and managed cloud services

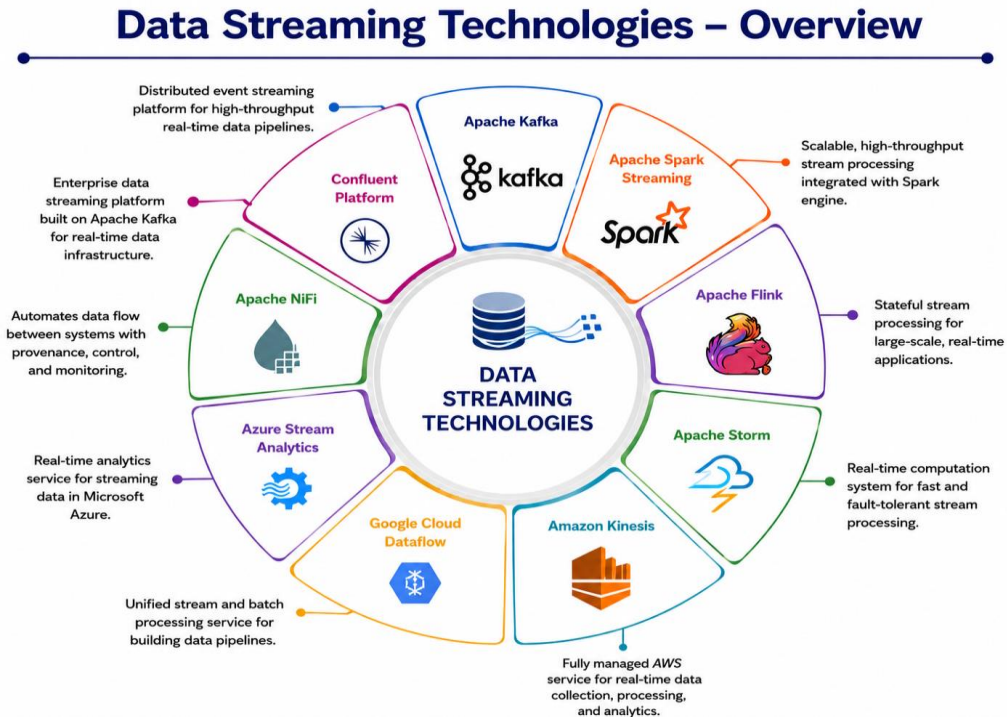


Figure 1.11: An overview map of the major data-streaming technologies and where each fits in the ecosystem.

1.6.1 Key Tools at a Glance

- **Apache Kafka** — a distributed streaming platform that acts as the durable, high-throughput backbone for real-time data pipelines.
- **Apache Spark Streaming** — an extension of the Spark ecosystem that enables (micro-batch) real-time processing alongside familiar batch APIs.
- **Apache Flink** — a true streaming-first framework built for high-performance, low-latency, fault-tolerant applications.
- **Apache Storm** — a distributed real-time computation system, one of the earliest frameworks purpose-built for unbounded streams.
- **Amazon Kinesis** — a fully managed, scalable, and durable real-time streaming service on AWS.
- **Google Cloud Dataflow** — a fully managed, unified service for both stream and batch processing, based on the Apache Beam model.
- **Azure Stream Analytics** — a real-time analytics service for streaming data within the Microsoft Azure ecosystem.

- **Apache NiFi** — an easy-to-use system for automating the flow and routing of data between systems.
- **Confluent Platform** — a complete, enterprise-grade data-streaming platform built around Apache Kafka.

Data Processing / Storage Stack

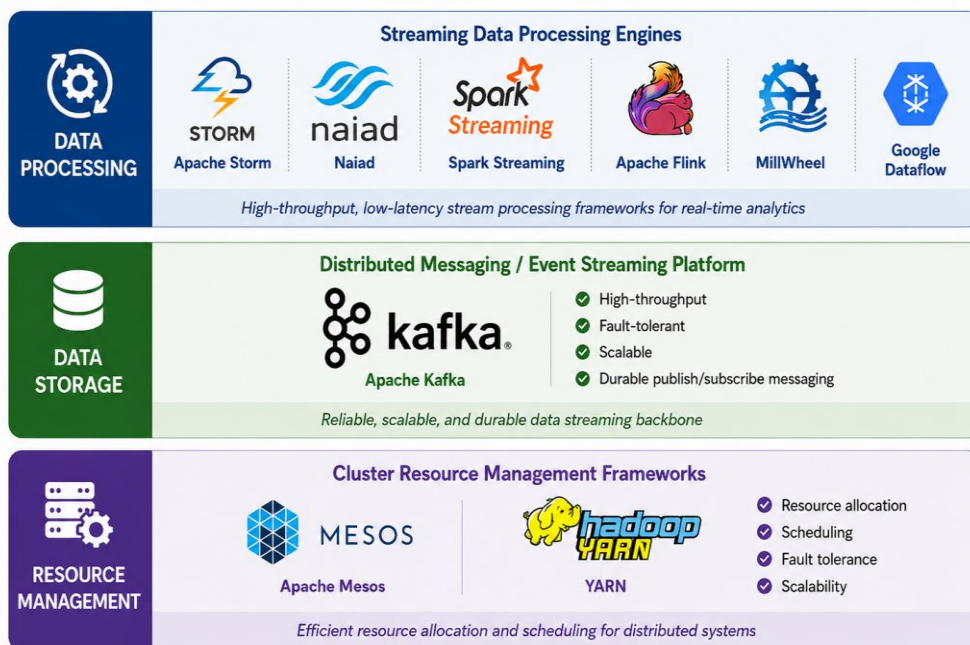


Figure 1.12: A layered view of a typical data processing and storage stack for streaming architectures.

1.7 Comparison of Tools and Technologies

No single tool is best for every situation. Table 1.2 compares the leading streaming platforms across the dimensions that matter most when architecting a real system: how they model processing, how quickly they respond, and how easy they are to operate.

Table 1.4: Feature; Kafka; Spark; Flink; Storm; Kinesis; GCP; Azure SA Streaming; Dataflow

Feature	Kafka	Spark Streaming	Flink	Storm	Kinesis	GCP Dataflow	Azure SA
Processing model	Pub-Sub	Micro-batch	True streaming	True streaming	True streaming	Stream / batch	True streaming

Feature	Kafka	Spark Streaming	Flink	Storm	Kinesis	GCP Dataflow	Azure SA
Latency	Low	Medium	Low	Low	Low	Low	Low
Scalability	High	High	High	High	High	High	High
Fault tolerance	High	High	High	High	High	High	High
Ease of use	Medium	Medium	Medium	Medium	High	High	High
Cost model	Varies	Varies	Varies	Varies	Pay-as-you-go	Pay-as-you-go	Pay-as-you-go

Two platforms are worth contrasting directly because students most often confuse them: Apache Kafka and Apache Spark Streaming. Kafka is, first and foremost, a publish-subscribe message broker it moves and durably stores streams of events but is not, by itself, a processing engine. Spark Streaming is a processing engine that consumes from a broker (often Kafka) and applies a micro-batch model, breaking the incoming stream into very small batches processed in rapid succession. Kafka therefore offers very low latency for moving data, while Spark Streaming's micro-batching introduces a small amount of additional latency in exchange for tight integration with the broader Spark ecosystem, including its mature batch and machine-learning libraries.

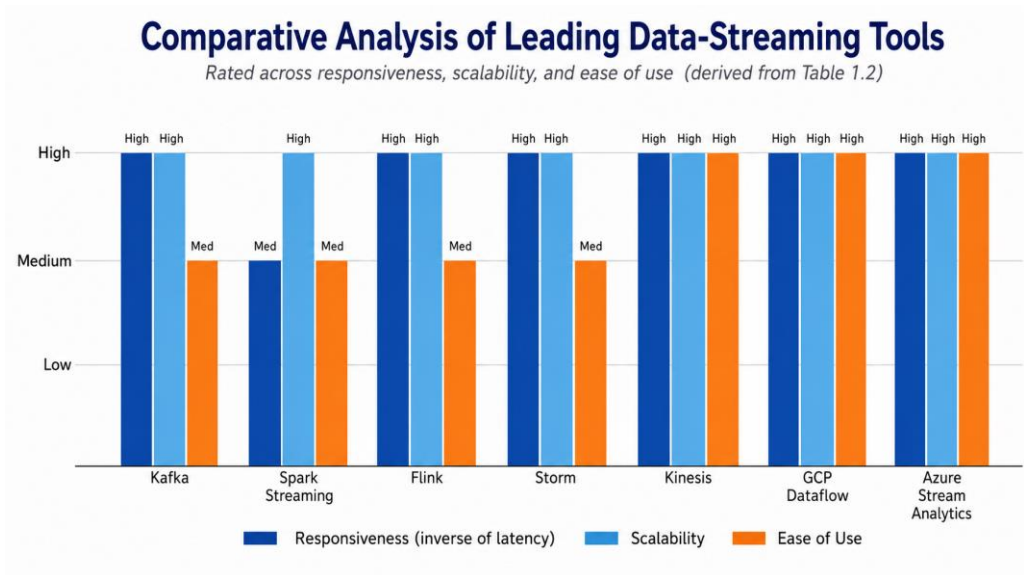


Figure 1.13: A comparative analysis chart of leading data-streaming tools across latency, scalability, and ease of use.

1.8 Strengths and Limitations of Tools and Technologies

Apache Kafka

- **Strengths:** high throughput, strong fault tolerance, proven horizontal scalability.
- **Limitations:** requires genuine operational expertise to deploy, tune, and manage at scale.

Apache Spark Streaming

- **Strengths:** easy integration with the wider Spark ecosystem; supports both batch and streaming from one API.
- **Limitations:** higher latency than true streaming frameworks, owing to its micro-batch design.

Apache Flink

- **Strengths:** genuine event-at-a-time streaming, low latency, strong fault tolerance via distributed checkpointing.
- **Limitations:** a steeper learning curve than Spark Streaming for teams new to stream processing.

Apache Storm

- **Strengths:** very low latency and highly customisable topologies.
- **Limitations:** comparatively complex to develop, tune, and maintain in production.

Managed Cloud Services (Kinesis, Dataflow, Azure Stream Analytics)

- **Strengths:** fully managed, easy to adopt quickly, and elastically scalable without operating your own cluster.
- **Limitations:** risk of vendor lock-in, and potentially higher long-run cost compared to self-managed open-source alternatives.

Choosing the Right Tool

There is rarely a single "correct" answer. The right choice depends on your latency requirements, expected data volume, required scalability, in-house operational expertise, and budget constraints. A start-up prioritising speed of delivery may prefer a managed cloud service; a large enterprise with strict latency requirements and dedicated infrastructure engineers may prefer the control offered by Kafka and Flink.

1.9 Module Summary

1. Data streaming is the continuous, real-time ingestion and processing of data, as distinct from batch processing, which operates on bounded data collected over time.

2. Data is broadly of two kinds: bounded (finite, well suited to batch processing) and unbounded (infinite, requiring genuine stream processing).
3. A streaming pipeline moves through five stages: Data Generation → Ingestion → Processing & Transformation → State Management → Output.
4. Five characteristics define genuine stream processing: real-time processing, continuous queries, windowing, fault tolerance, and scalability.
5. Windowing (tumbling, sliding, session) and event-time processing are the two central techniques for taming an otherwise infinite stream.
6. Streaming analytics now spans nearly every industry finance, e-commerce, manufacturing, healthcare, transportation, social media, cybersecurity, and IoT as demonstrated by Netflix, Amazon, Twitter, Tesla, and modern hospital systems.
7. Popular tools include Apache Kafka, Spark Streaming, Apache Flink, Apache Storm, Amazon Kinesis, Google Cloud Dataflow, and Azure Stream Analytics, each with distinct trade-offs in latency, ease of use, and cost.
8. Tool selection is ultimately a design decision balancing latency needs, scale, operational maturity, and budget not a search for one universally superior technology.

1.10 Key Terms

Table 1.5: Term; Meaning

Term	Meaning
Batch processing	Processing a bounded, previously collected set of data all at once.
Stream processing	Continuously processing unbounded data as individual events arrive.
Bounded data	A dataset with a known, finite size and a clear end.
Unbounded data	A dataset that grows continuously with no predefined end.
Windowing	Dividing an infinite stream into finite chunks for analysis (tumbling, sliding, or session).
Event time	The timestamp at which an event actually occurred at its source.
Processing time	The timestamp at which a system actually processes an event.
Watermark	A mechanism that tells a streaming system how long to wait for late-arriving events.
State	Intermediate data a streaming application retains across events to support aggregation, joins, or pattern detection.

Term	Meaning
Sink	The final destination of processed stream results a dashboard, database, or another application.

1.11 Review Questions

Short Answer

1. Define data streaming in your own words, and explain how it differs from batch processing.
2. Distinguish between bounded and unbounded data, giving one original example of each.
3. List and briefly explain the five stages of the stream-processing workflow.
4. What problem does windowing solve, and how do tumbling, sliding, and session windows differ?
5. Why is event-time processing necessary in real-world streaming systems?
6. Name three challenges involved in managing state in a distributed streaming application.

Essay / Discussion

7. Choose one industry not discussed in detail in this module (e.g., agriculture, insurance, or gaming) and design a plausible streaming-analytics use case for it, describing the data sources, processing steps, and expected benefits.
8. Compare Apache Kafka and Apache Flink in terms of their role in a streaming architecture. Are they competitors or complementary technologies? Justify your answer.
9. Argue for or against the following claim: "Every modern application should be built on streaming architecture, since real-time is always better than batch." Support your argument with the trade-offs discussed in this module.

1.12 References

- Official documentation of Apache Kafka, Apache Flink, and Apache Spark Structured Streaming.
- Tyler Akidau, Slava Chernyak, and Reuven Lax, Streaming Systems: The What, Where, When, and How of Large-Scale Data Processing (O'Reilly).
- Vendor documentation for Amazon Kinesis, Google Cloud Dataflow, and Azure Stream Analytics.

CHAPTER II

STREAMING DATA PROCESSING

2.1 Introduction to Streaming Data Processing

2.1.1 Batch Processing vs. Streaming Processing

Data processing systems can broadly be classified into two paradigms based on how they handle incoming data: batch processing and streaming (real-time) processing. Understanding the distinction between these two paradigms is essential before studying any streaming framework, because it explains why specialized streaming systems such as Apache Kafka and Apache Storm were developed in the first place.

In batch processing, data is first collected and stored over a period of time, and processing is triggered later on the accumulated dataset. A classic example is an end-of-day sales report generated by a retail company: transactions occurring throughout the day are stored in a database, and a batch job runs at midnight to compute the day's totals. Batch processing is well suited to scenarios where a delay between data generation and insight is acceptable, and where the goal is to process large, finite volumes of historical data efficiently.

In streaming processing, data is processed continuously as it arrives, typically within milliseconds to a few seconds of generation. Rather than waiting for a complete dataset, a streaming system processes each event, or a small window of events, as soon as it becomes available. This paradigm is essential for applications where the value of data diminishes rapidly with time for example, detecting a fraudulent bank transaction, monitoring the temperature of an industrial furnace, or updating a live sports score.

The table below summarizes the key distinguishing characteristics of the two paradigms.

Table 2.1: Characteristic; Batch Processing; Streaming Processing

Characteristic	Batch Processing	Streaming Processing
Data scope	Finite, bounded dataset	Continuous, unbounded dataset
Latency	Minutes to hours	Milliseconds to seconds
Trigger	Scheduled or on-demand jobs	Continuous, event-driven execution

Characteristic	Batch Processing	Streaming Processing
Typical use case	Payroll processing, monthly billing	Fraud detection, live dashboards, IoT monitoring
System examples	Hadoop MapReduce, traditional ETL jobs	Apache Kafka, Apache Storm, Apache Flink

It is worth noting that many real-world systems today adopt a hybrid approach, where streaming pipelines handle time-sensitive processing while batch pipelines periodically reprocess accumulated data for accuracy and auditing. This hybrid design is often referred to in industry as the Lambda or Kappa architecture, and it highlights that streaming processing is best understood as a complement to, rather than a wholesale replacement for, batch processing.

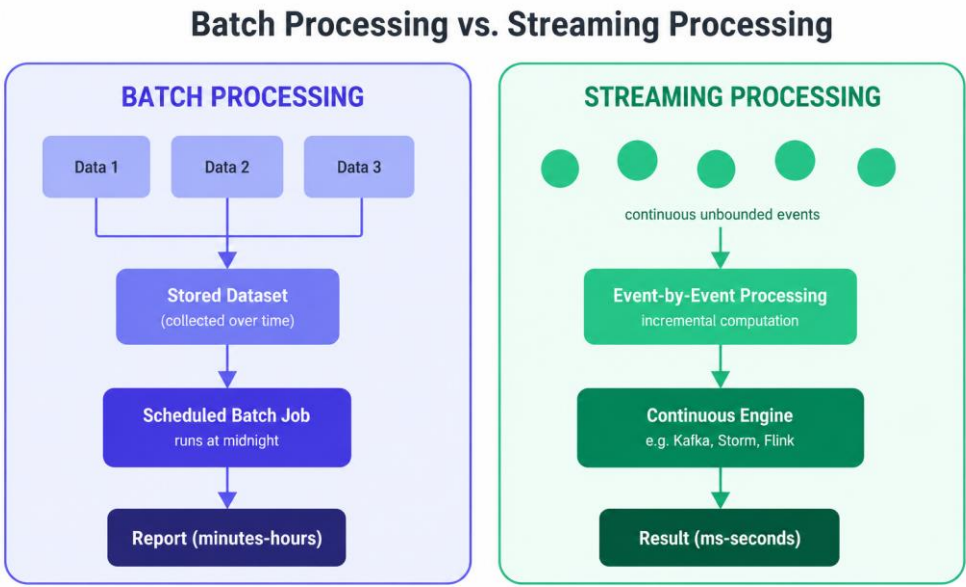


Figure 2.1: Batch processing versus streaming processing

2.1.2 Principles of Streaming Data Processing

Streaming data processing is guided by a set of foundational principles that distinguish it from conventional data processing. These principles shape the design of every streaming framework discussed later in this unit.

- **Continuous data flow:** Data is treated as an unbounded, ever-growing sequence of events rather than a fixed dataset. A streaming system must be designed to run indefinitely, processing events as they arrive without waiting for the stream to end.

- **Low-latency processing:** The primary objective of a streaming system is to minimize the time between an event occurring and the corresponding result being produced, often referred to as end-to-end latency.
- **Incremental computation:** Rather than recomputing results from scratch, streaming systems update results incrementally as new data arrives, which allows them to sustain high throughput without reprocessing the entire history of data.
- **Event-driven architecture:** Processing is triggered by the arrival of individual events or small batches of events, rather than by a fixed schedule.
- **Decoupling of producers and consumers:** Streaming systems typically separate the components that generate data (producers) from the components that consume and process it (consumers), allowing each to operate, scale, and fail independently.
- **Ordering and time-awareness:** Many streaming applications must reason about the order in which events occurred and the time at which they occurred, which introduces the related concepts of event time and processing time (discussed in Section 2.6). Together, these principles explain why streaming systems require fundamentally different architectural choices from those used in traditional batch-oriented systems, and they set the stage for the discussion of distributed system foundations in Section 2.2.

2.1.3 Key Challenges in Streaming Data Processing

While streaming processing offers clear advantages in terms of timeliness, it introduces a distinct set of engineering challenges that must be addressed by any production-grade streaming system.

- **Latency versus throughput trade-off:** Reducing the time to process each event (latency) often competes with the ability to process a large volume of events per second (throughput). Streaming systems must be tuned carefully to balance the two.
- **Ordering and out-of-order data:** Events may arrive out of sequence due to network delays or distributed data sources, making it difficult to guarantee that data is processed in the exact order in which it was generated.
- **Fault tolerance:** Because streaming systems run continuously, they must be able to recover from node failures, network partitions, or software crashes without losing data or duplicating results.
- **Scalability:** The system must be able to handle increasing data volumes and velocities by adding more processing resources, without requiring a redesign of the application.
- **State management:** Many streaming computations, such as running totals or aggregations, require the system to maintain state across events, which must be stored reliably and recovered correctly after a failure.

- **Exactly-once processing guarantees:** Ensuring that each event affects the final result exactly once, even in the presence of failures and retries, is significantly harder in a streaming context than in batch processing.
- **Backpressure:** When data producers generate events faster than consumers can process them, the system must have a mechanism to slow down ingestion or buffer data without becoming unstable.
- **Resource and cost management:** Continuous, always-on processing consumes computing resources around the clock, requiring careful capacity planning to balance performance against operational cost.

These challenges motivate the architectural patterns and distributed system concepts introduced in Section 2.2, and they reappear throughout this unit as we examine how Apache Kafka and Apache Storm are specifically engineered to address them.

2.2 Streaming System Architecture and Distributed Foundations

2.2.1 Role of Distributed Systems in Streaming Data Processing

No single machine can reliably ingest, process, and store the volume and velocity of data generated by modern applications such as ride-hailing platforms, social media feeds, or industrial sensor networks. Streaming data processing is therefore inherently a distributed systems problem: the workload is spread across many machines (nodes) that cooperate over a network to achieve throughput, reliability, and scale that a single machine cannot provide.

A distributed streaming system typically runs across a cluster a group of interconnected machines that jointly store data and execute processing tasks. Understanding a few core distributed-systems concepts is essential before studying Kafka and Storm, because both frameworks are built directly on top of these ideas.

- **Horizontal scalability:** Rather than making a single machine more powerful (vertical scaling), distributed streaming systems add more machines to the cluster to handle increased load. This allows the system to scale near-linearly as data volume grows.
- **Data partitioning:** Incoming data is divided into smaller, independent subsets (partitions or shards) that can be stored and processed in parallel across different nodes, which is fundamental to achieving high throughput.
- **Replication:** Copies of data or processing state are maintained on multiple nodes so that if one node fails, another can take over without data loss, forming the basis of fault tolerance.
- **Coordination and consensus:** Distributed nodes must agree on shared facts such as which node is currently responsible for a given partition even in the presence of failures. Coordination services such as Apache ZooKeeper are commonly used for this purpose in streaming frameworks.
- **Network communication and latency:** Because nodes are physically separate, communication between them introduces latency and the

possibility of message loss, both of which streaming systems must be explicitly designed to tolerate.

- **The CAP theorem:** Distributed systems must make trade-offs between Consistency, Availability, and Partition tolerance, since guaranteeing all three simultaneously in the presence of a network failure is provably impossible. Streaming systems typically favor availability and partition tolerance, accepting a degree of eventual consistency.

These distributed-systems principles directly explain design choices seen later in this unit for instance, why Kafka divides topics into partitions distributed across brokers, and why Storm relies on ZooKeeper for cluster coordination.

2.2.2 Stream Ingestion and Event Processing Core Concepts

Before examining any specific framework, it is useful to establish the vocabulary used to describe how data enters and moves through a streaming system.

- **Event:** The fundamental unit of data in a streaming system, representing something that happened at a point in time for example, a single sensor reading, a webpage click, or a bank transaction.
- **Stream:** An unbounded, ordered sequence of events, continuously generated by one or more sources.
- **Stream ingestion:** The process of collecting events from data sources such as applications, sensors, or databases and reliably introducing them into the streaming system for further processing.
- **Source and sink:** A source is the origin of a data stream (for example, a mobile application or an IoT device), while a sink is the final destination of processed results (for example, a dashboard, a database, or an alerting system).
- **Event processing:** The computation performed on events as they flow through the system, ranging from simple filtering and transformation to complex aggregation and pattern detection.
- **Producers and consumers:** In most streaming architectures, producers are the components that publish events into the system, while consumers are the components that read and process those events. This producer-consumer separation is a recurring architectural pattern, examined in detail for Apache Kafka in Section 2.3.

Together, ingestion and event processing form the two ends of a streaming pipeline: ingestion brings raw events into the system reliably and at scale, while event processing extracts meaning from those events in near real time.

2.2.3 Generic Architecture of a Streaming Pipeline

Although individual streaming frameworks differ in implementation, nearly all streaming pipelines can be described using a common four-stage architecture: data sources, ingestion layer, processing layer, and storage/serving layer.

- **Data sources:** Applications, IoT devices, log files, databases (via change-data-capture), or external APIs that continuously generate events.
- **Ingestion layer:** A messaging or event-streaming platform such as Apache Kafka that reliably collects events from sources, buffers them, and makes them available to downstream consumers, decoupling data producers from data consumers.
- **Processing layer:** A stream processing engine such as Apache Storm, Apache Flink, or Apache Spark that consumes events from the ingestion layer and applies transformations, aggregations, filtering, or analytics.
- **Storage and serving layer:** The destination for processed results, which may include databases, data warehouses, dashboards, or alerting systems that make the output available to end users or downstream applications.

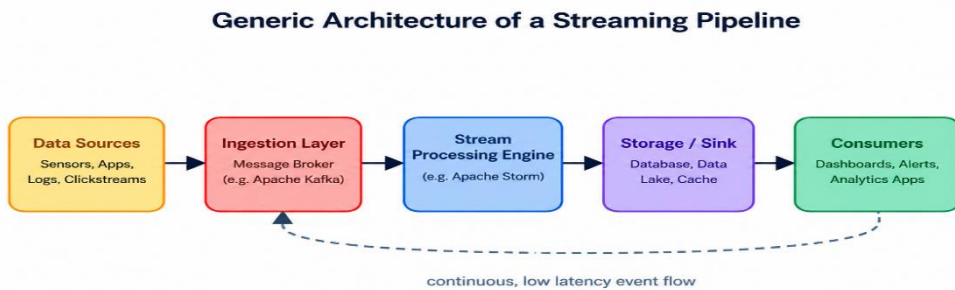


Figure 2.2: Generic architecture of a streaming pipeline

This generic architecture provides a mental model that will recur throughout this unit and the next: Apache Kafka is best understood as an ingestion-layer technology, Apache Storm as a processing-layer technology in Unit 2, and Apache Spark and Apache Flink as processing-layer technologies with additional analytics capabilities in Unit 3. Recognizing where each technology fits within this architecture makes it easier to understand why they are often used together rather than as competing alternatives.

2.3 Apache Kafka: Architecture and Working

Apache Kafka is a distributed event-streaming platform originally developed at LinkedIn and now maintained by the Apache Software Foundation. It is the most widely adopted ingestion-layer technology in modern streaming architectures, acting as a highly scalable, fault-tolerant, publish-subscribe messaging system capable of handling trillions of events per day across thousands of organizations. This section examines Kafka's architecture and internal working in detail, building the foundation needed to understand how real-time data streams are managed and processed.

2.3.1 Introduction to Apache Kafka

Kafka was designed to solve a specific problem faced by large organizations: reliably moving high volumes of continuously generated data between many independent systems, without those systems having to know about or depend directly on one another. Before Kafka, organizations typically built ad-hoc point-to-point connections between each data-producing system and each data-consuming system, which quickly became difficult to maintain as the number of systems grew.

Kafka addresses this problem by acting as a central, durable, high-throughput log through which data flows. Any system can publish (write) data to Kafka, and any authorized system can subscribe to (read) that data independently, at its own pace. This decoupling of producers and consumers is the defining characteristic of Kafka's design, and it is what allows Kafka to serve simultaneously as a messaging system, a storage system, and a stream-processing platform.

- **High throughput:** Capable of handling millions of messages per second on modest hardware, due to its efficient, sequential disk I/O design.
- **Durability:** Messages are persisted to disk and replicated across multiple brokers, so data is not lost even if individual machines fail.
- **Scalability:** Kafka clusters can be scaled horizontally by adding more brokers, with data and load automatically distributed across them.
- **Real-time and replayable:** Consumers can read data as it arrives, or rewind and re-read historical data, since Kafka retains messages for a configurable period rather than deleting them immediately after delivery.

2.3.2 Kafka Architecture

At a high level, a Kafka deployment consists of producers that publish data, a cluster of brokers that store and serve that data, and consumers that read the data. A coordination layer historically Apache ZooKeeper, and increasingly Kafka's own built-in Kraft consensus protocol in recent versions keeps track of cluster metadata such as broker membership and partition leadership.

The following subsections examine each of these architectural component's topics and partitions, producers, consumers, brokers, and offsets in detail, before tracing the complete data flow through the system in Section 2.3.8.

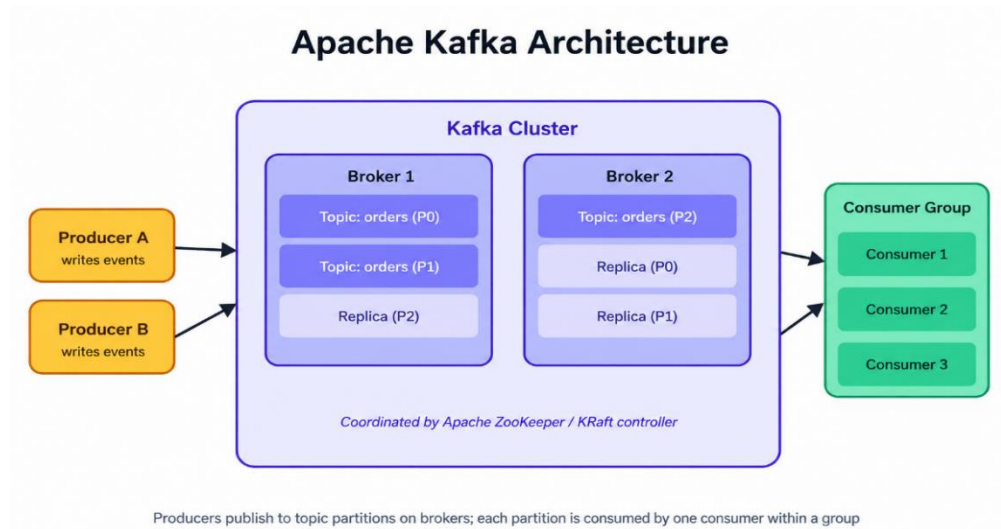


Figure 2.3: Architecture of Apache Kafka

2.3.3 Topics and Partitions

A topic is a named, logical channel to which producers publish events and from which consumers read them. Topics are the primary way data is organized in Kafka for example, an e-commerce platform might use separate topics named order-events, payment-events, and inventory-updates.

Each topic is divided into one or more partitions, which are the fundamental unit of parallelism and scalability in Kafka. A partition is an ordered, immutable sequence of messages, and each message within a partition is assigned a unique, sequentially increasing identifier called an offset. Splitting a topic into partitions allows Kafka to distribute the topic's data and processing load across multiple brokers and, correspondingly, allows multiple consumers to read from the topic in parallel.

- **Ordering guarantee:** Kafka guarantees message order only within a single partition, not across the entire topic. Applications that require strict ordering for a given entity (such as all events for one customer) typically ensure that all such events are written to the same partition.
- **Partition assignment:** Producers can specify a key for each message; Kafka uses this key to consistently route all messages with the same key to the same partition, which is how per-entity ordering is achieved.
- **Replication factor:** Each partition can be replicated across multiple brokers (discussed further in Section 2.3.9) to protect against data loss if a broker fails.

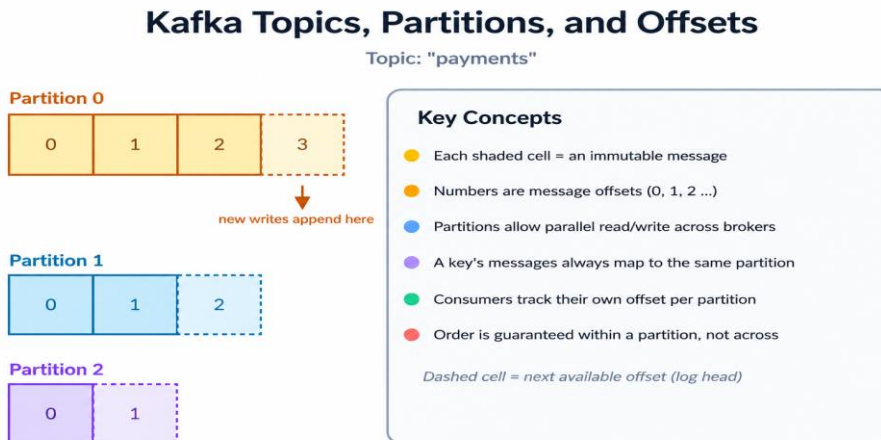


Figure 2.4: Kafka topics, partitions, and offsets

2.3.4 Producers

A producer is any client application that publishes (writes) events to one or more Kafka topics. Producers are responsible for deciding which topic, and within that topic which partition, a given message should be sent to.

- **Partitioning strategy:** If a message includes a key, the producer routes it to a partition using a deterministic hashing function; if no key is provided, messages are distributed across partitions in a round-robin fashion for load balancing.
- **Batching:** Producers group multiple messages into batches before sending them to a broker, which significantly improves throughput by reducing the number of network round-trips.
- **Acknowledgement (acks) settings:** Producers can configure how many broker replicas must confirm receipt of a message before it is considered successfully written, allowing applications to trade off latency against durability guarantees.

2.3.5 Consumers and Consumer Groups

A consumer is a client application that subscribes to one or more topics and processes the stream of published events. Consumers read messages in offset order within each partition they are assigned.

Kafka consumers are typically organized into consumer groups, which is the mechanism Kafka uses to support both parallel processing and independent consumption.

- **Within a consumer group:** Each partition of a topic is consumed by exactly one consumer instance within the group, which allows the workload to be

automatically load-balanced across multiple consumer instances as the group scales up or down.

- **Across consumer groups:** Each consumer group maintains its own independent read position, so multiple applications (for example, a real-time dashboard and a fraud-detection engine) can consume the same topic independently, at their own pace, without interfering with one another.
- **Rebalancing:** When a consumer joins or leaves a group, Kafka automatically reassigns partitions among the remaining consumers in that group, ensuring continued load distribution.

The table below summarizes how producers and consumers interact with the core building blocks introduced so far.

Table 2.2: Component; Role in the Kafka Architecture

Component	Role in the Kafka Architecture
Topic	Logical channel that categorizes a stream of related events
Partition	Ordered subdivision of a topic; unit of parallelism and distribution
Producer	Publishes events to a topic, choosing a partition per message
Consumer	Subscribes to a topic and reads events in offset order
Consumer group	Set of consumers that jointly and exclusively divide a topic's partitions for parallel processing

2.3.6 Brokers and the Kafka Cluster

A broker is a single Kafka server that stores data and serves client requests. A Kafka cluster consists of multiple brokers working together, which is what allows Kafka to scale horizontally and tolerate individual machine failures.

- **Partition storage:** Each broker stores a subset of the partitions belonging to the topics in the cluster, rather than every broker storing all the data. This distributes both storage and I/O load across the cluster.
- **Leader and follower replicas:** For each partition, one broker acts as the leader, handling all reads and writes for that partition, while other brokers hold follower replicas that passively copy data from the leader. If the leader fails, a follower is promoted automatically to take over.
- **Cluster coordination:** Brokers rely on a coordination layer (ZooKeeper in older deployments, or Kafka's built-in Kraft protocol in newer versions) to track which broker is the controller responsible for cluster-wide decisions such as electing partition leaders.
- **Horizontal scaling:** Adding more brokers to a cluster allows Kafka to hold more partitions and serve more clients, with existing tools available to redistribute partitions across the expanded cluster.

2.3.7 Offsets and Message Ordering

An offset is a unique, sequential integer assigned to each message within a partition, indicating its position in that partition's log. Offsets start at zero and increase by one for every new message written to the partition.

- **Position tracking:** Consumers use offsets to keep track of which messages they have already processed, allowing them to resume from the correct position after a restart rather than reprocessing the entire stream.
- **Committed offsets:** Kafka stores the last successfully processed offset for each consumer group and partition in a special internal topic, so that progress is not lost even if a consumer crashes.
- **Ordering within a partition:** Because offsets are strictly sequential within a partition, Kafka guarantees that messages within a single partition are always delivered to a consumer in the exact order they were written.
- **No global ordering:** Kafka does not guarantee ordering across partitions within a topic, which is why applications requiring per-entity ordering must route related messages to the same partition using a consistent key, as discussed in Section 2.3.3.

2.3.8 Kafka Data Flow

Bringing together the components discussed so far, a complete message's journey through Kafka follows a consistent sequence of steps, from creation at a producer to consumption by an application.

1. A producer application creates a message and, optionally, assigns it a key.
2. The producer's partitioner determines the target partition, either using the key's hash or a round-robin scheme.
3. The message is sent, often as part of a batch, to the broker currently acting as the leader for that partition.
4. The leader broker appends the message to its local log and assigns it the next available offset.
5. Follower brokers replicate the message from the leader to maintain redundant copies for fault tolerance.
6. Once the message is acknowledged according to the producer's configured durability requirement, the write is considered complete.
7. Consumers belonging to a consumer group poll the assigned partitions for new messages, beginning from their last committed offset.
8. After successfully processing a message (or a batch of messages), the consumer commits the new offset, marking its progress.

This flow illustrates how Kafka's architecture topics, partitions, brokers, replication, and offsets combines into a coherent system for reliable, ordered, and scalable data movement.

2.3.9 Reliability and Fault Tolerance in Kafka

Kafka is designed to continue operating correctly even when individual brokers fail, which is essential for its role as the central nervous system of a streaming architecture.

- **Replication:** Every partition can be configured with a replication factor greater than one, meaning multiple brokers hold a copy of its data. A replication factor of three, for example, allows the cluster to tolerate the failure of up to two brokers without losing data for that partition.
- **In-Sync Replicas (ISR):** Kafka maintains a set of replicas that are fully caught up with the leader, called the ISR. Only replicas in the ISR are eligible to be elected as the new leader if the current leader fails, which prevents data loss from promoting an out-of-date replica.
- **Leader election:** If a leader broker fails, Kafka's coordination layer automatically elects a new leader from the ISR for each affected partition, allowing producers and consumers to continue with minimal disruption.
- **Durability configuration:** Producers can require acknowledgement from all in-sync replicas before considering a write successful, trading a small amount of latency for a strong guarantee against data loss.
- **Data retention:** Because Kafka retains messages on disk for a configurable retention period (rather than deleting them immediately after consumption), consumers that fail and restart can recover and reprocess recent data without permanent loss.

2.3.10 Kafka Working Mechanism Step-by-Step Operational Flow

To consolidate the concepts covered in this section, the operational flow of Kafka can be summarized as a descriptive sequence, without reference to any specific programming implementation.

1. A producing application generates an event (for example, a customer placing an order) and publishes it to a designated topic.
2. Kafka determines the destination partition for the event and routes it to the broker currently serving as that partition's leader.
3. The event is durably stored in the partition's log and replicated to follower brokers for fault tolerance.
4. One or more consumer applications, organized into consumer groups, continuously poll the topic for new events.
5. Each event is delivered to exactly one consumer within a given consumer group, in the order it appears within its partition.
6. The consumer processes the event for example, updating an inventory system or triggering a notification and commits its offset to record progress.
7. If a consumer or broker fails at any point, Kafka's replication and offset-tracking mechanisms allow processing to resume correctly from the last known safe position.

This operational flow underlies every application built on Kafka, regardless of the specific business domain, and it forms the basis for understanding how Kafka integrates with processing frameworks such as Apache Storm, covered next in Section 2.4.

2.4 Apache Storm: Architecture and Working

Apache Storm is a distributed, real-time computation system designed to process unbounded streams of data with very low latency. Originally developed at Back Type, a social-media analytics startup, Storm was open-sourced in 2011 and later adopted and further developed by Twitter, where it was used to process trending topics, real-time search indexing, and advertising analytics at massive scale. Storm was one of the earliest frameworks purpose-built for true event-at-a-time stream processing, in contrast to the micro-batch approach used by some later systems such as Spark Streaming, discussed in Unit 3.

This section examines Storm's architecture and internal working in detail, situating it alongside Kafka as a processing-layer counterpart to Kafka's ingestion-layer role. Where Kafka answers the question of how data is reliably moved and stored, Storm answers the question of how that data is continuously transformed into useful results. Together, the two technologies form a natural pairing that appears repeatedly in real-world architectures, and this pairing is examined directly in Section 2.5.

2.4.1 Introduction to Apache Storm

While Kafka excels at reliably transporting and storing streams of events, it does not, by itself, perform complex computations on that data. Apache Storm fills this gap by providing a framework in which developers define a graph of processing steps that data flows through, with Storm handling the distribution, parallelization, and fault tolerance of that computation across a cluster. A developer using Storm focuses on describing what computation should happen to each unit of data; Storm itself takes care of where that computation runs, how it is scaled, and how it recovers from failure.

To appreciate why a framework like Storm was necessary, it is useful to recall the limitations of earlier large-scale processing systems such as Hadoop MapReduce. MapReduce was designed for batch processing of large, finite datasets and could take minutes or hours to produce a result. For applications such as fraud detection or live trending-topic computation, where a delayed answer is a useless answer, this latency was unacceptable. Storm was designed specifically to close this gap, processing each unit of data within milliseconds of its arrival rather than waiting for a batch boundary.

Several defining characteristics distinguish Storm from both traditional batch frameworks and from messaging systems such as Kafka:

- **True stream processing:** Storm processes each event individually as it arrives, rather than waiting to accumulate a batch, which allows it to achieve sub-second, often sub-100-millisecond, latency for many workloads.
- **Guaranteed processing:** Storm can guarantee that every incoming event is processed at least once, even in the presence of failures, through its built-in tuple-tracking and acknowledgement mechanism, examined in Section 2.4.6.
- **Horizontal scalability:** A Storm computation can be scaled by increasing the parallelism of its components (the number of parallel instances performing a given step) and distributing them across more machines in the cluster, without changing the logic of the computation itself.
- **Language flexibility:** Although Storm itself runs on the Java Virtual Machine and its core APIs are Java-based, it supports a multi-language protocol that allows processing logic to be written in other programming languages, making it accessible to teams with varied technical backgrounds.
- **Technology agnosticism:** Storm does not mandate a specific data source or destination. It can consume data from Kafka, message queues, or custom sources, and can write results to databases, dashboards, or other downstream systems.

The table below situates Storm relative to the two paradigms introduced in Section 2.1, reinforcing why a dedicated real-time computation engine is necessary in a modern data architecture.

Table 2.3: Aspect; Why Apache Storm Was Designed

Aspect	Why Apache Storm Was Designed
Batch limitation addressed	Removes the delay inherent in accumulating data before processing (as in MapReduce-style batch jobs)
Messaging limitation addressed	Adds computation capability on top of raw event transport, which messaging systems like Kafka intentionally do not provide
Core design goal	Deliver low-latency, guaranteed, and horizontally scalable computation over continuous data streams

It is worth explicitly addressing a question that students often ask at this point: if Kafka can store and serve streams of data, why is a separate framework like Storm needed at all? The answer lies in the distinction between transport and computation. Kafka is deliberately designed to be a simple, highly optimized log

it does not know or care what the content of a message means, and it has no built-in concept of joining two streams, maintaining a running aggregate, or evaluating a

condition across multiple related events. Storm, by contrast, is built specifically to express and execute such computations, while delegating the reliable transport of raw data to systems like Kafka. This division of responsibility reliable transport versus flexible computation is a recurring theme in streaming architectures and reappears in Unit 3 when Apache Flink and Apache Spark are introduced as alternative computation-layer technologies.

Storm found early adoption in domains where the cost of a delayed decision was measurable in lost revenue or missed opportunity: real-time advertising bid optimization, trending-topic detection on social media platforms, and continuous fraud-pattern detection in financial transactions. These use cases share a common characteristic the value of an individual insight decays rapidly with time, making Storm's sub-second processing model a natural fit. This same characteristic reappears in the domain-specific case studies presented in Section 2.9.

2.4.2 Storm Architecture

A Storm cluster follows a master-worker architecture, coordinated through Apache ZooKeeper, and consists of three principal types of components: Nimbus, Supervisor nodes, and the ZooKeeper coordination layer. Understanding how these components interact also requires understanding the internal hierarchy through which a computation is actually executed on a supervisor node.

- **Nimbus:** The master node of a Storm cluster, responsible for distributing computation code around the cluster, assigning tasks to machines, and monitoring the cluster for failures. Nimbus is comparable in role to a job scheduler or resource manager in other distributed systems, and in modern Storm deployments can itself be run in a highly available, multi-instance configuration to avoid becoming a single point of failure.
- **Supervisor nodes:** Worker machines that run on each node in the cluster, each managing a set of worker processes and starting or stopping them as instructed by Nimbus. A supervisor does not perform computation itself; it manages the lifecycle of the worker processes that do.
- **ZooKeeper:** A coordination service used to store cluster state, such as task assignments and heartbeats, which allows Nimbus and the supervisors to remain loosely coupled. Because cluster state lives in ZooKeeper rather than in Nimbus's own memory, the cluster can recover gracefully even if Nimbus itself restarts. Within a single supervisor node, computation is organized into a further hierarchy that governs how parallelism is actually achieved:
- **Worker process:** A single Java Virtual Machine process running on a supervisor node, executing a portion of one topology. A supervisor node typically runs several worker processes simultaneously, each potentially belonging to a different topology.

- **Executor:** A thread running within a worker process, responsible for executing one or more tasks. Executors are the unit at which Storm implements parallel execution within a worker.
- **Task:** An instance of a spout or bolt's logic. Each executor runs one or more tasks, and the total number of tasks for a given spout or bolt is configured by the developer as its parallelism hint.

Apache Storm Cluster Architecture

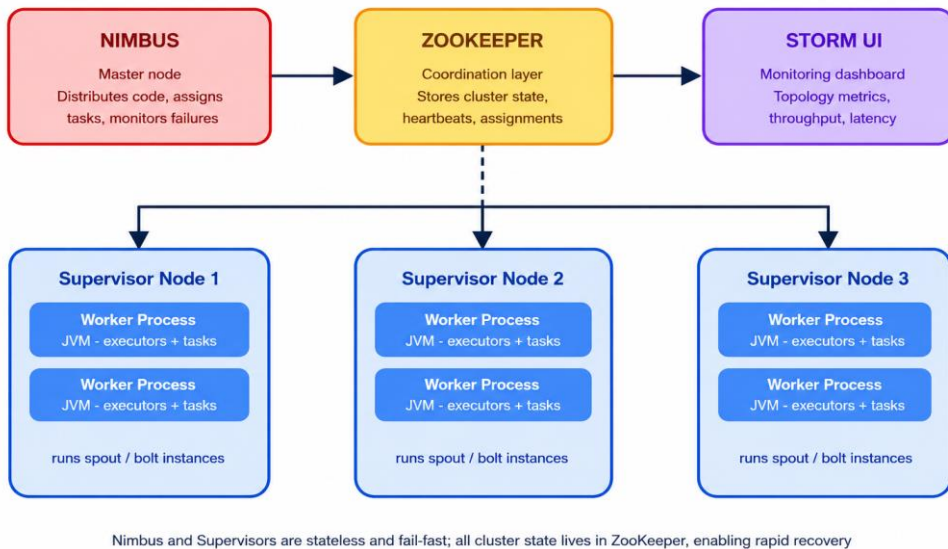


Figure 2.5: Architecture of an Apache Storm cluster

This layered architecture cluster, supervisor, worker process, executor, task allows Storm to scale computation at a fine granularity. A developer can increase the number of tasks for a bottleneck bolt without altering the topology's logic, and Storm's scheduler will distribute those additional tasks across available worker slots in the cluster. This design separates the concern of deciding what work should run where (Nimbus, with the help of Zoo Keeper) from the concern of actually executing that work (supervisors, worker processes, executors, and tasks), a separation that also appears, in a different form, in Kafka's controller-and-broker design discussed in Section 2.3.6.

2.4.2.1 Parallelism and Resource Configuration

Each supervisor node advertises a fixed number of worker slots, representing the number of worker processes it is capable of running simultaneously; this number is typically chosen based on the machine's available CPU cores and memory. When a topology is submitted, the developer specifies, for each spout and bolt, a parallelism hint the number of tasks that should be created for that component.

Nimbus and the supervisors then cooperate to distribute these tasks as evenly as possible across the available worker slots in the cluster.

- **Scaling a bottleneck:** If, for example, an enrichment bolt in a topology becomes a bottleneck because looking up product-category information is comparatively slow, the developer can raise its parallelism hint from four tasks to sixteen, allowing sixteen instances of the enrichment logic to run concurrently across the cluster, without any change to the upstream spout or downstream aggregation bolt.
- **Re balancing:** Storm supports re balancing a running topology, which allows the number of worker processes or the parallelism of individual components to be adjusted while the topology continues to run, avoiding the need to stop and resubmit the entire computation to respond to changing load.
- **Resource-aware scheduling:** More recent versions of Storm support a resource-aware scheduler, which assigns tasks to worker slots based on each component's declared CPU and memory requirements, rather than treating every worker slot as interchangeable, allowing heterogeneous clusters to be used more efficiently.

Correctly tuning parallelism is one of the most important practical skills in working with Storm: under-provisioning a bottleneck component causes tuples to queue up and processing latency to grow, while over-provisioning wastes cluster resources that could be allocated to other topologies.

2.4.3 Topologies, Spouts, and Bolts

The unit of computation in Storm is the topology a directed graph describing how data flows between processing components. Unlike a batch job, which has a defined beginning and end, a Storm topology runs continuously until it is explicitly deactivated, much as a factory assembly line runs continuously rather than processing one finished batch of products at a time.

- **Spout:** The source of data within a topology, responsible for reading events from an external system such as a Kafka topic, a message queue, or a sensor feed and emitting them into the topology as a stream of tuples. A spout can be reliable, meaning it tracks whether each tuple emitted was fully processed and can re-emit it on failure, or unreliable, meaning it emits tuples without tracking their outcome.
- **Bolt:** A processing unit within a topology that consumes tuples from spouts or other bolts, performs some operation on them (such as filtering, transformation, aggregation, or joining two streams), and optionally emits new tuples to downstream bolts. A single topology may contain many bolts arranged in a simple pipeline or a complex directed graph with branches and merges.

- **Tuple:** The basic unit of data passed between spouts and bolts, consisting of an ordered list of named values, comparable conceptually to a single row in a table or a single record in a Kafka message.
- **Stream:** A sequence of tuples flowing between components, conceptually similar to a Kafka topic but scoped to the internal data flow of a topology. A single spout or bolt can emit more than one named stream, allowing different downstream bolts to subscribe to different subsets of its output.

A typical topology therefore begins with one or more spouts injecting data, followed by a chain or graph of bolts that progressively transform, filter, and aggregate that data before producing a final result, which is often written out to an external system such as a database or dashboard.

Apache Storm Topology: Spouts and Bolts

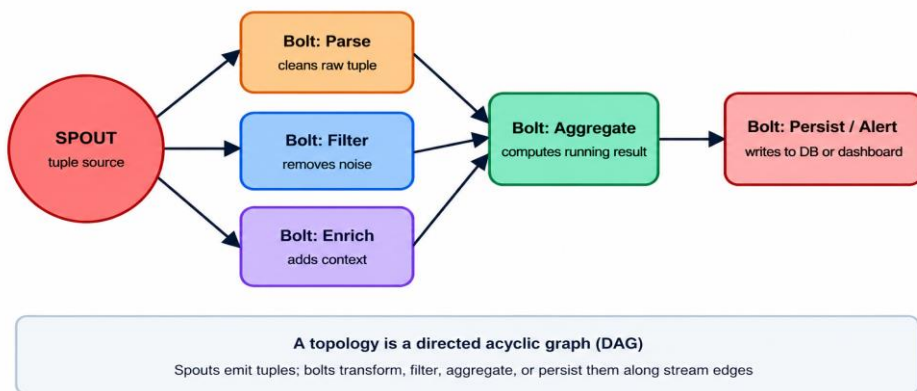


Figure 2.6: A Storm topology: spouts, bolts, and streams

A tuple's fields are named and typed when a spout or bolt declares its output, which allows downstream bolts to refer to specific values (such as a product-category field) by name rather than by position, improving both readability and maintainability of the topology's logic. A single spout or bolt is also permitted to declare and emit more than one named stream for instance, the bot-filtering bolt in the example above might emit a primary stream of legitimate clicks for further processing, alongside a separate, lower-volume stream of flagged suspicious events routed to a security-monitoring bolt. Downstream bolts subscribe explicitly to the particular named stream they are interested in, allowing a single upstream component to serve multiple, independent downstream purposes without duplicating logic.

Every tuple emitted within a topology can also be anchored to the tuple (or tuples) that caused it to be emitted. Anchoring is what allows Storm to construct the tuple tree used for reliability tracking, described in Section 2.4.6: when a bolt emits a new tuple derived from an input tuple, it anchors the new tuple to the input, and Storm's

acker tasks use this chain of anchors to determine when the entire tree of work triggered by an original spout tuple has been completed.

2.4.4 Stream Grouping Strategies

When a bolt has multiple parallel tasks running across the cluster, Storm must decide how tuples emitted by an upstream component are distributed among those tasks. This decision is governed by a stream grouping strategy, specified when the connection between two components is defined in the topology.

- **Shuffle grouping:** Tuples are distributed randomly and evenly across the target bolt's tasks, useful for simple load balancing when the processing of one tuple does not depend on any other tuple, such as the bot-filtering stage in the example above.
- **Fields grouping:** Tuples are routed based on the value of one or more specified fields, ensuring that tuples with the same field value always go to the same task analogous to Kafka's key-based partitioning discussed in Section 2.3.3. This is essential when a bolt must aggregate values per key, such as computing a rolling count per product category.
- **All grouping:** A copy of every tuple is sent to every task of the target bolt, typically used for broadcasting configuration or control signals, such as instructing all tasks to reset an in-memory cache.
- **Global grouping:** All tuples are routed to a single task, useful when a computation must be performed on the entire stream by one instance, such as maintaining a single strict global counter, though this sacrifices the parallelism benefits of the rest of the topology.
- **Direct grouping:** The producer of a tuple decides explicitly which task of the consuming bolt should receive it, giving the developer fine-grained control in advanced use cases.

Storm Stream Grouping Strategies

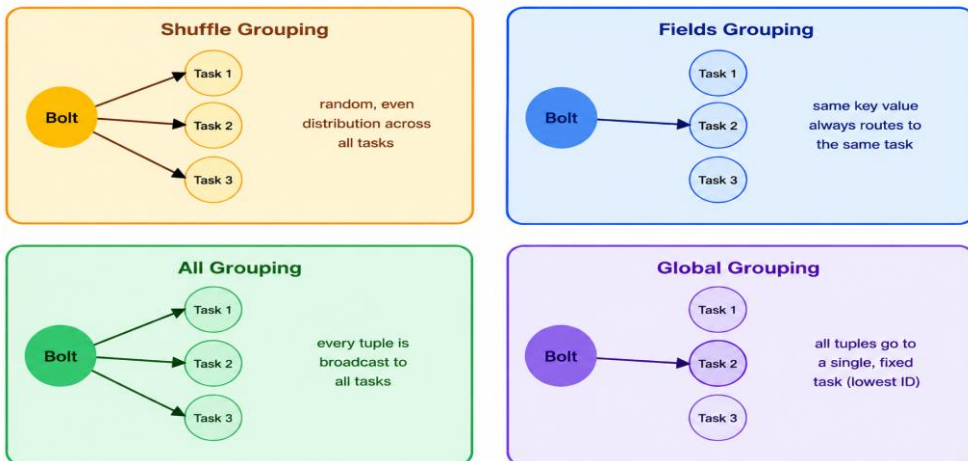


Figure 2.7: Storm stream grouping strategies

The table below summarizes these strategies alongside a representative use case for each, which is useful when designing a new topology.

Table 2.4: Grouping Strategy; Representative Use Case

Grouping Strategy	Representative Use Case
Shuffle grouping	Stateless filtering or validation, where any task can handle any tuple
Fields grouping	Per-key aggregation, such as counting events per user ID or per product category
All grouping	Broadcasting configuration changes or control signals to every task
Global grouping	Maintaining a single, strictly ordered global computation such as an overall running total
Direct grouping	Custom routing logic defined explicitly by the upstream component

Choosing the appropriate grouping strategy is a key design decision when building a Storm topology, directly affecting both correctness (for example, whether related tuples are processed together by the same task) and performance (how evenly the workload is balanced across the cluster). In the click stream example, the bot-filtering bolt would typically use shuffle grouping, while the per-category aggregation bolt would require fields grouping on the product-category field to ensure all clicks for a given category reach the same aggregating task.

A common mistake among students new to Storm is to assume that fields grouping alone is sufficient to guarantee correctness for any stateful aggregation. It is important to note that fields grouping guarantees only that tuples sharing the same field value are routed to the same task; it does not, by itself, guarantee the order in which those tuples arrive relative to tuples from other upstream tasks, nor does it protect the aggregation from being reset if the task itself is restarted after a failure. As discussed in Section 2.4.6, any aggregate that must survive a task restart needs to be checkpointed to, or reconstructed from, an external and reliable store rather than relying purely on the bolt's in-memory state. This distinction between routing guarantees and durability guarantees is a recurring subtlety across all the streaming frameworks discussed in this unit and the next, and revisiting it here reinforces the more general discussion of state management introduced in Section 2.6.4.

2.4.5 Storm Data Flow

The following sequence traces the path of data through a Storm topology, from ingestion by a spout to the final emitted result, consolidating the architectural components discussed in the preceding subsections.

1. A spout reads an event from an external source (for example, a Kafka topic) and emits it into the topology as a tuple, optionally marking it for reliable tracking.
2. Based on the stream grouping strategy configured for the connection, Storm routes the tuple to a specific task of the first bolt in the chain.
3. The bolt processes the tuple filtering, transforming, aggregating, or enriching it and may emit zero, one, or more new tuples, each anchored to the original tuple for reliability tracking.
4. Emitted tuples are routed to the next bolt in the topology according to its configured grouping strategy, and this process repeats across the topology's graph of bolts, potentially branching into multiple parallel paths.
5. The final bolt or bolts in the chain typically write the processed result to an external system, such as a database, dashboard, or alerting service.
6. Throughout this flow, Storm's internal ack tasks track the processing of each tuple to support the acknowledgment and reliability mechanisms described in Section 2.4.6.
7. Once every tuple derived from the original spout tuple has been fully processed and acknowledged, the spout is notified and may safely discard its record of that tuple.

This flow illustrates that, unlike a batch job with a clear start and finish, a Storm topology's data flow is a continuous cycle: steps 1 through 7 occur simultaneously and repeatedly for a very large number of tuples in flight at any given moment, with the cluster's parallelism (executors and tasks) determining how many tuples can be in progress at once.

2.4.6 Reliability and Fault Tolerance in Storm

Storm provides configurable reliability guarantees, allowing developers to choose the appropriate trade-off between processing guarantees and performance for their application. Reliability in Storm operates at two levels: the reliability of individual tuple processing, and the reliability of the cluster's physical infrastructure.

2.4.6.1 Tuple-Level Reliability

- **Acking mechanism:** Storm can track the complete tree of tuples produced from a single input tuple (the original tuple plus all tuples derived from it as it flows through the topology). Specialized internal components called acker tasks are responsible for tracking this tree. Whenever every tuple in

the tree has been fully processed, the originating spout is notified that the original tuple was successfully handled.

- **Replay on failure:** If a tuple is not fully acknowledged within a configured timeout for example, because a bolt failed while processing it, or a worker process crashed the spout automatically replays (re-emits) the original tuple. This provides an at-least-once processing guarantee: every tuple is guaranteed to be processed, though in rare failure scenarios it may be processed more than once.
- **Trident and exactly-once semantics:** For applications that cannot tolerate any duplicate processing, Storm offers a higher-level abstraction called Trident, built on top of the core spout-and-bolt API, which groups tuples into small batches and uses per-batch identifiers to achieve exactly-once processing semantics at the cost of somewhat higher latency.

2.4.6.2 Infrastructure-Level Reliability

- **Task supervision:** If a worker process or an entire supervisor node fails, Nimbus detects the failure through missing heartbeats recorded in Zoo Keeper and reassigns the affected tasks to other available worker slots in the cluster.
- **Nimbus fault tolerance:** Because cluster and task-assignment state is stored in Zoo Keeper rather than solely in Nimbus's memory, a restarted or newly elected Nimbus instance can resume cluster management without losing track of running topologies.
- **Stateless design of bolts:** Because individual bolt tasks can fail and be restarted at any time, Storm encourages designing bolts to be largely stateless, or to externalize any required state to a reliable external store, so that a restarted task can resume correct processing without depending on in-memory data lost during the failure.

These mechanisms allow a Storm topology to continue producing correct results even as individual machines in the cluster fail, which is essential for the always-on nature of streaming applications such as the click stream monitoring example introduced earlier. A brief network interruption or server restart should never cause permanently incorrect trending-category counts.

2.4.6.3 Tuning Reliability Versus Performance

Enabling full reliability tracking is not without cost: anchoring tuples, maintaining tuple trees in the acker tasks, and holding tuples in memory pending acknowledgment all consume additional cluster resources and add a small amount of latency compared to running a topology with reliability tracking disabled entirely. Storm therefore exposes several tunable parameters that allow a development team to match reliability guarantees to the actual requirements of the application.

- **Message timeout:** The maximum time Storm will wait for a tuple's entire processing tree to be acknowledged before considering it failed and eligible for replay. Setting this value too low can cause unnecessary replays under normal, slightly elevated load; setting it too high delays the detection of genuine failures.
- **Number of acker tasks:** Because acker tasks themselves consume cluster resources proportional to the volume of tuples flowing through the topology, their count can be tuned independently, and in topologies where reliability is not required, the number of acker tasks can be set to zero to eliminate this overhead.
- **Choosing unreliable spouts:** For applications such as approximate real-time dashboards, where an occasional dropped event is an acceptable trade-off for lower latency and reduced resource consumption, developers may deliberately choose an unreliable spout rather than paying the cost of full tuple tracking.

This tunability reflects a broader theme that recurs throughout this unit: streaming systems rarely offer a single, one-size-fits-all guarantee, and part of designing a production streaming pipeline is deliberately selecting the point on the latency–reliability–cost spectrum that matches the business requirement at hand.

2.4.7 Storm Working Mechanism Step-by-Step Operational Flow

The overall operation of a Storm topology, from submission to continuous execution, can be summarized descriptively as follows, consolidating the concepts introduced throughout this section.

1. A developer defines a topology the directed graph of spouts and bolts, along with the stream grouping strategy for each connection and submits it to the cluster.
2. Nimbus receives the topology, distributes the necessary code to the cluster, and assigns spout and bolt tasks to worker slots on available supervisor nodes, recording these assignments in ZooKeeper.
3. Each supervisor node launches the worker processes needed to run its assigned tasks, with each worker process running one or more executors and tasks as configured.
4. A spout begins emitting tuples, typically sourced from an external ingestion system such as Kafka, and marks each tuple for reliability tracking if guaranteed processing is required.
5. Tuples flow through the topology's bolts according to the configured stream grouping strategies, undergoing filtering, transformation, enrichment, and aggregation at each stage.
6. Storm's ack tasks track each tuple's processing tree, and the corresponding spout replays any tuple that fails to complete processing within the expected timeout.

7. The final bolt or bolts in the topology write processed results to an external sink, such as a dashboard, database, or alerting service, completing the flow for that tuple.
8. The topology continues running indefinitely, continuously processing new tuples as they are emitted by the spout, until it is explicitly deactivated or killed by an administrator.

It is instructive to observe what these three worked scenarios clickstream monitoring, payment fraud detection, and industrial equipment monitoring have in common. In each case, the topology combines a stateless filtering stage, a stateful per-key aggregation stage using fields grouping, and a decision or output stage, arranged as a short pipeline of bolts. This pattern of filter, aggregate-by-key, and decide is sufficiently common in real-time computation that recognizing it is one of the most transferable skills a student can take from this section, since the same pattern reappears, expressed through different concrete mechanisms, in Apache Flink's windowed and keyed-state operators in Unit 3.

Having examined Kafka and Storm individually Kafka as the reliable, high-throughput ingestion layer, and Storm as the low-latency, guaranteed-processing computation layer the next section compares the two technologies directly, clarifying when each is used and how they complement one another within a complete streaming architecture.

2.4.8 Common Design Considerations for Storm Topologies

Before moving on, it is useful to summarize a small set of practical design considerations that experienced Storm developers apply when building production topologies, drawing together the architectural and reliability concepts covered in this section.

- **Avoid unnecessary bottleneck bolts:** A single bolt with insufficient parallelism can throttle an entire topology, since upstream components will eventually be unable to emit new tuples if Downstream queues fill up. Identifying and provisioning the slowest stage appropriately is usually the single most impactful tuning step.
- **Keep bolts focused and stateless where possible:** Bolts that perform one clearly defined operation are easier to test, reason about, and scale independently than bolts that combine several unrelated responsibilities.
- **Choose grouping strategies deliberately:** Defaulting to shuffle grouping everywhere may seem simpler, but using fields grouping wherever per-key aggregation is required is essential for correctness, not merely a performance optimization.
- **Externalize state that must survive task restarts:** Since any task may be restarted on a different machine at any time, state that must persist beyond

a single tuple's processing should be kept in an external, reliable store rather than solely in a bolt's local memory.

- **Match reliability configuration to business need:** As discussed above, full tuple tracking is not always necessary; applying it uniformly to every topology regardless of requirements wastes cluster capacity that could serve other workloads.

These considerations echo the more general principles of building scalable and fault-tolerant streaming pipelines introduced in Section 2.7, and they will be revisited in the context of specific real-world case studies in Section 2.9.

Finally, it is worth situating Storm within the broader timeline of stream-processing technology covered in this textbook. Storm represents an early and influential generation of dedicated stream-processing engines, and its core abstractions a directed graph of sources and processing units, configurable parallelism, and explicit reliability tracking established a vocabulary that later frameworks would build upon and refine. Apache Flink, introduced in Unit 3, can be understood as addressing many of the same problems as Storm, but with a more unified programming model for both streaming and batch workloads, built-in support for event-time windowing, and a more efficient checkpoint-based fault-tolerance mechanism in place of Storm's per-tuple acking. Recognizing Storm's design as a historical and conceptual foundation, rather than as an isolated topic, will make the architectural choices made by Flink and Spark considerably easier to appreciate when they are introduced later in this book.

2.4.9 Storm Processing Guarantees at a Glance

As a final consolidation of the reliability concepts discussed in Section 2.4.6, the table below summarizes the three levels of processing guarantee available to a Storm developer, along with their typical cost and suitability.

Table 2.5: Guarantee Level; Description, Typical Cost, and Suitability

Guarantee Level	Description, Typical Cost, and Suitability
Best-effort (unreliable spout)	No tracking of tuple outcomes; lowest latency and resource cost; suitable for approximate dashboards or metrics where an occasional lost event is acceptable
At-least-once (reliable spout with tracking)	Every tuple is guaranteed to be processed, with rare duplicate processing possible on failure; moderate overhead from tuple tracking; suitable for most alerting and monitoring applications, including the examples in this section
Exactly-once (Trident)	Every tuple affects the final result exactly once, achieved through micro-batching with batch identifiers; highest latency and

Guarantee Level	Description, Typical Cost, and Suitability
	implementation complexity of the three; suitable for financial or billing-related computations where duplicate processing is unacceptable

Selecting among these three levels is not a purely technical decision; it should be made jointly with an understanding of the business consequences of an occasional duplicate or lost event. A dashboard that briefly over counts a trending product category by a handful of views carries little real consequence, whereas

A billing system that double-charges a customer even rarely can cause significant reputation and financial harm. This same trade-off between guarantee strength and system cost reappears when Apache Flink's check pointing model and Apache Spark's micro-batch model are examined in Unit 3, allowing a direct comparison to be drawn once both frameworks have been introduced.

2.4.10 Section 2.4 Key Terminology Recap

Table 2.6: Term; Definition

Term	Definition
Nimbus	Master node responsible for code distribution, task assignment, and failure monitoring
Supervisor	Worker node that manages worker processes as instructed by Nimbus
Worker process	A JVM process on a supervisor node executing part of a topology
Executor	A thread within a worker process that runs one or more tasks
Task	An individual instance of a spout's or bolt's logic
Topology	The directed graph of spouts and bolts defining a continuous computation
Spout	The data source component of a topology
Bolt	A processing component of a topology
Tuple	The basic unit of data passed between spouts and bolts
Stream grouping	The strategy governing how tuples are distributed to a bolt's parallel tasks
Acker	Internal task that tracks whether a tuple's full processing tree has completed

2.5 Comparative Analysis: Apache Kafka vs. Apache Storm

Sections 2.3 and 2.4 examined Apache Kafka and Apache Storm independently, in each case studying their architecture, core abstractions, data flow, and fault-tolerance mechanisms. A question that naturally follows is how these two technologies relate to one another: are they competitors, offering two solutions to the same problem, or do they serve fundamentally different purposes? This section answers that question directly, comparing the two technologies across several dimensions before examining how they are typically deployed together in a single streaming architecture.

The short answer, developed in more detail below, is that Kafka and Storm are rarely direct substitutes for one another. Kafka is fundamentally a data movement and storage technology an ingestion layer, in the terminology introduced in Section 2.2.3 while Storm is fundamentally a computation technology a processing layer. In most production systems, the two are used together, with Kafka reliably delivering data to, and often receiving results back from, one or more Storm topologies.

2.5.1 Core Comparison

The table below compares Kafka and Storm across the dimensions most relevant to an undergraduate understanding of streaming architecture: their fundamental purpose, core abstractions, processing model, latency characteristics, and approach to fault tolerance.

Table 2.7: Dimension; Apache Kafka; Apache Storm

Dimension	Apache Kafka	Apache Storm
Primary role	Distributed messaging and storage (ingestion layer)	Distributed real-time computation (processing layer)
Core abstraction	Topic, divided into partitions	Topology, composed of spouts and bolts
Unit of data	Message (with key, value, offset)	Tuple (ordered list of named values)
Processing model	Stores and serves data; does not itself transform data	Continuously transforms data as it flows through a topology
Data retention	Persists data on disk for a configurable retention period, allowing replay	Does not persist data; tuples exist only while flowing through the topology
Typical latency	Low latency for delivery; sub-second to a few seconds depending on configuration	Very low latency for computation; often sub-second, including sub-100-millisecond for simple operations

Dimension	Apache Kafka	Apache Storm
Fault-tolerance mechanism	Partition replication with leader election(Section 2.3.9)	Tuple acking and replay, plus task reassignment (Section 2.4.6)
Coordination layer	ZooKeeper (legacy) or built-in KRaft protocol	ZooKeeper
Scaling unit	Partitions distributed across brokers	Tasks distributed across worker slots (Section 2.4.2)
Can operate standalone for an application?	Yes, as a durable buffer/log even without a downstream processor	Rarely; almost always consumes from and/or produces to an external system such as Kafka

A few entries in this table merit further explanation. The distinction between Kafka's message and Storm's tuple, for instance, is not merely terminological: a Kafka message is a durable record with a persistent identity (its offset) that continues to exist on disk long after it has been read, whereas a Storm tuple is a transient unit of computation that exists only for the duration of its journey through a topology, and is discarded once fully processed and acknowledged. Similarly, Kafka's ability to "operate standalone" reflects that a Kafka topic is useful even if no consumer ever reads from it in real time, it still serves as a durable record of everything that occurred whereas a Storm topology with no data source and no data sink performs no useful work at all.

2.5.2 Programming Model Comparison

Beyond their architecture, Kafka and Storm also present quite different programming models to the developers who build on top of them, and understanding this difference clarifies why the two are not interchangeable even at the level of day-to-day development work.

- **Kafka's programming model is centered on producing and consuming records:** an application either writes structured records to a named topic or reads records from a topic starting at some offset. Kafka itself imposes very little structure on what an application does with a record once it has been produced or consumed that logic lives entirely within the client application.
- **Storm's programming model is centered on describing a computation graph:** a developer declares a set of spouts and bolts, specifies how they are wired together and how tuples are grouped between them, and implements the transformation logic that executes inside each bolt. The framework itself is responsible for scheduling, parallelizing, and fault-tolerantly executing this graph continuously.

A useful analogy for students is to think of Kafka as being similar in spirit to a well-designed, durable, and horizontally scalable mailbox system: messages are deposited and collected reliably, but the mailbox itself has no opinion about what the messages mean. Storm, by contrast, is similar to a configurable assembly line: the developer specifies each processing station and how items move between stations, and the assembly line framework worries about running multiple copies of each station in parallel and recovering automatically if a station breaks down.

2.5.3 Common Misconceptions

Before moving on, it is worth explicitly correcting a few misconceptions that students frequently form when first encountering these two technologies together.

- **Misconception:** Kafka and Storm solve the same problem and a system should choose one or the other. As this section has shown, they address different concerns: data transport versus computation and are usually deployed together rather than as alternatives.
- **Misconception:** Storm stores data the way Kafka does. Storm has no built-in long-term storage; any state that must persist beyond an individual tuple's processing, or beyond a task restart, must be explicitly kept in an external store, as discussed in Section 2.4.6.
- **Misconception:** Kafka performs computation on the data it stores. Kafka's role is limited to reliable storage and delivery; any transformation, filtering, or aggregation of the data requires a separate processing-layer technology such as Storm, or the frameworks introduced in Unit 3.
- **Misconception:** Replacing Storm with a newer framework such as Flink automatically removes the need for Kafka. As Unit 3 will show, frameworks such as Flink and Spark still typically consume from and produce to Kafka topics; they replace Storm's role as the computation layer, not Kafka's role as the ingestion layer.

2.5.4 Complementary Roles in a Streaming Architecture

Rather than viewing Kafka and Storm as alternatives, it is more accurate and more useful in practice to view them as adjacent layers in the generic streaming pipeline architecture introduced in Section - 2.2.3. Kafka occupies the ingestion layer, providing a durable, replayable, and highly available buffer between data sources and any number of downstream consumers. Storm occupies the processing layer, subscribing to one or more Kafka topics as its source of tuples and, after computation, frequently publishing its results back into another Kafka topic for further downstream use, in addition to or instead of writing directly to a database or dashboard.

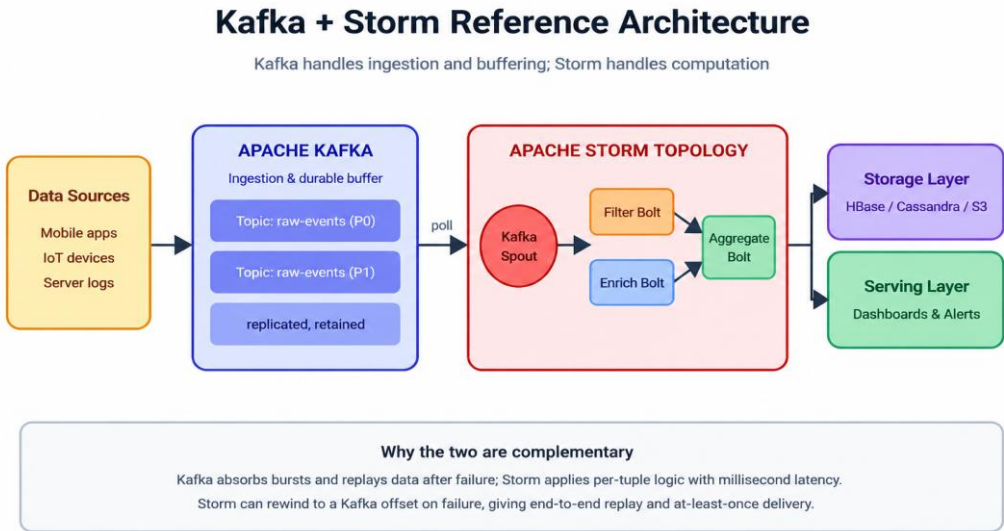


Figure 2.8: Kafka-Storm reference architecture

This pairing offers several practical advantages that neither technology provides on its own:

- **Decoupling of ingestion rate from processing rate:** Because Kafka durably buffers incoming data, a temporary slowdown in the Storm topology for example, during a deployment or a burst of unusually complex events does not cause data loss; the topology simply catches up once normal processing resumes.
- **Independent scaling:** The Kafka cluster and the Storm cluster can be scaled independently based on where the actual bottleneck lies, rather than being forced to scale a single monolithic system.
- **Replay ability for development and recovery:** Because Kafka retains data for a configurable period, a Storm topology can be stopped, modified, and restarted, and will resume processing from its last committed position rather than losing data generated during the outage.
- **Multiple independent consumers of the same data:** The same Kafka topic can simultaneously feed a Storm topology for real-time alerting and a separate batch job for historical analysis, without either consumer affecting the other, as discussed in Section 2.3.5.

2.5.5 When to Use Kafka Alone, Storm Alone, or Both Together

Although the combination of Kafka and Storm is the most common pattern in production streaming systems, it is useful to recognize the narrower situations in which one technology might reasonably be used without the other.

- **Kafka alone:** Appropriate when the requirement is simply to move data reliably between systems without any real-time transformation for example, feeding raw application logs into a data lake for later batch analysis, where no immediate computation on the stream is required.
- **Storm alone:** Appropriate only when data already arrives through some other reliable channel (for example, a message queue already provided by an existing system) and there is no need for Kafka's long-term retention or replay capability; this is comparatively uncommon in new system designs.
- **Kafka and Storm together:** Appropriate for the majority of real-time analytics use cases described in this unit fraud detection, live dashboards, IoT monitoring, and similar applications where both reliable, replayable ingestion and low-latency computation are required simultaneously.

As a general rule of thumb for this course: if a system design problem requires an immediately computed response to incoming events a rolling aggregate, an alert, an enriched record Storm (or a comparable processing-layer technology, such as those introduced in Unit 3) is required in addition to, not instead of, an ingestion-layer technology such as Kafka.

2.5.6 Fault-Tolerance Mechanisms Compared

Although both systems are designed to survive node failures without losing data, the specific mechanisms by which they achieve this differ in ways that reflect their different core responsibilities: durable storage for Kafka, and continuous computation for Storm.

Table 2.8: Failure Scenario; Kafka Response vs. Storm Response

Failure Scenario	Kafka Response vs. Storm Response
A single cluster node fails	Kafka: a follower replica with up-to-date data is elected as the new partition leader (Section 2.3.9). Storm: Nimbus reassigns the failed node's tasks to another available worker slot (Section 2.4.6).
A message or tuple is lost mid-processing	Kafka: not applicable in the same sense, since messages are durably stored before being considered written. Storm: the origin spout replays the tuple after its acknowledgement timeout expires.
The master/coordinator process fails	Kafka: cluster metadata persists in ZooKeeper/Kraft, allowing a new controller to be elected. Storm: cluster and task-assignment state persists in ZooKeeper, allowing a restarted or newly elected Nimbus to resume management.

Failure Scenario	Kafka Response vs. Storm Response
Recovery after extended downtime	Kafka: retained data allows consumers to resume from their last committed offset once systems are restored. Storm: the topology resumes processing new tuples; any tuples in flight during the outage are handled according to the spout's replay behavior.

This comparison reinforces a theme introduced in Section 2.2.1: both systems ultimately rely on the same underlying distributed-systems techniques replication, coordination, and automatic failover but apply them to different kinds of state. Kafka applies them to persisted log data, while Storm applies them to in-flight computation and, where necessary, externally stored aggregation state.

2.5.7 Operational Considerations

Beyond their architectural differences, Kafka and Storm also differ somewhat in the operational effort required to run them reliably in production, which is a relevant practical consideration when designing a system for a course project or an industry deployment.

- **Shared dependency on ZooKeeper:** In many deployments, both Kafka and Storm rely on ZooKeeper for coordination (although Kafka's newer Kraft mode removes this dependency for Kafka itself), meaning Zookeeper's own availability becomes a shared point of operational concern for the combined system.
- **Monitoring surface:** A combined Kafka–Storm deployment requires monitoring metrics from both systems for example, Kafka consumer lag (how far behind a consumer is from the latest message) alongside Storm's tuple-processing latency and failure/replay rates to get a complete picture of end-to-end pipeline health.
- **Learning curve:** Teams adopting this combination must become proficient in two distinct sets of concepts and operational tools, which is a genuine cost compared to unified frameworks such as Apache Flink, discussed in Unit 3, that combine ingestion-adjacent and computation capabilities more closely within a single system.

These operational considerations do not diminish the value of the Kafka–Storm pairing for the use cases this unit has examined, but they are worth bearing in mind as these textbook progresses to Apache Flink and Apache Spark in Unit 3, both of which take somewhat different design approaches to reducing this operational overhead.

2.5.8 A Practical Decision Checklist

For students approaching a system-design question or a course project involving streaming data, the following checklist consolidates this section's discussion into a set of practical questions that can guide the choice between Kafka, Storm, and their combination.

- Does the system need to reliably move data between multiple independent producers and consumers, possibly at different rates? If yes, Kafka (or a comparable ingestion-layer technology) is required.
- Does the system need to compute something from the data as it arrives a filter, an aggregate, an alert, an enrichment within a strict latency budget? If yes, a processing-layer technology such as Storm is required in addition to the ingestion layer.
- Does the system need to replay historical data, recover cleanly from extended downtime, or support multiple independent consumers of the same raw data? If yes, Kafka's durable retention makes it the appropriate choice for the ingestion layer rather than a non-durable message queue.
- Does the required computation need strict exactly-once correctness, such as in a financial settlement system? If yes, the reliability configuration discussed in Section 2.4.6 (Ortho checkpoint-based guarantees of Flink, introduced in Unit 3) must be evaluated carefully rather than assumed.
- Is the processing logic complex, involving multiple stages, branching streams, or per-key state? If yes, an explicit topology-based framework such as Storm is more maintainable than attempting to embed this logic directly inside a Kafka consumer application. Working through this checklist systematically, rather than defaulting to a single familiar technology, is itself a core engineering skill that this unit aims to develop, and the same style of checklist will be revisited when Apache Flink and Apache Spark are introduced as additional processing-layer choices in Unit 3.

2.5.9 Summary of the Comparison

In summary, Apache Kafka and Apache Storm are best understood not as competing solutions to the same problem, but as complementary layers within a single streaming architecture: Kafka provides the durable, replayable, and highly scalable transport of data, while Storm provides the low-latency, guaranteed computation applied to that data. The remainder of this unit builds on this pairing directly Section 2.6

examines how real-time data streams are handled and managed once they are flowing through such a pipeline, and Section 2.7 examines how to design the pipeline itself for scalability and fault tolerance, drawing on the mechanisms introduced for both Kafka and Storm in this and the preceding two sections.

2.6 Handling and Managing Real-Time Data Streams

Having examined Kafka and Storm as concrete technologies, this section steps back to examine a set of conceptual techniques that apply broadly to real-time data stream handling, regardless of which specific framework is used to implement them. These concepts the distinction between real-time and micro-batch processing, event time versus processing time, windowing, state management, and ordering guarantees form the shared vocabulary that will also be used when Apache Flink and Apache Spark are introduced in Unit 3, where several of these ideas are developed in significantly greater technical depth.

2.6.1 Real-Time vs. Micro-Batch Processing

Section 2.1.1 introduced the broad distinction between batch and streaming processing. Within streaming processing itself, however, there is a further important distinction between two execution models: true realtime (event-at-a-time) processing and micro-batch processing.

- **Real-time (event-at-a-time) processing:** Each event is processed individually, as soon as it is available, without waiting for other events to accumulate. Apache Storm, examined in Section 2.4, follows this model a tuple is processed by a bolt as soon as it arrives at that bolt's task.
- **Micro-batch processing:** Incoming events are grouped into very small batches often covering a window of a few hundred milliseconds to a few seconds and each micro-batch is processed as a unit using batch-style computation. Apache Spark Streaming, introduced in Unit 3, is the most prominent example of this model. Neither model is universally superior; each represents a different point on the latency-versus-throughput trade-off introduced as a key challenge in Section 2.1.3. Event-at-a-time processing typically achieves the lowest possible latency, since there is no waiting for a batch boundary, but can incur higher per-event overhead. Micro-batch processing sacrifices a small, bounded amount of latency in exchange for higher throughput and, often, a simpler programming model that can reuse batch-processing abstractions.

Table 2.9: Aspect; Real-Time vs. Micro-Batch

Aspect	Real-Time vs. Micro-Batch
Latency	Real-time: near-instant, often sub-100ms. Micro-batch: bounded by batch interval, typically hundreds of milliseconds to a few seconds
Throughput	Real-time: high, but per-event overhead can limit peak throughput. Micro-batch: typically higher, due to batch-level efficiencies
Programming model	Real-time: explicit event-by-event handlers (as in Storm bolts). Micro-batch: familiar batch-style transformations applied per small batch

Aspect	Real-Time vs. Micro-Batch
Representative framework	Real-time: Apache Storm (Section 2.4). Micro-batch: Apache Spark Streaming (Unit 3)

2.6.2 Event Time vs. Processing Time Semantics

A subtle but critical concept in stream processing is that every event is associated with (at least) two distinct notions of time, and confusing the two is a common source of incorrect results in real-world systems.

- **Event time:** The time at which an event actually occurred in the real world for example, the moment a sensor recorded a temperature reading, or the moment a customer clicked a button. Event time is typically embedded as a field within the event itself, set by the source system at the moment of generation.
- **Processing time:** The time at which an event is observed and processed by the streaming system for example, the moment a Storm bolt executes its logic on that event. Processing time is determined by the streaming system's clock at the moment of computation.

In an ideal system, event time and processing time would always be equal every event would be processed the instant it occurred. In practice, network delays, system outages, mobile devices operating offline before reconnecting, and clock differences between machines all cause events to arrive for processing later than the moment they actually occurred, sometimes by a significant margin.

This distinction directly motivates the concept of windowing, discussed next, since a window is typically defined in terms of event time rather than processing time whenever accurate, time-faithful results are required. It also foreshadows the discussion of late and out-of-order data as a challenge for data transformation and analysis, revisited in Section 3.8.2.

A closely related concept, introduced here only briefly and developed further alongside Apache Flink in Unit 3, is the watermark — a mechanism by which a streaming system estimates how far event time has progressed, and therefore when it is safe to consider a given window's results final. A watermark is, in effect, the system's running assumption of the form "it is now reasonable to assume no further events with an event time earlier than this point will arrive." Setting this assumption too aggressively risks closing a window before all its rightful data has arrived, producing an incomplete result, while setting it too conservatively delays the availability of results unnecessarily. This tension between result completeness and result timeliness is a direct consequence of the event-time/processing-time

distinction introduced above, and it is one of the more conceptually subtle ideas in modern stream processing.

2.6.3 Introduction to Windowing in Stream Processing

Because a data stream is unbounded, it is generally not possible to compute an aggregate such as a sum, an average, or a count over the entire stream at once; there is no defined "end" at which such a computation could be considered final. Windowing addresses this problem by dividing the continuous stream into finite, manageable slices of time (or, less commonly, of event count) over which an aggregate can be meaningfully computed. This subsection introduces windowing conceptually; a detailed, framework-specific treatment appears in Section 3.3.4, in the context of Apache Flink, which offers particularly rich windowing capabilities.

- **Tumbling windows:** Fixed-size, non-overlapping windows that partition the stream into consecutive, disjoint intervals for example, a tumbling window of one minute would compute a separate aggregate for the intervals 09:00–09:01, 09:01–09:02, and so on, with every event belonging to exactly one window.
- **Sliding windows:** Fixed-size windows that advance by an interval smaller than the window's own size, causing consecutive windows to overlap for example, a five-minute window that slides forward every one minute would produce a new aggregate every minute, each covering the preceding five minutes of data, meaning a single event may contribute to several overlapping windows.
- **Session windows:** Dynamically sized windows defined not by a fixed clock interval but by periods of activity separated by a gap of inactivity for example, grouping a user's website clicks into a single "session" window that closes automatically after the user has been inactive for a configured period, such as thirty minutes.

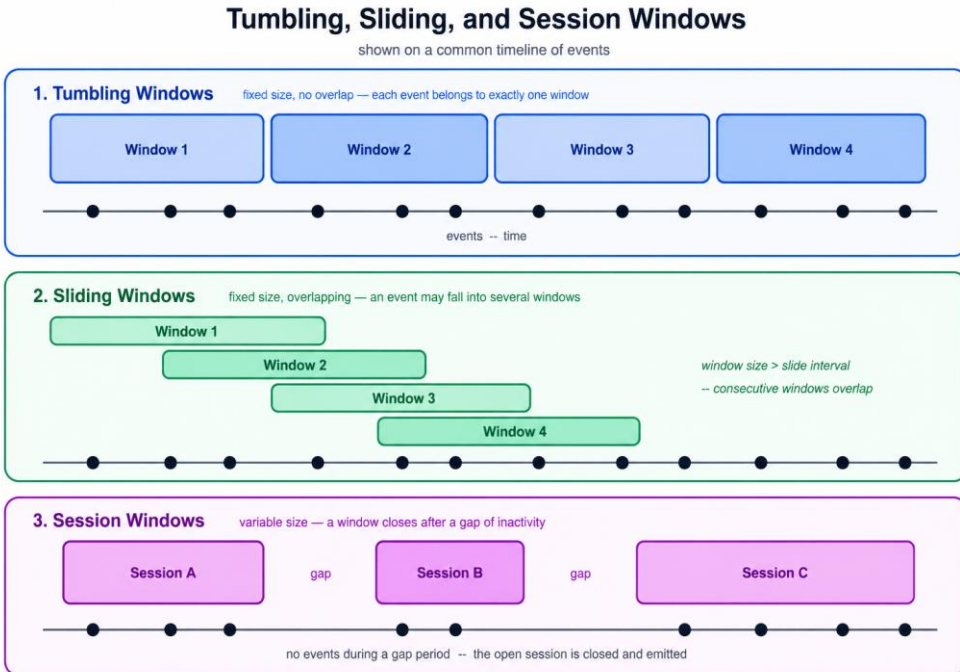


Figure 2.9: Tumbling, sliding, and session windows on a timeline

Choosing the appropriate windowing strategy depends entirely on the question being asked of the data: tumbling windows suit simple periodic reporting ("orders per minute"), sliding windows suit smoothed, continuously updated metrics ("a rolling five-minute average, updated every minute"), and session windows suit behavior-driven groupings where the natural unit of analysis is an episode of activity rather than a fixed clock interval, such as a single user session or a single equipment operating cycle.

2.6.4 Introduction to State Management in Streaming Systems

Many of the computations examined in this unit the rolling driver index in Section 2.5's ride-hailing scenario, the rolling transaction history in its banking scenario, and windowed aggregates in general require the streaming system to remember information across multiple events, rather than treating each event in complete isolation. This remembered information is referred to as state, and managing it correctly is one of the more challenging aspects of building a production streaming system.

- **What state represents:** State can be as simple as a running count or sum, or as complex as an entire in-memory index, such as the driver-location index described in Section 2.5. In all cases, state is information derived from past events that influences how future events are processed.

- **Why state management is difficult:** Unlike a traditional application with a single, reliably running database, a streaming computation's state is distributed across many parallel tasks (as seen with Storm's fields-grouped bolts in Section 2.4.4) and must survive the failure and restart of any individual task without being lost or corrupted.
- **Local versus externalized state:** State can be kept locally in memory within a processing task for speed, but, as discussed in Section 2.4.6, this makes it vulnerable to loss when a task restarts unless it is periodically checkpointed to, or entirely maintained in, a reliable external store such as a distributed database or a dedicated state backend.
- **Keyed state:** In most practical systems, state is partitioned by a key such as a customer ID, a card number, or a machine identifier mirroring the fields-grouping approach discussed in Section 2.4.4, so that each parallel task is responsible for maintaining the state of only the keys routed to it. The choice between keeping state purely local and externalizing it to a reliable store is itself a trade-off worth making explicit, since it recurs whenever a streaming pipeline is designed.

Table 2.10: Approach; Trade-off

Approach	Trade-off
Local, in-memory state only	Fastest possible read/write access; simplest to implement; state is lost if the task fails or is rescheduled, unless the framework provides its own transparent recovery mechanism
Externalized state (e.g., a distributed database or dedicated state backend)	Survives task failure and rescheduling; can be shared or inspected outside the streaming job; introduces additional network latency and an additional operational dependency

State management is treated only conceptually in this unit; Section 3.3.5 revisits the topic in the specific context of Apache Flink, which provides more sophisticated, built-in state-management primitives than Storm's core API, including automatic, efficient checkpointing of large amounts of keyed state.

2.6.5 Ensuring Data Ordering and Consistency in Streams

The final concept examined in this section is the practical challenge of maintaining correct ordering and consistency as data flows through a distributed streaming pipeline, drawing together ideas already introduced separately for Kafka and Storm.

- **Ordering within a partition or key:** As discussed in Sections 2.3.7 and 2.4.4, both Kafka and Storm can guarantee ordering only within a single partition or a single field-grouped key, not across an entire topic or topology.

System designers must therefore deliberately choose a partitioning or grouping key that matches the entity for which ordering actually matters.

- **Handling out-of-order arrivals:** Even within a correctly partitioned stream, events may still arrive out of event-time order due to the network delays discussed in Section 2.6.2. Systems that require strict event-time correctness must therefore incorporate some tolerance for lateness, such as briefly delaying the finalization of a window's result to accommodate slightly late-arriving events, a technique examined further in Section 3.8.2.
- **Consistency across parallel tasks:** Because a stream is processed by many parallel tasks simultaneously, ensuring that all tasks agree on shared state for example, all replicas of a rolling aggregate reflecting the same set of processed events requires the coordination and consensus mechanisms introduced in Section 2.2.1.
- **Trade-offs with availability:** Consistent with the CAP theorem discussed in Section 2.2.1, streaming systems generally favor continued availability during network partitions over perfect, immediate consistency, accepting that results may be briefly approximate before converging to a fully consistent state once the partition is resolved.

The table below consolidates how the two technologies studied in this unit approach ordering specifically, reinforcing material introduced separately in Sections 2.3 and 2.4.

Table 2.11: System; Ordering Guarantee

System	Ordering Guarantee
Apache Kafka	Strict ordering guaranteed only within a single partition, based on strictly increasing offsets (Section 2.3.7); no ordering guarantee across partitions of the same topic
Apache Storm	No inherent ordering guarantee across a topology; consistent per-key ordering can be approximated by combining fields grouping (Section 2.4.4) with a single-threaded task per key, though this is a design choice rather than a built-in guarantee

This comparison highlights an important general principle for system design: strict ordering is never free. It is achieved by deliberately routing related events to the same partition or task sacrificing some parallelism for that subset of data rather than being an automatic property of any distributed streaming system. Designers must identify, for each specific application, exactly which entities require ordered processing (a single card's transactions, a single sensor's readings, a single user's actions) and structure their partitioning or grouping keys accordingly.

It is worth closing this section by observing how tightly its five subsections interlock in practice. A design decision made for one concept nearly always has

consequences for another: choosing event-time windowing (Section 2.6.3) requires a policy for late-arriving data, which in turn depends on how far processing time can lag behind event time (Section 2.6.2); maintaining a windowed aggregate requires state (Section 2.6.4); and correctly updating that state per key requires the same partitioning discipline used to guarantee ordering (Section 2.6.5). Rather than five independent topics, this section is best understood as five interconnected facets of a single underlying engineering problem: producing a timely, correct, and resource-efficient summary of an unbounded stream of real-world events.

The table below previews where each concept introduced in this section is picked up again in greater technical depth later in this textbook, which may be a useful map for students revisiting these ideas while studying Unit 3.

Table 2.12: Concept from Section; Revisited In 2.6

Concept from Section 2.6	Revisited In
Event time vs. processing time; watermarks	Section 3.3.4 (Flink windowing) and Section 3.8.2 (late/out-of-order data)
Tumbling, sliding, and session windows	Section 3.3.4 (Flink-specific windowing implementation)
Keyed state and externalized state	Section 3.3.5 (Flink state management)
Ordering and consistency trade-offs	Section 3.8 (challenges and solutions in transformation and analysis)

Taken together, the five subsections of this section establish the conceptual toolkit execution model, time semantics, windowing, state, and ordering/consistency needed to reason precisely about any realtime data stream, independent of the specific technology used to implement it. Section 2.7 builds directly on this toolkit, applying it to the practical problem of designing an end-to-end streaming pipeline that is both scalable and fault-tolerant.

2.6.6 Section 2.6 Key Terminology Recap

Table 2.13: Term; Definition; Introduced In

Term	Definition	Introduced In
Event time	The time an event actually occurred in the real world	Section 2.6.2
Processing time	The time an event is observed/processed by the system	Section 2.6.2

**Table 2.14: Tumbling window; Fixed-size, non-overlapping time-based
Section 2.6.3 window**

Tumbling window	Fixed-size, non-overlapping time-based window	Section 2.6.3
Sliding window	Fixed-size, overlapping window advancing by a smaller interval	Section 2.6.3
Session window	Activity-based window closed by a gap of inactivity	Section 2.6.3
State	Information from past events retained to influence future processing	Section 2.6.4
Keyed state	State partitioned by an entity key such as a customer or device ID	Section 2.6.4

2.7 Building Scalable and Fault-Tolerant Streaming Pipelines

The preceding sections examined Kafka, Storm, and the general concepts of real-time stream handling largely as independent building blocks. This section brings those building blocks together, addressing the practical engineering question that a systems designer ultimately faces: how does one combine these technologies and concepts into a complete pipeline that can scale to handle growing data volumes and continue operating correctly when individual components fail? The principles developed here apply not only to Kafka-and-Storm pipelines, but equally to the Flink- and Spark-based pipelines introduced in Unit3.

2.7.1 Principles of Scalability in Streaming Systems

Scalability is the ability of a system to handle increasing amounts of work by adding resources, rather than by redesigning the system itself. In the context of a streaming pipeline, "work" can be measured along two related but distinct dimensions: the volume of data flowing through the system, and the complexity of the computation applied to that data.

- **Horizontal scaling:** Adding more machines to a cluster, allowing the workload to be distributed across a larger number of parallel units additional Kafka brokers, or additional Storm worker slots and tasks. This is the primary scaling strategy for streaming systems, since it has no theoretical upper bound and matches the distributed architecture introduced in Section 2.2.1.
- **Vertical scaling:** Increasing the resources (CPU, memory) of individual machines already in the cluster. This is simpler to apply but has a practical ceiling, and does not, by itself, improve fault tolerance, since a more

powerful single machine remains a single point of failure for the work assigned to it.

- **Elasticity:** The ability of a system to scale resources up during periods of high load and back down during periods of low load, rather than being provisioned for permanent peak capacity, which is particularly relevant for streaming workloads that often exhibit strong daily or seasonal traffic patterns.

A useful way to reason about scalability is to identify the bottleneck resource for a given pipeline the single component whose capacity limits the throughput of the entire system and to scale that component specifically, rather than uniformly scaling every component regardless of need. This mirrors the discussion of tuning a bottleneck Storm bolt in Section 2.4.2, applied now at the level of an entire pipeline.

2.7.2 Partitioning and Load-Balancing Techniques

Partitioning dividing a dataset or workload into smaller, independently processable pieces is the foundational technique that makes horizontal scaling possible, and it appears at every layer of the pipelines studied in this unit.

- **Kafka partitioning:** As discussed in Section 2.3.3, a Kafka topic's partitions are the unit across which both storage and consumer parallelism are distributed; adding partitions (within limits) allows more consumers to read from a topic in parallel.
- **Storm task parallelism:** As discussed in Section 2.4.2, a bolt's parallelism hint determines how many tasks share its workload, with stream grouping strategies (Section 2.4.4) determining how that workload is distributed among them.
- **Key-based partitioning for correctness:** Beyond raw load balancing, partitioning by a consistent key (customer ID, card number, device ID) is what allows related events to be processed together and in order, as discussed in Section 2.6.5, choosing partition key both a performance and a correctness decision.
- **Load balancing:** Even with a fixed number of partitions, the distribution of load across those partitions may be uneven if certain keys are much more frequent than others a phenomenon known as data skew. Mitigating skew may require choosing a more granular partitioning key, or, in Storm, deliberately over-provisioning the parallelism of a bolt that receives a disproportionate share of a hot key's traffic.

The number of partitions or tasks chosen for a given component is rarely a one-time decision; production systems typically monitor throughput and consumer lag (in Kafka) or queue buildup and processing latency (in Storm) and adjust partitioning and parallelism as data volumes grow over the system's lifetime.

2.7.3 Replication and Fault-Tolerance Techniques

Where Section 2.7.2 addressed scaling to handle growing load, this subsection addresses the complementary requirement of continuing to operate correctly when individual components of that now-larger, more distributed system inevitably fail.

- **Data replication:** As discussed in Section 2.3.9, Kafka maintains multiple copies of each partition across different brokers, so the failure of any single broker does not result in data loss, provided the replication factor is greater than one.
- **Computation redundancy:** Storm's supervisor architecture (Section 2.4.6) automatically reassigns a failed task's work to another available worker slot, ensuring that computation, and not only data, survives individual machine failures.
- **Avoiding single points of failure:** Both Kafka's controller and Storm's Nimbus can, in modern deployments, run in a highly available configuration with automatic failover to a standby instance, removing what would otherwise be a single point of failure for cluster-wide coordination.
- **Geographic redundancy:** For the highest-criticality systems, entire clusters may be replicated across physically separate data centers, protecting against failures such as a power outage or a natural disaster that could affect an entire single location at once.

Table 2.15: Failure Scope; Typical Mitigation

Failure Scope	Typical Mitigation
Single disk or single machine failure	Data and task replication within a single cluster (Sections 2.3.9, 2.4.6)
Coordinator/master failure (Nimbus, Kafka controller)	Highly available master configuration with automatic failover
Entire data center or region failure	Cross-region replication of clusters and data

2.7.4 Check pointing and Recovery Mechanisms

Check pointing is the practice of periodically saving a consistent snapshot of a system's progress or state, so that after a failure the system can resume from the checkpoint rather than restarting all processing from the very beginning.

- **Kafka's implicit checkpoint the committed offset:** As discussed in Section 2.3.7, a consumer's committed offset functions as a lightweight checkpoint of its progress through a partition, allowing it to resume exactly where it left off after a restart, without needing a separate checkpointing mechanism of its own.
- **Storm's tuple-tracking as a recovery mechanism:** Rather than periodic snapshots, Storm relies on the per-tuple acking mechanism described in

Section 2.4.6 to detect and replay failed work, which can be understood as a very fine-grained, continuous form of recovery rather than a periodic checkpoint.

- **Application-level state checkpointing:** For any state maintained by a bolt beyond a single tuple (Section 2.6.4), the application itself is generally responsible for periodically persisting that state to an external store, since Storm's core API does not checkpoint bolt state automatically a limitation directly addressed by Apache Flink's built-in checkpointing mechanism, introduced in Section 3.3.6.

Together, offset commits and tuple acking mean that a Kafka-plus-Storm pipeline can recover from most single-component failures without manual intervention or permanent data loss, though, as just noted, correctly recovering any custom in-bolt state remains the responsibility of the application developer.

Table 2.16: Mechanism; What It Protects and How It Recovers

Mechanism	What It Protects and How It Recovers
Kafka committed offset	Protects consumer read progress; on restart, the consumer resumes from the last committed offset for its assigned partitions
Storm tuple acking	Protects individual tuple processing; on failure or timeout, the originating spout replays the affected tuple
Application-managed state checkpoint	Protects custom bolt state (e.g., rolling aggregates); must be explicitly saved to, and reloaded from, an external store by the developer

A subtle but important point for students to internalize is that these three mechanisms operate at different granularities and are not interchangeable: correctly committing Kafka offsets says nothing about whether a bolt's in-memory aggregate has been safely preserved, and a correctly acked tuple says nothing about whether the consumer's offset has actually been committed. A robust pipeline design must consider all three mechanisms together, rather than assuming that handling one automatically covers the others.

2.7.5 Illustrative Scenario: Designing an End-to-End Scalable Pipeline

Consider a retail company's order-processing pipeline, which must handle a steady baseline of a few hundred orders per minute on an ordinary day, rising to several thousand orders per minute during a flash sale. The design walkthrough below applies the concepts introduced in this section to that scenario.

1. Ingestion is handled by a Kafka topic partitioned by a moderately high number of partitions, chosen so that the system can scale consumer

- parallelism well beyond the baseline load, with a replication factor of three to tolerate the loss of up to two brokers without data loss.
2. A Storm topology consumes the orders topic, with each bolt's parallelism hint set according to its relative processing cost for example, a lightweight validation bolt configured with fewer tasks than a heavier inventory-check bolt that must query an external service.
 3. During the flash sale, cluster operators (or an automated scaling policy) increase the number of Storm worker slots and rebalance the topology's parallelism upward, as discussed in Section 2.4.2, without stopping the pipeline.
 4. If a Storm worker fails under the increased load, Nimbus reassigns its tasks to another available slot, and any tuples that were mid-processing are replayed automatically, guaranteeing that no order is silently dropped even during peak load.
 5. Processed order confirmations are published to a second, similarly replicated Kafka topic, which independently feeds a customer-notification service and a separate analytics pipeline monitoring sales in real time, illustrating the multiple-independent-consumer pattern discussed in Section 2.3.5.
 6. Once the flash sale ends, the additional Storm resources can be scaled back down, returning the pipeline to its baseline resource footprint, consistent with the elasticity principle introduced in Section 2.7.1.

This scenario demonstrates that scalability and fault tolerance are not separate concerns bolted onto a finished design, but properties that must be designed into the pipeline from the outset in the choice of partition count, replication factor, and parallelism strategy a perspective that carries forward directly into the pipeline designs built around Apache Flink and Apache Spark in Unit 3.

It is also worth noting what this walkthrough deliberately did not require: at no point was it necessary to redesign the pipeline's fundamental structure to accommodate the flash-sale traffic spike. The same topics, the same topology, and the same overall data flow handled both the ordinary baseline load and the elevated peak load; only the degree of parallelism and, temporarily, the amount of allocated cluster capacity changed. This is precisely the practical benefit that the principles introduced throughout Section 2.7 are intended to deliver a pipeline whose capacity can be adjusted to match demand without requiring the underlying architecture to be rebuilt each time that demand changes significantly, which is as important a measure of good streaming-system design as raw throughput or latency figures alone.

2.7.6 Section-2.7 Key Terminology Recap

Table 2.17: Term; Definition

Term	Definition
Horizontal scaling	Adding more machines to a cluster to increase capacity
Vertical scaling	Increasing the resources of an individual machine
Elasticity	Scaling resources up and down automatically in response to load
Data skew	Uneven distribution of load across partitions or keys
Replication factor	The number of copies maintained of a given partition
Checkpointing	Periodically saving progress or state to enable recovery after failure

2.8 Real-World Domains for Streaming Applications

The concepts and technologies developed throughout this unit find application across a wide range of industries. This section briefly surveys six representative domains, each of which relies on the same underlying principles of ingestion, low-latency computation, scalability, and fault tolerance covered in Sections 2.2 through 2.7, applied to a different real-world problem.

2.8.1 IoT and Sensor Data Streaming

Networks of connected sensors in agriculture, logistics, and smart appliances continuously emit readings such as temperature, location, or vibration. Streaming pipelines ingest this high-volume, high-velocity data via Kafka and apply real-time filtering and aggregation, as illustrated by the industrial monitoring scenario in Section 2.4.7, to detect anomalies without waiting for a batch report. The sheer number of independent, often low-powered devices involved makes horizontal scalability (Section 2.7.1) a particularly central concern in this domain.

2.8.2 Banking and Financial Fraud Detection

Card-payment networks and banks must evaluate each transaction for fraud risk within milliseconds of authorization. As shown in the payment-fraud scenario in Section 2.4.5, this requires per-account state (Section 2.6.4) and low-latency rule evaluation, since a delayed decision defeats the purpose of real-time fraud prevention. Strict ordering of transactions per card, achieved through consistent partitioning (Section 2.6.5), is essential to this domain's correctness.

2.8.3 Social Media Analytics

Platforms that surface trending topics, live engagement counts, or content-moderation flags must process enormous volumes of posts, likes, and shares continuously. This domain was part of Apache Storm's original motivation, discussed in Section 2.4.1, and remains a canonical example of windowed aggregation (Section 2.6.3) applied at very large scale, often requiring thousands of parallel tasks distributed across a large cluster.

2.8.4 Transportation and Logistics

Ride-hailing, fleet management, and delivery-tracking systems depend on continuous location data to match supply with demand and optimize routes, as illustrated by the ride-hailing scenario in Section 2.5. These systems typically combine geographic partitioning with stateful, per-region matching logic, and must remain responsive even as demand shifts rapidly between regions throughout the day.

2.8.5 Healthcare Monitoring

Continuous patient monitoring vital signs from bedside devices or wearable sensors requires streaming pipelines that can detect abnormal readings and trigger alerts immediately, since a delayed alert in this domain carries direct patient-safety consequences, making guaranteed processing (Section 2.4.6) especially critical. Regulatory requirements around data retention also make Kafka's durable storage model (Section 2.3.9) particularly relevant in this domain.

2.8.6 Cybersecurity (Intrusion and Anomaly Detection)

Security teams monitor network traffic and system logs in real time to detect intrusion patterns, unusual login behavior, or denial-of-service activity. Because attacks unfold over seconds, not hours, this domain depends heavily on the low-latency, stateful computation capabilities of frameworks such as Storm, combined with Kafka's ability to durably retain logs for later forensic analysis and compliance auditing.

2.9 Case Studies on Real-World Data Streaming Applications

The three case studies below each combine Kafka and Storm to solve a concrete problem drawn from the domains above, consolidating the architectural patterns developed throughout this unit. Each case study deliberately follows the same underlying structure partitioned Kafka ingestion feeding a Storm topology that filters, aggregates, and acts on the data to reinforce that this pattern, rather than any single domain-specific detail, is the transferable skill this unit aims to build.

2.9.1 Case Study: Kafka-Based Log Aggregation in an E-Commerce Platform

A large e-commerce platform's many micro services each generate operational logs continuously. Rather than writing logs to local files, every service publishes structured log events to a shared Kafka topic, partitioned by service name. A Storm topology consumes this topic to detect error-rate spikes in real time, while a separate batch job periodically archives the same topic's data into long-term storage for later analysis. This case study demonstrates Kafka's role in decoupling many independent producers from multiple independent consumers, as introduced in Section 2.3.5, and shows how a single ingestion topic can simultaneously serve both real-time alerting and long-term batch analytics needs.

2.9.2 Case Study: Real-Time IoT Sensor Pipeline Using Kafka and Storm

A smart-agriculture company deploys thousands of soil-moisture and weather sensors across multiple farms. Sensor readings are published to Kafka, partitioned by farm, and a Storm topology performs the filter-aggregate-decide pattern introduced in Section 2.4.7: discarding faulty readings, computing rolling moisture averages per field, and triggering irrigation alerts when thresholds are crossed. This case study illustrates the generic IoT architecture referenced in Section 2.8.1 applied to a specific, scalable production system, where the number of farms and sensors can grow over time without requiring a redesign of the underlying pipeline.

2.9.3 Case Study: Smart City Traffic Monitoring Using Streaming Data

A municipal traffic authority collects real-time data from road sensors and traffic cameras via Kafka, partitioned by intersection. A Storm topology aggregates vehicle counts using sliding windows (Section 2.6.3) to estimate current congestion levels and dynamically adjusts traffic signal timing at high-congestion intersections. This case study demonstrates how the windowing and stream-grouping concepts from Sections 2.4.4 and 2.6.3 combine to support a genuinely public-facing, safety-relevant streaming application, where both scalability and fault tolerance are essential to maintaining public trust.

2.9.4 Summary Table of Domains and Case Studies

The table below consolidates the seven domains and three detailed case studies covered in this section, cross-referencing each to the specific scenario or mechanism earlier in this unit that most closely illustrates its core technical challenge, providing a quick-reference map for revision.

Table 2.18: Domain; Representative Challenge; Related Case Study / Scenario

Domain	Representative Challenge	Related Case Study / Scenario
IoT	High-volume sensor ingestion and anomaly detection	2.9.2 Smart Agriculture
Banking	Millisecond-latency, per-account fraud scoring	2.4.5 Payment Fraud Scenario
Social Media	Massive-scale windowed aggregation	2.4.1 Storm's Origins at Twitter
Transportation	Geographic partitioning and matching	2.5 Ride-Hailing Scenario
Healthcare	Guaranteed, low-latency alerting	2.4.6 Reliability Mechanisms
Cybersecurity	Real-time pattern detection over logs	2.9.1 E-Commerce Log Aggregation
Smart Cities	Sliding-window traffic aggregation	2.9.3 Traffic Monitoring

2.9.5 Common Patterns Across Domains and Case Studies

Reviewing the seven domains and three case studies above as a group reveals that they are variations on a small number of recurring architectural patterns, rather than seven or ten unrelated problems. Recognizing these patterns is more valuable for an engineering student than memorizing any single domain's details, since the same patterns will reappear when new, unfamiliar application areas are encountered in industry.

First, nearly every example partitions its Kafka ingestion topic by the entity whose events must be processed together and in order service name, card number, farm, or intersection directly applying the ordering and partitioning principles introduced in Sections 2.3.3 and 2.6.5. Second, nearly every example follows the filter-aggregate-decide structure first identified in Section 2.4.7: discard invalid or irrelevant events, compute a rolling aggregate or index over a meaningful key, and act on that aggregate by raising an alert, making a match, or adjusting a downstream system. Third, every domain that involves a safety-, financial-, or trust-sensitive outcome banking, healthcare, Cyber security, and smart-city traffic places particular emphasis on the reliability and fault-tolerance mechanisms introduced in Sections 2.3.9, 2.4.6, and 2.7.3, since an undetected failure in these domains carries consequences well beyond a minor inconvenience.

This section closes Unit 2's exploration of streaming data processing technologies and concepts. Unit 3 now turns to the complementary problem of data transformation and analysis, introducing Apache Flink and Apache Spark as

processing-layer alternatives to Storm, and examining how the raw, filtered streams produced by pipelines such as those described in this section are cleaned, aggregated, and visualized to support real-time decision-making.

2.10 Unit Summary

This unit introduced streaming data processing as a paradigm distinct from batch processing, driven by principles of continuous flow, low latency, and incremental computation, and shaped by challenges such as ordering, fault tolerance, scalability, and state management. Apache Kafka was examined as the ingestion-layer technology, providing durable, repayable, partitioned messaging built on topics, producers, consumers, and brokers. Apache Storm was examined as the processing-layer counterpart, providing low-latency, guaranteed computation through topologies of spouts and bolts coordinated by Nimbus and ZooKeeper. Building on these two technologies, the unit developed a conceptual toolkit event time versus processing time, windowing, state management, and ordering/consistency for reasoning about real-time streams generally, and applied this toolkit to the practical problem of designing scalable, fault-tolerant pipelines through partitioning, replication, and checkpointing. The unit closed by surveying real-world domains and detailed case studies spanning IoT, banking, social media, transportation, healthcare, and cybersecurity, demonstrating that the same architectural patterns partitioned ingestion and filter-aggregate-decide processing recur across otherwise unrelated industries. This foundation prepares students for Unit 3, which extends these ideas to Apache Flink and Apache Spark and the broader problem of data transformation and analysis.

2.11 Key Terms

Table 2.19: Term; Definition

Term	Definition
Event	A single unit of data representing something that happened at a point in time
Topic / Partition	Kafka's logical channel and its ordered, parallel subdivisions
Producer / Consumer	Clients that publish to / read from a Kafka topic
Broker	A single Kafka server storing and serving partitions
Offset	A message's sequential position within a Kafka partition
Topology / Spout / Bolt	Storm's computation graph and its source/processing components
Tuple	The basic unit of data passed between Storm components

Term	Definition
Stream grouping	The strategy governing tuple distribution to a bolt's parallel tasks
Nimbus / Supervisor	Storm's master node and worker-managing nodes
Acking	Storm's mechanism for tracking and guaranteeing tuple processing
Event time / Processing time	When an event occurred vs. when it is processed
Windowing	Dividing an unbounded stream into finite slices for aggregation
State / Keyed state	Information retained across events, partitioned by an entity key
Replication factor	The number of copies maintained of a Kafka partition
Checkpointing	Periodically saving progress/state to enable failure recovery

2.12 Review Questions

1. Distinguish between batch processing and streaming processing, giving one example application of each.
2. Explain the role of topics and partitions in Apache Kafka, and describe how they enable parallelism.
3. What is a consumer group, and how does it differ from a single, standalone consumer?
4. Describe the roles of Nimbus, Supervisor, and Zoo Keeper in an Apache Storm cluster.
5. Explain the difference between fields grouping and shuffle grouping in Storm, with an example use case for each.
6. Why are Kafka and Storm typically deployed together rather than as alternatives to one another?
7. Distinguish between event time and processing time, and explain why this distinction matters for windowed computations.
8. Compare tumbling, sliding, and session windows, giving a suitable use case for each.
9. What is the difference between local and externalized state in a streaming system, and what trade-off does this choice involve?
10. Describe how Kafka and Storm each achieve fault tolerance, referring to specific mechanisms discussed in this unit.

2.13 Multiple Choice Questions

1. Which of the following best describes a Kafka partition?
 - a. An unordered collection of messages
 - b. An ordered, immutable subdivision of a topic
 - c. A single consumer application

- d. A Zoo Keeper coordination node
- 2. In Storm, which component is responsible for reading data from an external source into a topology?
 - a. Bolt
 - b. Nimbus
 - c. Spout
 - d. Supervisor
- 3. Which stream grouping strategy ensures all tuples with the same field value go to the same task?
 - a. Shuffle grouping
 - b. Fields grouping
 - c. All grouping
 - d. Global grouping
- 4. A rolling five-minute average recomputed every minute is an example of which type of window?
 - a. Tumbling window
 - b. Sliding window
 - c. Session window
 - d. Global window
- 5. Which mechanism allows a Kafka consumer to resume processing after a restart without reprocessing all data?
 - a. Replication factor
 - b. Stream grouping
 - c. Committed offset
 - d. Acking
- 6. What does Storm's acking mechanism primarily guarantee?
 - a. Exactly-once delivery with zero latency
 - b. At-least-once processing of tuples
 - c. Strict ordering across all partitions
 - d. Automatic horizontal scaling

(Answer key available separately for instructor use.)

2.14 Application/Problem-Solving Questions

- 1. Design a Kafka topic structure (topics, partition keys) for an online food-delivery platform that must track order placement, preparation status, and delivery-rider location. Justify your partitioning choices.
- 2. A Storm topology's aggregation bolt is falling behind during peak hours. Propose two distinct remedies, referring to concepts from Sections 2.4.2 and 2.7.1, and explain the trade-off of each.
- 3. For a hospital patient-monitoring system, argue whether Storm's at-least-once guarantee is sufficient, or whether stronger guarantees are warranted, and justify your answer.
- 4. Sketch, in descriptive terms (no code required), an end-to-end pipeline for a smart-city air-quality monitoring system, identifying the ingestion layer,

processing layer, partitioning key, and windowing strategy you would choose.

2.15 Practical/Laboratory Activities

1. Install a single-node Kafka cluster and manually create a topic with multiple partitions; observe message distribution using command-line producer and consumer tools.
2. Trace, on paper or a whiteboard, the complete journey of a message through a Kafka–Storm pipeline for a scenario of your choosing, labeling each component from Sections 2.3 and 2.4.
3. Given a sample dataset of timestamped events, manually compute the results of a tumbling and a sliding window over the data to compare their outputs.
4. Diagram a Storm topology (spouts, bolts, and stream groupings) for a simple word-frequency-counting application, and identify an appropriate grouping strategy for each connection.

2.16 References

- Kreps, J., Narkhede, N., & Rao, J. – *Apache Kafka: A Distributed Messaging System for Log Processing*. LinkedIn Engineering.
- Carbone, P. et al. – *Apache Flink: Stream and Batch Processing in a Single Engine*. IEEE Data Engineering Bulletin.
- Kleppmann, M. – *Designing Data-Intensive Applications*. O'Reilly Media.
- Akidau, T., Chernyak, S., & Lax, R. – *Streaming Systems*. O'Reilly Media.
- Confluent – *Kafka: The Definitive Guide* (2nd ed.), O'Reilly Media.
- Apache Kafka Documentation – kafka.apache.org/documentation
- Apache Flink Documentation – nightlies.apache.org/flink/flink-docs-stable

CHAPTER III

DATA TRANSFORMATION AND ANALYSIS

3.1 Introduction to Data Transformation and Analysis

3.1.1 Concept and Need for Data Transformation in Streaming System

Unit 2 established how streaming data is ingested and reliably moved through a system using Apache Kafka, and how low-latency computation can be applied to that data using Apache Storm. This unit turns to a closely related but distinct concern: once a stream of raw events has been ingested and filtered, how is it cleaned, transformed, aggregated, and ultimately turned into insight that a human decision-maker or a downstream system can act upon? This is the domain of data transformation and analysis, and it is examined in this unit through two further processing-layer technologies, Apache Spark and Apache Flink, both of which extend the ideas introduced in Unit 2 with richer carries, and while Storm can express arbitrary per-event computation through its bolts, it provides comparatively low-level primitives for the kinds of structured, table-like operations joins, grouped aggregations, and windowed statistical summaries that data analysis typically requires. Apache Spark and Apache Flink, introduced in Sections 3.2 and 3.3, were built specifically to provide higher-level, more expressive abstractions for exactly this class of computation, while retaining the scalability and fault-tolerance properties expected of any modern streaming or big-data framework.

3.1.2 Principles of Data Transformation and Analysis

- **Correctness before insight:** Any aggregate or analysis computed over dirty, duplicated, or inconsistent data is unreliable regardless of how sophisticated the analysis itself is; data cleaning (Section 3.5) is therefore a prerequisite to, not an afterthought of, meaningful analysis.
- **Structured abstraction over raw events:** Rather than manipulating individual tuples as in Storm's core API, modern transformation frameworks operate on structured, table-like abstractions Spark's Data Frames and Flink's Table API among them which make common operations such as filtering, joining, and grouping considerably more concise and less error-prone to express.

- **Unified treatment of batch and streaming:** A recurring theme in this unit is that the same transformation logic a filter, a join, an aggregation is often applicable whether the underlying data is a bounded historical dataset or an unbounded live stream, and both Spark and Flink are designed around this unification to varying degrees.
- **Timeliness of insight:** As in Unit 2, the value of an analytical result generally decreases the longer it takes to produce; real-time dashboards (Section 3.7) exist specifically to close the gap between an event occurring and a human being able to act on its implications.

3.1.3 From Streaming Ingestion to Real-Time Analytics

It is useful to place this unit explicitly within the generic streaming pipeline architecture introduced in Section 2.2.3. Where Unit 2 focused on the ingestion layer (Kafka) and an early processing-layer technology (Storm), this unit focuses on two further processing-layer technologies Spark and Flink that specialize in the transformation, aggregation, and analysis stages of that same architecture, and on the storage/serving layer in the specific form of real-time dashboards.

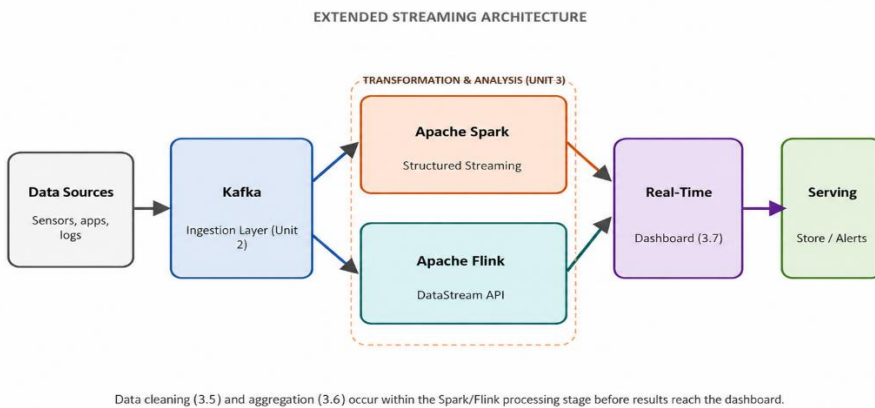


Figure 3.1: Extended streaming architecture Kafka ingestion feeding Spark/Flink transformation and analysis, serving a real-time dashboard.

A brief cross-reference to Unit 2 is useful here: exactly as Kafka and Storm were shown in Section 2.5 to be complementary rather than competing technologies, Spark and Flink are frequently deployed downstream of Kafka in a production system, consuming the same partitioned, ordered topics discussed in Section 2.3.3 and applying the richer transformation and aggregation capabilities introduced in this unit. Section 3.4 revisits this comparison directly once both frameworks have been introduced, mirroring the Kafka-versus-Storm comparison conducted in Section 2.5.

It is worth setting expectations clearly at the outset of this unit: Spark and Flink are not presented here as replacements for Storm, but as additional processing-layer options that a systems designer may select from, according to the specific analytical requirements of a given application, exactly as the Kafka-and-Storm decision checklist in Section 2.5 encouraged a deliberate, requirements-driven choice rather than a default one. A production system may reasonably use Storm for one low-latency alerting pipeline and Flink or Spark for a separate, more analytically rich pipeline, both consuming from the same underlying Kafka topics, and both units of this textbook should be read as building a shared, composable toolkit rather than a sequence of increasingly "better" technologies that supersede one another.

With this framing established, the remainder of this unit proceeds in the same logical sequence used throughout Unit 2: architecture and working mechanism first (Sections 3.2 and 3.3, covering Spark and Flink respectively), followed by a direct comparison (Section 3.4), then the practical techniques of cleaning, aggregating, and visualizing data (Sections 3.5 through 3.7), then the challenges inherent to this domain (Section 3.8), and finally real-world case studies (Section 3.9) that draw together every concept introduced along the way.

3.2 Apache Spark: Architecture and Working

Apache Spark is a distributed data processing engine originally developed at the University of California, Berkeley, and released as an Apache project in 2014. Spark was designed to overcome the performance limitations of Hadoop MapReduce for iterative and interactive workloads by keeping intermediate data in memory rather than repeatedly writing it to disk, and it has since grown into a comprehensive platform supporting batch processing, streaming, machine learning, and graph computation within a single unified engine.

3.2.1 Introduction to Apache Spark

Spark's central design goal is to provide a single, unified engine capable of expressing a wide variety of data-processing workloads batch ETL jobs, iterative machine-learning algorithms, ad-hoc interactive queries, and streaming computations using a common set of abstractions and a common execution engine. This unification stands in contrast to earlier practice, where an organization might operate entirely separate systems for batch processing, streaming, and machine learning, each with its own operational overhead.

- **In-memory computation:** Spark caches intermediate results in distributed memory across the cluster, allowing iterative computations (common in machine learning) and interactive queries to run

substantially faster than disk-based approaches such as classical Map Reduce.

- **Unified APIs:** The same core Data Frame abstraction, introduced in Section 3.2.3, underlies Spark's batch, streaming, and SQL processing modes, meaning skills and code developed for one workload transfer largely unchanged to another.
- **Rich ecosystem:** Spark includes dedicated libraries for SQL queries (Spark SQL), machine learning (MLlib), graph processing (GraphX), and streaming (Structured Streaming), all built atop the same core engine.
- **Language support:** Spark provides APIs in Scala, Java, Python, and R, making it accessible to data engineers and data scientists working in a range of programming environments.

It is instructive to compare Spark directly against the batch-processing framework it was designed to improve upon, Hadoop Map Reduce, since this comparison clarifies exactly what problem Spark's in-memory design solves.

Table 3.1: Aspect; Hadoop Map Reduce; Apache Spark

Aspect	Hadoop Map Reduce	Apache Spark
Intermediate data storage	Written to disk between every map and reduce stage	Kept in distributed memory where possible
Iterative workloads (e.g., machine learning)	Slow, since each iteration re-reads from disk	Fast, since cached data is reused directly from memory
Programming model	Low-level map and reduce functions only	Rich set of transformations, Data Frame/SQL, and dedicated libraries
Streaming support	None (purely batch)	Native, via Structured Streaming (Section 3.2.4)

Beyond its core engine, Spark's broader ecosystem is a significant factor in its widespread industry adoption. Spark SQL allows structured data to be queried using standard SQL syntax directly against Data Frames, MLlib provides distributed implementations of common machine-learning algorithms that operate natively on Spark's Data Frame abstraction, and Graph X (and its successor, Graph Frames) provides primitives for graph-structured computation such as network analysis. Because all of these libraries share the same underlying Data Frame representation and execution engine, a single Spark application can combine structured querying, machine learning, and streaming analytics within one coherent code base for example, a fraud-detection pipeline (examined as a full case study in Section 3.9.1) might use Structured Streaming to ingest transactions, Spark SQL to join against reference data, and an MLlib model to score each transaction's risk, all within the same Data Frame-based application.

3.2.2 Spark Architecture

A Spark application runs as a set of coordinated processes across a cluster, following a driver-and-executor model broadly analogous in spirit, though different in detail, to the Nimbus-and-supervisor model of Apache Storm studied in Section 2.4.2.

- **Driver program:** The process that runs an application's main logic, translates it into a sequence of tasks, and coordinates their execution across the cluster. The driver maintains the overall state of the application and communicates with the cluster manager to acquire resources.
- **Cluster manager:** A separate service such as Apache YARN, Kubernetes, or Spark's own standalone manager responsible for allocating cluster resources (CPU and memory) to Spark applications, analogous in role to the resource-allocation responsibilities of Nimbus in Storm.
- **Executors:** Worker processes launched on cluster nodes that actually execute the tasks assigned by the driver and hold cached data in memory for the duration of the application. Each executor runs multiple tasks concurrently across several CPU cores.

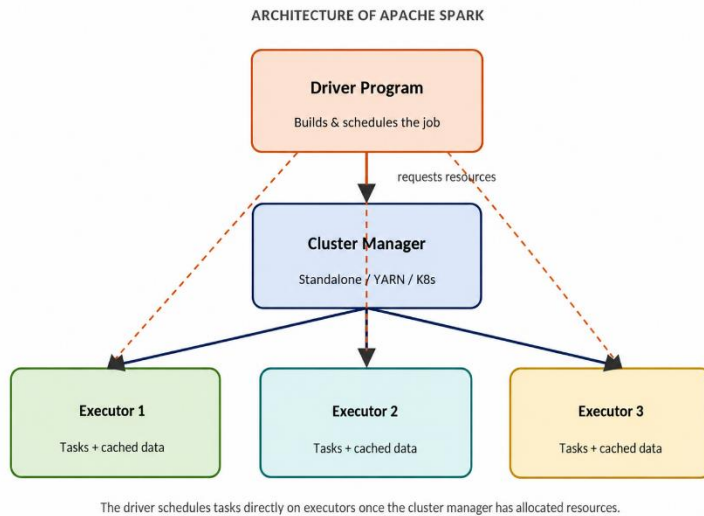


Figure 3.2: Architecture of Apache Spark Driver Program, Cluster Manager, and Executors distributed across worker nodes.

A Spark application's work is organized into jobs, which are further divided into stages, and finally into individual tasks the smallest unit of work, each operating on one partition of the underlying data. This hierarchy allows Spark to parallelize a computation across the cluster in a manner conceptually similar to how Storm decomposes a topology into executors and tasks, discussed in Section 2.4.2, though the two frameworks arrive at their parallel task structure through different underlying execution models.

Spark supports several deployment modes for its cluster manager, and the choice among them typically reflects an organization's existing infrastructure rather than a difference in Spark's own execution logic. Standalone mode uses Spark's own simple, built-in cluster manager, suitable for dedicated Spark clusters. YARN mode integrates with Hadoop's resource manager, allowing Spark to share a cluster's resources alongside other Hadoop-ecosystem workloads. Kubernetes mode runs Spark's driver and executors as containerized pods, aligning with modern, container-based infrastructure and enabling straightforward integration with an organization's broader container-orchestration practices.

Regardless of deployment mode, understanding the driver's role is essential to reasoning about a Spark application's performance. Because the driver builds the execution plan and collects certain results (such as the outcome of a count or collect action), a driver that is under-resourced relative to the size of results it must handle can itself become a bottleneck or even fail with an out-of-memory error, independent of how well-resourced the executors are. This is a common early mistake for students new to Spark: assuming that adding more or larger executors will always resolve a performance problem, when the actual bottleneck may lie in the single, non-parallelized driver process itself, particularly for actions that collect a large result back to the driver rather than leaving it distributed across the executors.

3.2.3 Resilient Distributed Datasets (RDDs) and DataFrames

Spark's original core abstraction is the Resilient Distributed Dataset (RDD) an immutable, partitioned collection of records distributed across the cluster, capable of being rebuilt automatically after a failure using a recorded lineage of the transformations that produced it, rather than relying on data replication alone.

- **Immutability:** An RDD cannot be modified in place; every transformation (such as a filter or a map) produces a new RDD, which simplifies reasoning about parallel computation and enables the lineage-based fault-tolerance approach described in Section 3.2.6.
- **Lazy evaluation:** Transformations on an RDD are not executed immediately; Spark instead builds up a logical execution plan and only performs the actual computation when an action (such as counting or saving results) requires a concrete output, allowing Spark to optimize the overall plan before execution.
- **DataFrames:** A higher-level abstraction built atop RDDs, representing data as a structured table with named, typed columns, similar in spirit to a table in a relational database or a DataFrame in the Python `panda`'s library. DataFrames allow Spark's query optimizer to apply relational-

style optimizations that are not possible with the more general, untyped RDD API.

In modern Spark applications, DataFrames (and the closely related, strongly typed Dataset API in Scala and Java) are the primary interface used by developers, with RDDs remaining as the lower-level foundation on which these higher-level abstractions, and Spark's fault tolerance, are built.

Illustrative Example: *Consider a retail analytics team working with a DataFrame of a day's sales transactions. Applying a filter to remove cancelled orders, and then a groupBy-and-sum to compute total revenue per store, are both transformations: Spark records these steps but performs no actual computation yet. Only when the team requests a concrete result for example, by asking Spark to display or save the resulting per-store revenue table, an action does Spark examine the full chain of recorded transformations, optimize it as a single execution plan, and then execute it across the cluster. This lazy-evaluation behavior allows Spark to apply optimizations, such as combining the filter and aggregation into a single efficient pass over the data, that would not be possible if each transformation were executed eagerly and independently.*

3.2.4 Spark Streaming and Structured Streaming

Spark's approach to processing unbounded data streams has evolved through two successive APIs, both worth understanding since the underlying execution model has significant implications for latency and correctness.

- **Spark Streaming (DStream API):** The original streaming API, which implements the micro-batch model introduced in Section 2.6.1 an incoming stream is divided into a sequence of small, discrete batches (a Discretized Stream, or DStream), each processed using Spark's batch-processing engine as though it were a small, bounded dataset.
- **Structured Streaming:** A newer, higher-level streaming API built directly on the DataFrame abstraction, which allows a developer to express a streaming computation using the same DataFrame operations used for batch processing, while Spark internally handles the incremental, continuous execution of that computation as new data arrives. Structured Streaming also introduced a continuous-processing execution mode that can approach true event-at-a-time latency for a subset of workloads, narrowing, though not entirely closing, the latency gap with Storm's native event-at-a-time model.

Structured Streaming's unification of batch and streaming APIs is one of Spark's most distinctive contributions to the field, directly reflecting the principle of unified batch-and-streaming treatment introduced in Section 3.1.2, and it is the API assumed throughout the remainder of this section unless otherwise noted.

Table 3.2: Aspect; Spark Streaming; Structured Streaming (DStream API)

Aspect	Spark Streaming (DStream API)	Structured Streaming
Core abstraction	DStream (a sequence of RDDs)	Streaming DataFrame
API style	Lower-level, RDD-oriented operations	Declarative DataFrame/SQL operations
Batch/streaming code reuse	Limited; largely separate from batch RDD code	High; same DataFrame code often works for both
Status	Considered a legacy API in modern Spark	The recommended, actively developed streaming API

3.2.5 Spark Data Flow

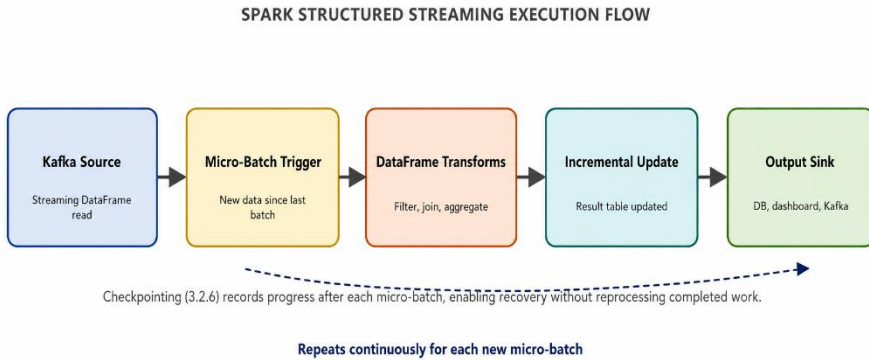


Figure 3.3: Spark Structured Streaming execution flow, from Kafka source through micro-batch processing to output sink.

1. A streaming Data Frame is defined, typically reading from a source such as a Kafka topic, describing the schema and structure of the incoming data.
2. The developer expresses the desired computation filters, joins, aggregations, or windowed operations using standard Data Frame operations, exactly as they would for a static, batch Data Frame.
3. Spark's Structured Streaming engine translates this logical computation into an incremental execution plan, capable of processing only the newly arrived data on each trigger rather than reprocessing the entire stream from the beginning.
4. At each micro-batch trigger interval (or continuously, in continuous-processing mode), Spark executes the incremental plan against the

newly available data, distributing the work as tasks across the cluster's executors.

5. Results are written to an output sink such as a file system, a database, or another Kafka topic using one of several supported output modes (append, update, or complete), depending on the nature of the computation.

6. Spark's checkpointing mechanism, described in Section 3.2.6, records the streaming query's progress, allowing it to resume correctly after a failure or a planned restart.

Illustrative Example: *Consider an online retailer using Structured Streaming to maintain a live count of items currently in each customer's shopping cart, for use in an abandoned-cart notification system. A streaming Data Frame reads add-to-cart and remove-from-cart events from Kafka, keyed by customer ID. On each micro-batch trigger, Spark identifies only the newly arrived cart events since the previous batch, incrementally updates each affected customer's current cart state, and writes the updated state to a fast-access store. A separate, longer-running query periodically scans this store for carts that have remained non-empty and unmodified for more than thirty minutes, triggering a reminder notification. This example illustrates how a single Structured Streaming pipeline can support two different consumers of its output an immediate, low-latency state update and a longer-horizon batch-style scan from the same underlying incremental computation.*

3.2.6 Fault Tolerance in Spark

- **Lineage-based recovery:** Because every RDD (and, by extension, every Data Frame) records the sequence of transformations used to derive it from its original source, Spark can recompute any lost partition of data by replaying this lineage, rather than depending solely on data replication as Kafka does (Section 2.3.9).
- **Checkpointing:** For long-running streaming applications, relying purely on lineage would require replaying an ever-growing chain of transformations after a failure; Spark therefore periodically checkpoints both data and streaming-query progress to reliable storage, bounding the amount of recomputation needed after a restart.
- **Write-ahead logs:** For certain streaming sources, Spark can additionally maintain a write-ahead log of received data, ensuring that data is not lost even if it has not yet been processed or reflected in a checkpoint at the moment of failure.

This lineage-and-checkpoint approach to fault tolerance is architecturally distinct from Storm's per-tuple acking mechanism (Section 2.4.6): rather than tracking

every individual tuple's processing tree, Spark tracks how to regenerate an entire partition of data if it is lost, which aligns naturally with its batch-oriented, Data Frame-centric execution model.

Table 3.3: Mechanism; What It Protects

Mechanism	What It Protects
RDD lineage	Ability to recompute a lost data partition from its recorded transformation history
Streaming checkpoint	Bounded recovery time for long-running streaming queries by periodically persisting progress and state
Write-ahead log	Received-but-not-yet-processed data for certain streaming sources

3.2.7 Spark Working Mechanism — Step-by-Step Operational Flow

The overall operation of a Spark Structured Streaming application can be summarized descriptively as follows, consolidating the concepts introduced throughout this section.

1. A developer defines a streaming Data Frame sourced from an external system such as Kafka, along with the desired transformations and aggregations, expressed declaratively rather than as an explicit sequence of low-level operations.
2. The driver program submits this logical plan to Spark's query optimizer, which produces an efficient incremental execution plan for continuously processing new data.
3. The cluster manager allocates executors across the cluster's worker nodes according to the application's resource requirements.
4. At each micro-batch interval, the driver identifies the newly available data since the previous batch and schedules tasks across the executors to process it in parallel.
5. Executors process their assigned partitions, applying the DataFrame transformations, and report results back through the execution plan toward the configured output sink.
6. Spark records the query's progress via checkpointing, ensuring that if the driver or an executor fails, the application can resume from its last safe position without reprocessing already-completed batches.

7. The application continues running, processing each new micro-batch (or, in continuous mode, each new record) as it becomes available, until explicitly stopped.

Illustrative Example: *A telecommunications company uses Spark Structured Streaming to monitor call-quality metrics arriving continuously from its network infrastructure via Kafka. A streaming DataFrame reads raw call-event records; a series of DataFrame transformations filters out test traffic, joins each event with a reference table of cell-tower locations, and computes a rolling average call-drop rate per tower over five-minute windows. Results are written continuously to a dashboard-serving database. Because this entire computation is expressed using the same DataFrame API used for the company's nightly batch reports, the data-engineering team is able to reuse substantial portions of existing batch logic when building the real-time version, directly illustrating the unified batch-and-streaming principle introduced in Section 3.1.2.*

A Second Illustrative Example: *Contrast the telecommunications scenario above with a simpler, purely periodic use case: a marketing team wants an hourly summary of website conversions by traffic source, updated throughout the day. Because a one-hour delay is entirely acceptable for this use case, the team configures a Structured Streaming query with a one-hour trigger interval rather than the default, more frequent micro-batch trigger. Spark processes a full hour's accumulated data in each batch, trading timeliness the application does not need for a substantially simpler, lower-overhead deployment than would be justified for the telecommunications company's much more time-sensitive call-quality monitoring. This contrast illustrates that Structured Streaming's trigger interval is itself a tunable parameter that should be set according to the specific latency requirement of the application, rather than always minimized by default.*

This concludes the treatment of Apache Spark in this unit. The next section examines Apache Flink, a framework that, as noted in Section 3.2.4, approaches streaming from the opposite architectural direction—natively, rather than as an extension of a batch engine—and Section 3.4 then compares the two directly once both have been established, allowing students to draw their own conclusions about which framework's design trade-offs best fit a given application before consulting the textbook's own decision checklist.

Before moving on, it is worth summarizing why Spark has achieved such broad industry adoption despite Flink's architectural advantages for pure low-latency streaming: for a great many real-world analytics workloads, the dominant practical concern is not shaving the last hundred milliseconds of latency, but rather minimizing the total engineering effort needed to build, maintain, and evolve a data pipeline across batch reporting, ad-hoc SQL analysis, machine

learning, and streaming use cases simultaneously. Spark's unified DataFrame abstraction directly targets this broader concern, which is why, as seen in the banking case study of Section 3.9.1, an organization may deliberately choose Spark over a lower-latency alternative specifically to maximize code reuse across its batch and streaming workloads.

Spark Performance Tuning Considerations

- **Partition sizing:** Just as Kafka topics must be partitioned appropriately for parallelism (Section 2.3.3), Spark Data Frames are internally divided into partitions, and having too few partitions under-utilizes available executor cores, while having too many introduces excessive scheduling overhead relative to the small amount of work each partition represents.
- **Shuffle minimization:** Operations that require reorganizing data across partitions such as a group by on a differently partitioned Data Frame trigger an expensive shuffle across the network between executors; structuring a pipeline to minimize unnecessary shuffles is often the single most effective Spark performance optimization available to a developer.
- **Caching judiciously:** While Spark's ability to cache intermediate Data Frames in memory (Section 3.2.1) is a core strength, caching data that is only used once provides no benefit and consumes memory that could otherwise be used for computation, so caching should be applied deliberately to data that is genuinely reused across multiple actions.
- **Broadcast joins:** When joining a large streaming Data Frame against a small reference table such as the product-category lookup in the clickstream example of Section 2.4.3, reimplemented in Spark broadcasting the small table to every executor avoids an expensive shuffle-based join, substantially improving performance for this common enrichment pattern

3.3 Apache Flink: Architecture and Working

Apache Flink is a distributed stream-processing framework that, unlike Spark's micro-batch-derived streaming model, was designed from its foundations as a true, native streaming engine, with batch processing treated as the special case of a stream that eventually ends. Originating from the Stratosphere research project and becoming an Apache top-level project in 2014, Flink has become particularly well regarded for its sophisticated, low-latency windowing and state-management capabilities, examined in detail in this section.

3.3.1 Introduction to Apache Flink

Where Spark began as a batch engine and extended itself to handle streaming through the micro-batch and, later, continuous-processing models described in

Section 3.2.4, Flink inverts this relationship: its core execution engine processes an unbounded stream of events natively, one event (or small group of events) at a time, and batch processing is implemented as processing a stream that happens to be finite. This architectural choice gives Flink particularly strong support for the event-time semantics, watermarks, and fine-grained windowing introduced conceptually in Section 2.6.

- **True streaming execution:** Flink processes events with very low latency, comparable to or exceeding Storm's event-at-a-time model (Section 2.4.1), while providing higher-level, more expressive APIs than Storm's core spout-and-bolt interface.
- **Sophisticated event-time processing:** Flink provides first-class, built-in support for event time, watermarks, and a rich set of windowing options, making it particularly well suited to applications where correctness with respect to when events actually occurred (Section 2.6.2) is paramount.
- **Exactly-once state consistency:** Through the checkpointing mechanism described in Section 3.3.6, Flink provides exactly-once processing guarantees for application state without requiring the batching trade-offs of Storm's Trident extension (Section 2.4.6).
- **Unified batch and streaming APIs:** Like Spark, Flink increasingly unifies its batch and streaming APIs (notably through its Table API and SQL support), allowing similar logic to be expressed for both bounded and unbounded data.

Because Flink and Storm both aim for native, low-latency stream processing, it is useful to briefly contrast them directly before proceeding, since students may otherwise wonder why both frameworks were included across this textbook's two units.

Table 3.4: Aspect; Apache Storm (Unit 2) Apache Flink

Aspect	Apache Storm (Unit 2)	Apache Flink
Programming abstraction	Low-level spouts and bolts	Higher-level <code>DataStream</code> and <code>Table</code> APIs
Windowing support	Must be implemented manually by the developer	Rich, built-in window and watermark APIs (Section 3.3.4)
State management	Manual, external state required for anything beyond a single tuple	Built-in, automatically checkpointed keyed state (Section 3.3.5)
Fault-tolerance guarantee	At-least-once by default; exactly-once via Trident extension	Exactly-once by default via distributed snapshots (Section 3.3.6)

3.3.2 Flink Architecture

A Flink cluster follows a distributed architecture with clear parallels to both Kafka's broker-based design and Storm's Nimbus-and-supervisor design studied in Unit 2, while introducing its own specific terminology.

- **JobManager:** The master process responsible for coordinating the execution of a Flink application (a job), including scheduling tasks, coordinating checkpoints, and handling recovery after a failure — a role broadly analogous to Storm's Nimbus (Section 2.4.2), though JobManager also directly coordinates the checkpointing mechanism described in Section 3.3.6.
- **TaskManager:** Worker processes that execute the actual computation, each offering a number of task slots representing the parallel processing capacity available on that worker, analogous to Storm's worker processes and executors.
- **JobGraph and ExecutionGraph:** A submitted Flink application is represented internally as a graph of operators, which the JobManager parallelizes into an ExecutionGraph distributed across the available TaskManagers, conceptually similar to how a Storm topology's spouts and bolts are distributed as tasks across supervisor nodes (Section 2.4.2).

ARCHITECTURE OF APACHE FLINK

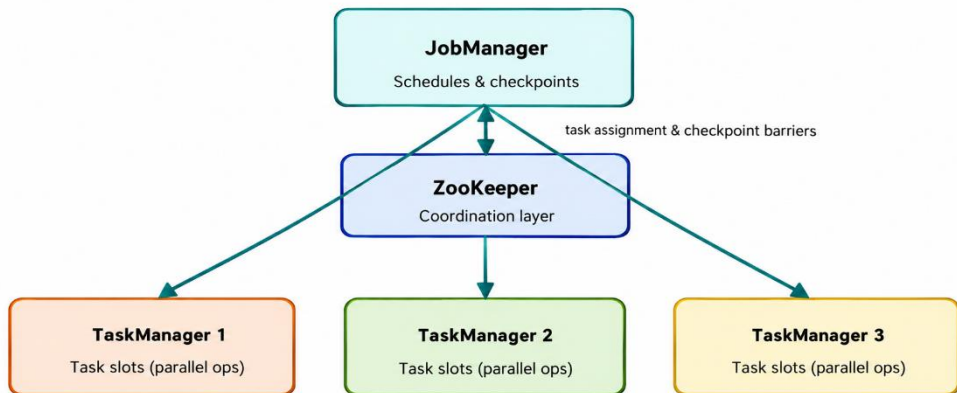


Figure 3.4: Architecture of Apache Flink Job Manager coordinating an Execution Graph distributed across Task Managers with task slots.

A useful point of comparison for students is that Flink's Job Manager-and-Task Manager architecture, Storm's Nimbus-and-Supervisor architecture, and Spark's Driver-and-Executor architecture all solve the same underlying distributed-systems problem coordinating a master process that plans and schedules work against a set of worker processes that execute it using largely analogous mechanisms, differing mainly in the specific execution model (native streaming, per-tuple acking, or micro-batch/lineage) each framework layers on top.

Like Spark, Flink can be deployed in several cluster configurations, including a standalone cluster, integration with YARN, or as containerized deployments on Kubernetes, and this deployment flexibility is one of the practical reasons both frameworks have seen broad adoption across organizations with widely varying existing infrastructure. Flink's ecosystem also includes a Table API and SQL support layered atop the core DataStream API, allowing relationally minded developers to express many streaming computations declaratively, similar in spirit to Spark SQL, and a dedicated library, Flink CEP, for expressing complex event-pattern detection for example, identifying a specific sequence of suspicious login attempts that goes beyond simple windowed aggregation.

3.3.3 Flink Streaming Dataflow Model

Flink represents a computation as a dataflow graph of operators—sources, transformations, and sinks through which a stream of events flows continuously, conceptually similar to a Storm topology's graph of spouts and bolts (Section 2.4.3), but with a richer, more strongly typed set of built-in transformation operators (map, filter, keyBy, window, and others) directly available in its API.

- **Sources and sinks:** A Flink job begins with one or more source operators (for example, a Kafka consumer source) and ends with one or more sink operators (for example, a database or another Kafka topic), mirroring the source/sink terminology introduced generically in Section 2.2.2.
- **Parallel operator instances:** Each operator in the dataflow graph can be executed with a configurable degree of parallelism, with Flink automatically partitioning the stream across parallel instances of downstream operators, similar in spirit to Storm's parallelism hints and stream grouping (Sections 2.4.2 and 2.4.4).
- **KeyBy and keyed streams:** Flink's keyBy operation logically partitions a stream by a specified key, ensuring that all events sharing that key are processed by the same parallel operator instance. Flink's direct counterpart to Storm's fields grouping (Section 2.4.4) and Kafka's key-based partitioning (Section 2.3.3).

This consistent thread of key-based partitioning running through Kafka, Storm, and now Flink is not a coincidence: it reflects the same fundamental requirement, introduced in Section 2.6.5, that any computation requiring per-entity ordering or stateful aggregation must consistently route related events to the same processing unit.

Illustrative Example: *Consider a Flink job monitoring web-application error logs. A source operator reads log events from Kafka; a filter operator discards informational-level log entries, keeping only warnings and errors; a key By operation partitions the remaining stream by service name; a downstream*

windowed operator then counts errors per service over a one-minute tumbling window; and a final sink operator writes any service whose error count exceeds a threshold to an alerting topic. Each of these stages corresponds to a distinct operator in Flink's dataflow graph, and the key *By step* is precisely what allows the windowed count to be computed correctly and independently for each service, mirroring the role fields grouping played in the equivalent Storm-based topology design discussed in Section 2.4.4.

A Second Illustrative Example: Complex event processing extends this pattern further. Using Flink CEP, a security team could define a pattern matching a suspicious sequence of events for a single user three failed login attempts followed within two minutes by a successful login from a different geographic region directly over the keyed event stream, without manually implementing the sequence-tracking logic as custom state (Section 3.3.5). Flink's CEP library handles the pattern-matching bookkeeping internally, emitting a match as a new event whenever the specified sequence is observed, illustrating how Flink's ecosystem extends beyond simple windowed aggregation to expressive, declarative pattern detection over ordered event sequences.

3.3.4 Windowing in Flink

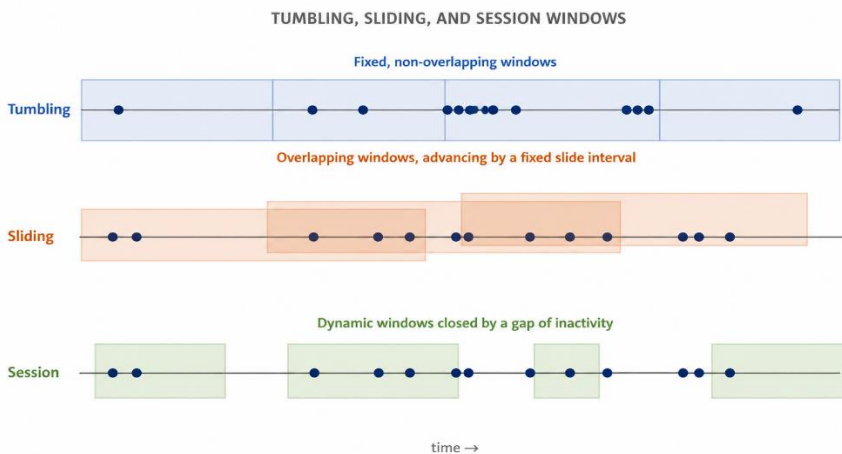


Figure 3.5: Tumbling, sliding, and session windows applied to the same timeline of events.

Section 2.6.3 introduced windowing conceptually, describing tumbling, sliding, and session windows in framework-agnostic terms. Flink provides one of the most complete and configurable implementations of this concept among mainstream streaming frameworks, and this subsection develops that implementation in greater technical depth.

- **Tumbling windows:** Implemented directly as a built-in window assigner, dividing a keyed stream into fixed-size, non-overlapping windows based on event time or processing time, exactly as introduced conceptually in Section 2.6.3.
- **Sliding windows:** Similarly available as a built-in assigner, with independently configurable window size and slide interval, allowing the overlapping-window behavior described in Section 2.6.3 to be expressed directly.
- **Session windows:** Flink's session window assigner groups events by periods of activity separated by a configurable gap of inactivity, directly implementing the session-window concept introduced in Section 2.6.3, with the gap duration itself able to vary dynamically per key if required.
- **Watermarks:** Flink's watermarking mechanism introduced conceptually in Section 2.6.2 allows a developer to specify how event-time progress should be estimated and how long a window should wait for late-arriving data before being considered complete, directly addressing the completeness-versus-timeliness tension discussed in that earlier section.
- **Allowed lateness and side outputs:** Beyond the watermark's initial completeness estimate, Flink allows a window to accept a limited amount of further-delayed data (allowed lateness), and can route data that arrives even later than this into a separate side output stream for special handling, rather than silently discarding it.

Illustrative Example: Recall the factory temperature-sensor scenario introduced in Section 2.6.3. Implemented in Flink, a tumbling window of one minute would be expressed by keying the stream by sensor identifier and applying a one-minute tumbling event-time window with an associated watermark strategy tolerant of a few seconds of network jitter. A sliding five-minute window advancing every minute would use the corresponding sliding-window assigner over the same keyed stream. If a particular reading arrives ten minutes late for example, due to a temporary network outage at the factory Flink's allowed-lateness setting could permit it to still update a recently closed window's result, while a reading arriving hours late might instead be routed to a side output for manual inspection, rather than being silently dropped or causing incorrect results to be emitted.

Table 3.5: Window Type; Flink Assigner Behavior Typical Use Case

Window Type	Flink Assigner Behavior	Typical Use Case
Tumbling	Fixed, non-overlapping intervals	Periodic per-key reporting
Sliding	Fixed size, configurable overlapping slide interval	Smoothed, continuously updated rolling metrics
Session	Dynamically closed after a configurable inactivity gap	Per-episode behavioral analysis (e.g., user sessions)

3.3.5 State Management in Flink

Section 2.6.4 introduced state management conceptually, distinguishing local from externalized state and noting that Storm's core API does not automatically checkpoint bolt state. Flink addresses this gap directly, providing state management as a first-class, built-in feature of the framework rather than leaving it entirely to the application developer.

- **Keyed state:** State that is automatically partitioned according to a stream's key (via the `keyBy` operation described in Section 3.3.3), with Flink transparently managing the storage, access, and checkpointing of this state per key, directly implementing the keyed-state concept introduced generically in Section 2.6.4.
- **Operator state:** State associated with a parallel operator instance as a whole, rather than with a specific key, used less frequently than keyed state but useful for certain source and sink connectors that must track their own internal progress.
- **State backends:** Flink allows the actual storage mechanism for state to be configured for example, kept in memory for the fastest access on smaller state, or spilled to a local embedded database for state too large to fit comfortably in memory while checkpointing this state consistently to durable, remote storage regardless of the chosen backend.

This built-in, automatically checkpointed state management is one of Flink's most significant practical advantages over Storm's core API for applications with substantial per-key state, such as the rolling per-card transaction history described in the banking scenario of Section 2.5, since a Flink implementation of that same scenario would not require the application developer to manually design an external state-persistence mechanism, as would be necessary in Storm.

Illustrative Example: *Reimplementing the banking fraud-detection scenario from Section 2.5 in Flink, the rolling per-card transaction history would be expressed directly as keyed state, associated automatically with each card number via the stream's `keyBy` partitioning. As new transactions arrive for a given card, the corresponding parallel operator instance reads and updates this state directly, with Flink transparently including it in each periodic checkpoint (Section 3.3.6). If the `TaskManager` responsible for that card's state fails, Flink automatically restores the exact state the card's recent transaction history from the most recent checkpoint when restarting the affected task, with no additional code required from the application developer beyond declaring the state itself, in contrast to the manual externalization to a reliable store that Storm's core API would require for the same guarantee.*

3.3.6 Fault Tolerance — Checkpointing and Save points in Flink

- **Distributed snapshots:** Flink periodically takes a consistent, distributed snapshot of the entire application's state across all parallel operator instances using an algorithm that allows this snapshotting to occur without pausing the stream's processing, based on lightweight marker records injected into the data flow.
- **Checkpointing:** These periodic snapshots are stored to durable storage and used automatically to recover an application's exact state after a failure, restoring both the computation's progress and every operator's keyed and operator state to a consistent point, providing the exactly-once state consistency mentioned in Section 3.3.1.
- **Savepoints:** A related but distinct mechanism, savepoints are manually triggered, versioned snapshots that allow a running application to be intentionally stopped and later resumed for example, after a planned upgrade to the application's logic without losing accumulated state, a capability with no direct equivalent in Storm's core reliability model.

This snapshot-based approach differs architecturally from both Storm's per-tuple acking (Section 2.4.6) and Spark's lineage-and-checkpoint approach (Section 3.2.6): rather than tracking individual tuples or relying on recomputation from source data, Flink periodically captures the entire distributed application's state as a single, globally consistent point, which is what enables its strong exactly-once guarantees for both computation and state simultaneously.

Table 3.6: Aspect; Checkpoint; Save point

Aspect	Checkpoint	Save point
Trigger	Automatic, periodic	Manually triggered by an operator
Primary purpose	Recovery after an unplanned failure	Planned stop/resume, e.g., for application upgrades
Lifecycle	Typically retained briefly, superseded by newer checkpoints	Explicitly retained and versioned until manually deleted

3.3.7 Flink Working Mechanism — Step-by-Step Operational Flow

1. A developer defines a Flink job as a dataflow graph of sources, keyed and windowed transformations, and sinks, along with a chosen watermark strategy for event-time processing.
2. The Job Manager receives the submitted job, builds its Execution Graph, and schedules parallel operator instances across the task slots offered by available Task Managers.

3. Source operators begin ingesting events, typically from a Kafka topic, and events are routed to downstream keyed operators according to the key By partitioning scheme.
4. As events arrive, windowed and stateful operators update their state incrementally, guided by the configured watermark strategy to determine when a window's results are ready to be emitted.
5. The Job Manager periodically triggers a distributed checkpoint, capturing a consistent snapshot of every operator's state across the cluster without pausing ongoing processing.
6. Sink operators write results window aggregates, alerts, or enriched records to an external system such as a dashboard-serving database or another Kafka topic.
7. If a Task Manager fails, the Job Manager restarts the affected tasks and restores their state from the most recent successful checkpoint, resuming processing with exactly-once consistency.

Illustrative Scenario: *A logistics company uses Flink to track delivery-vehicle telemetry in real time. The stream is keyed by vehicle ID; a windowed operator computes a five-minute sliding average speed and fuel-consumption rate per vehicle, using event-time processing with a watermark strategy tolerant of brief connectivity gaps as vehicles pass through areas with poor mobile coverage. A second, stateful operator maintains each vehicle's current delivery-route progress, comparing actual position against the planned route and emitting an alert if a vehicle deviates significantly. Checkpointing runs every ten seconds, so that if a Task Manager fails during a large delivery day, the entire application including every vehicle's in-progress route state resume within seconds from its last consistent checkpoint, with no vehicle's tracking data lost or duplicated.*

Comparing this Flink-based operational flow with the equivalent Storm-based flow described in Section 2.4.7 highlights the central theme of this section: both frameworks decompose a computation into a distributed graph of operators executed in parallel across a cluster, and both provide mechanisms for surviving individual machine failures but Flink's native event-time windowing, built-in keyed state, and distributed-snapshot checkpointing provide considerably more of this functionality directly within the framework itself, reducing the amount of custom application logic a developer must write compared to an equivalent Storm topology.

This concludes the treatment of Apache Flink in this unit. Having now examined both major processing-layer technologies covered in this textbook Storm in Unit 2, and Spark and Flink in this unit students are positioned to compare Flink

directly against Spark in the following section, drawing on the architectural, windowing, state-management, and fault-tolerance details developed across both frameworks' dedicated sections.

It is worth noting that Flink's design philosophy treating batch as a special case of streaming, rather than treating streaming as an extension of batch represents a genuinely different starting point from Spark's, even though both frameworks have converged, over successive versions, toward offering broadly similar unified capabilities. This convergence is itself an instructive lesson for students of system design: two frameworks that began from opposite architectural assumptions have each been driven, by the practical demands of real production workloads, toward supporting the same combination of low latency, exactly-once correctness, and unified batch-and-streaming APIs, even though the specific mechanisms each uses to achieve these properties remain visibly rooted in their differing origins.

Flink Performance and Scalability Considerations

- **Parallelism configuration:** As with Spark's partition sizing, setting an operator's parallelism too low under-utilizes the cluster's available task slots (Section 3.3.2), while setting it excessively high relative to the actual workload introduces unnecessary coordination overhead, particularly for operators maintaining substantial keyed state.
- **State backend selection:** For applications with very large keyed state such as the per-card transaction history discussed in Section 3.3.5 at very large scale choosing an appropriate state backend that spills to local disk rather than requiring all state to fit in memory simultaneously can be essential for scalability, at some cost to access latency.
- **Checkpoint interval tuning:** A shorter checkpoint interval (Section 3.3.6) reduces the amount of reprocessing required after a failure but increases the ongoing overhead of the checkpointing process itself; this mirrors the reliability-versus-performance tuning discussion for Storm's message timeout introduced in Section 2.4.6.
- **Watermark and window sizing:** Very large windows, or watermark strategies that tolerate very long lateness (Section 3.3.4), require Flink to retain correspondingly more in-flight state before a window can be finalized, directly trading memory and state-backend load against result completeness.

3.4 Comparative Analysis: Apache Spark vs. Apache Flink

Having examined Spark and Flink individually, this section compares them directly, following the same format used to compare Kafka and Storm in Section 2.5. As with that earlier comparison, the goal is not to declare one framework universally superior, but to understand the trade-offs each embodies and the considerations that should guide a choice between them for a given application.

APACHE SPARK VS. APACHE FLINK — EXECUTION MODEL

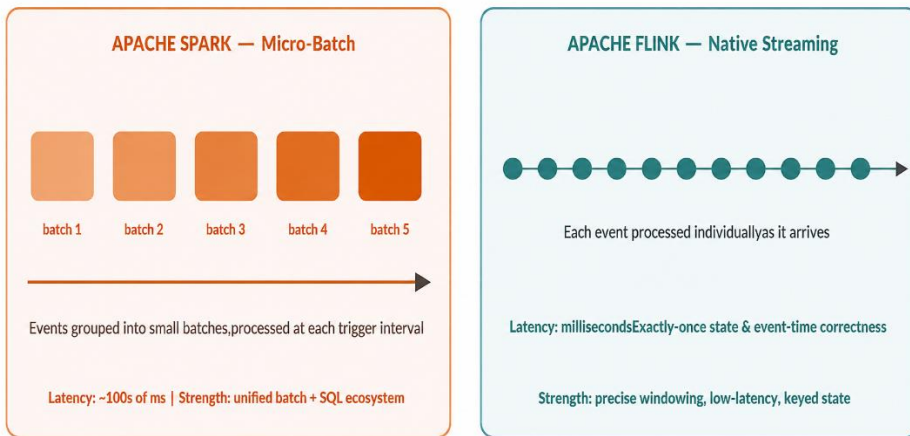


Figure 3.6: Spark's micro-batch execution model contrasted with Flink's native, event-at-a-time streaming model.

Table 3.7: Dimension; Apache Spark; Apache Flink (Structured Streaming)

Dimension	Apache Spark (Structured Streaming)	Apache Flink
Core execution model	Micro-batch (with an optional continuous-processing mode)	Native, event-at-a-time streaming
Typical latency	Hundreds of milliseconds in standard micro-batch mode	Milliseconds, closer to Storm's native model
Core abstraction	DataFrame / Dataset	DataStream / Table API
State management	Managed internally; less exposed for fine-grained custom use	First-class keyed state API (Section 3.3.5), highly configurable
Fault tolerance	Lineage recomputation plus periodic checkpointing	Distributed snapshot-based checkpointing (Section 3.3.6)
Event-time/windowing support	Supported, added onto the DataFrame model	Extensive, native support including watermarks and allowed lateness
Batch processing	Original strength; streaming added later	Treated as a special case of streaming
Ecosystem breadth	Very broad: SQL, MLlib, GraphX, strong data-science adoption	Narrower, more specialized around stream processing

Spark's micro-batch heritage makes it an excellent choice when an application's core need is periodic, structured analysis over recent data with strong integration

into a broader data-science and SQL-oriented ecosystem, and where a latency of a few hundred milliseconds to a few seconds is acceptable. Flink's native streaming architecture makes it the stronger choice when an application demands the lowest possible latency, sophisticated event-time correctness, or fine-grained control over large amounts of custom per-key state for example, the payment-fraud and industrial-monitoring scenarios introduced in Section 2.4, if reimplemented using a processing-layer technology from this unit rather than Storm.

- **Choose Spark when:** the workload naturally spans both batch and streaming use cases, heavy use of SQL or machine-learning libraries is anticipated, and near-real-time (rather than true real-time) latency is acceptable.
- **Choose Flink when:** sub-second latency and precise event-time correctness are critical requirements, or the application maintains substantial custom state that benefits from Flink's built-in, exactly-once state management.
- **Consider both alongside Storm (Unit 2):** for applications already built on Storm, the choice to migrate to Flink or Spark is typically driven by a need for richer, higher-level abstractions (DataFrames, windowing APIs) rather than by raw latency alone, since Storm already provides strong low-latency guarantees for well-tuned topologies.

Illustrative Migration Scenario: *A media-streaming company originally built its viewer-engagement analytics on Apache Storm, following an architecture similar to the social-media analytics domain introduced in Section 2.8.3. As the analytics requirements grew to include increasingly complex windowed joins between viewing events and a large, frequently updated content-metadata catalog, the engineering team found that expressing these joins as hand-written Storm bolts had become difficult to maintain and error-prone, particularly around correctly handling late-arriving events from viewers with unstable connections. The team evaluated both Spark and Flink as replacements and ultimately selected Flink, primarily because its native event-time and windowed-join support (Sections 3.3.3 and 3.3.4) directly replaced a substantial amount of custom Storm logic with declarative, framework-managed operators, while its sub-second latency preserved the responsiveness of the existing viewer-facing features. This migration scenario illustrates a common real-world pattern: the decision to move from Storm to a Unit 3 framework is frequently driven less by a single latency or throughput benchmark and more by the cumulative engineering cost of maintaining increasingly complex custom logic within Storm's lower-level programming model.*

Common Pitfalls When Choosing Between Spark and Flink

- **Over-weighting latency benchmarks in isolation:** A published micro-benchmark showing Flink outperforming Spark on raw event-at-a-time latency is not, by itself, sufficient justification for a system whose actual bottleneck lies elsewhere for example, in a slow external database lookup performed during enrichment (Section 3.6.2), where the choice of streaming engine has comparatively little effect on end-to-end latency.
- **Underestimating operational familiarity:** A team highly proficient in Spark SQL and the broader Spark ecosystem may deliver a correct, maintainable Flink-based system more slowly than an equally correct Spark-based one, simply due to the learning curve, even if Flink is architecturally the better long-term fit; this operational cost should be weighed explicitly rather than ignored.
- **Assuming the choice is permanent:** Because both frameworks consume from and produce to the same Kafka topics (Section 3.1.3), migrating a well-modularized pipeline's processing layer from one framework to the other at a later date, should requirements change, is a significant but not insurmountable undertaking a fact that can reduce the perceived stakes of an initial choice made under incomplete information. As with the Kafka-versus-Storm decision checklist in Section 2.5, students should approach the Spark-versus-Flink choice systematically, working through an application's specific latency, state, and ecosystem requirements rather than defaulting to whichever framework is most familiar.

A Practical Decision Checklist

- Does the application require sub-second latency with precise event-time correctness? If yes, Flink's native streaming model and watermark handling (Section 3.3.4) make it the stronger default choice.
- Does the workload need to share substantial code or infrastructure with existing batch ETL or machine-learning pipelines? If yes, Spark's unified DataFrame API and broad ecosystem (Section 3.2.1) offer a smoother integration path.
- Does the application maintain large amounts of custom per-key state that must survive failures with exactly-once guarantees? If yes, Flink's built-in keyed state and checkpointing (Sections 3.3.5–3.3.6) reduce the amount of custom engineering required compared to Spark.
- Is the team already proficient in SQL and relational-style data manipulation, with less need for the lowest possible latency? If yes, Spark SQL and Structured Streaming provide a comparatively gentle learning curve.
- Does the application need to be intentionally paused and resumed for planned upgrades without losing accumulated state? If yes, Flink's savepoint mechanism (Section 3.3.6) directly supports this operational pattern.

3.5 Data Cleaning Techniques for Streaming Data

Section 3.1.2 established that data cleaning is a prerequisite to reliable analysis. This section examines the specific techniques used to clean streaming data using the frameworks introduced in Sections 3.2 and 3.3, extending the brief mentions of erroneous-reading filtering seen in the Storm-based scenarios of Unit 2 into a more systematic treatment.

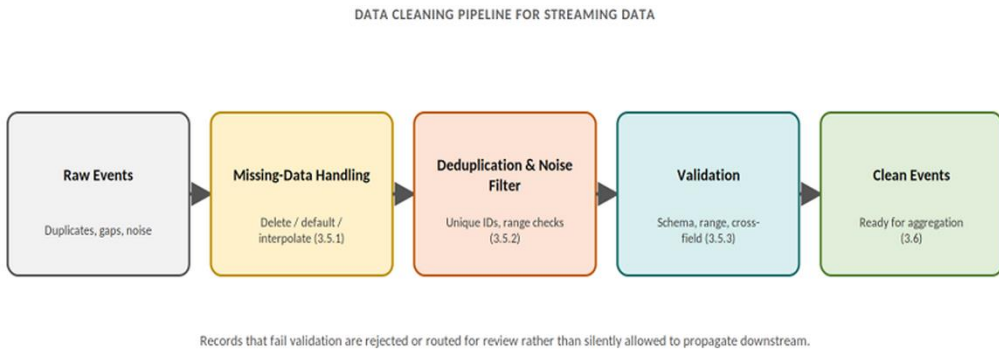


Figure 3.7: Data cleaning pipeline from raw events through missing-data handling, deduplication, and validation to clean events.

3.5.1 Handling Missing and Incomplete Data

Streaming sources frequently produce records with missing fields a sensor that fails to report humidity while still reporting temperature, or a web-analytics event missing a user identifier due to a client-side error.

- **Deletion:** Discarding records with missing critical fields entirely, appropriate when such records are rare and the missing field is essential to the analysis, but risky if applied carelessly to fields that are only occasionally necessary.
- **Default substitution:** Replacing a missing value with a sensible default (such as zero, or a defined "unknown" category), appropriate for non-critical fields where a placeholder does not distort downstream aggregation.
- **Interpolation:** Estimating a missing numeric value from surrounding data points in the same stream for example, interpolating a single missing sensor reading from the readings immediately before and after it appropriate for smoothly varying signals such as temperature or pressure.
- **Carrying forward the last known value:** A common technique for slowly changing state, such as a device's configuration status, where the last successfully received, value is assumed to remain valid until a new value arrives.

The choice among these techniques is highly context-dependent, and an inappropriate choice can itself introduce error: interpolating a missing value in a rapidly changing signal, for instance, may produce a misleadingly smooth result that obscures genuine volatility the analysis was meant to detect.

Illustrative Example: *A weather-monitoring network occasionally loses a single reading from a given station due to a brief transmission fault. For temperature, which changes gradually, interpolating the missing reading from the values immediately before and after is generally safe and produces a negligible distortion. For rainfall, however, which can begin or stop abruptly, the same interpolation approach could incorrectly imply a period of light, continuous rain when the true pattern was a sudden downpour that started and stopped within the missing interval illustrating why the choice of missing-data technique must be grounded in the specific statistical behavior of the field in question, not applied uniformly across all fields in a dataset.*

3.5.2 Handling Duplicate and Noisy Data

- **Duplicate detection:** Streaming systems, particularly those providing at-least-once delivery guarantees (Section 2.4.6), can produce duplicate records during retries after a failure. Deduplication typically relies on a unique event identifier, with a windowed or time-bounded cache of recently seen identifiers used to discard repeats without needing to remember every event ever seen.
- **Noise filtering:** Sensor and log data frequently contain isolated, implausible outlier values caused by transient hardware or transmission faults echoing the erroneous-reading filtering bolt introduced in the industrial-monitoring scenario of Section 2.4.7. Simple range checks, or more sophisticated statistical outlier detection based on a rolling distribution of recent values, are both common approaches.
- **Smoothing:** For genuinely noisy but valid signals (as opposed to erroneous outliers), a rolling average or a more sophisticated smoothing technique can reduce short-term fluctuation while preserving the underlying trend, at the cost of introducing a small amount of lag into the signal.

Illustrative Example: *An e-commerce platform's clickstream ingestion, built on the at-least-once Kafka-and-Storm pipeline studied in Unit 2, occasionally redelivers a click event after a brief consumer restart. Without deduplication, the platform's page-view counts would be inflated during periods of instability, directly skewing the trending-category dashboard discussed in Section 2.4.3. By assigning each click event a unique identifier at generation time and maintaining a short, time-bounded cache of recently seen identifiers within the Spark or Flink transformation stage, the platform can discard redelivered duplicates while keeping the deduplication cache itself small and bounded, since only identifiers from the last few minutes need to be retained for this particular pipeline's redelivery window.*

3.5.3 Data Validation and Consistency Checks

- **Schema validation:** Verifying that each incoming record conforms to an expected structure and set of data types before it proceeds further into the pipeline, catching malformed records early rather than allowing them to cause failures deeper in the processing graph.
- **Range and referential checks:** Verifying that numeric fields fall within physically or logically plausible bounds, and that reference fields (such as a product or customer identifier) correspond to genuinely existing entities in a reference dataset.
- **Cross-field consistency checks:** Verifying logical relationships between multiple fields within the same record for example, that a transaction's recorded end time is not earlier than its recorded start time.

In both Spark and Flink, these validation checks are typically expressed as an early filtering stage in the transformation pipeline, directly analogous to the bot-filtering and erroneous-reading bolts introduced for Storm in Section 2.4.3, but expressed using the higher-level DataFrame or DataStream APIs introduced in Sections 3.2 and 3.3 rather than hand-written, per-tuple logic.

Illustrative Example: *The healthcare patient-monitoring pipeline introduced in Section 3.9.4 applies all three validation categories in sequence. A schema check first confirms that each incoming vital-sign record contains the expected fields with the expected data types, immediately rejecting malformed records that could indicate a device malfunction. A range check then verifies that reported values fall within physiologically plausible bounds for example, rejecting a heart-rate reading of zero from a patient who is otherwise reporting normally, which more likely indicates a sensor disconnection than an actual medical event. Finally, a cross-field consistency check compares the reported blood-oxygen level against the reported heart rate using known physiological relationships, flagging combinations that are individually plausible but jointly implausible for further review before any value is allowed to influence the alerting logic described in Section 3.7.3.*

Table 3.8: Cleaning Technique; Primary Purpose

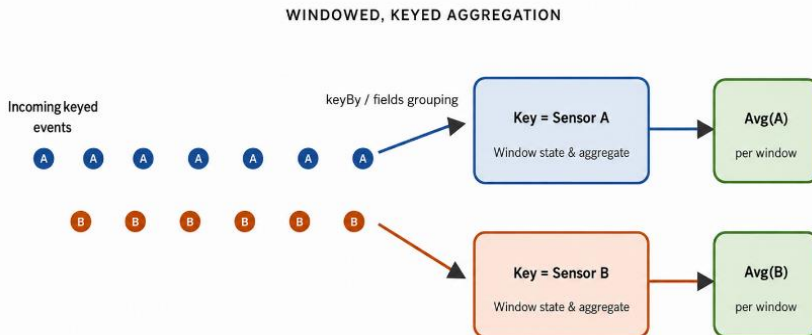
Cleaning Technique	Primary Purpose
Deletion / default substitution / interpolation	Handling missing or incomplete fields (Section 3.5.1)
Deduplication	Removing repeated records from at-least-once delivery (Section 3.5.2)
Noise filtering / smoothing	Handling erroneous or noisy sensor values (Section 3.5.2)
Schema / range / cross-field validation	Rejecting structurally or logically invalid records (Section 3.5.3)

It is worth emphasizing, in closing this section, that data-cleaning logic is not a one-time preprocessing step to be written once and forgotten. As upstream data sources evolve new sensor firmware, a redesigned mobile application, a new payment provider integration the specific cleaning rules appropriate for a given pipeline often need to be revisited and updated. Treating data cleaning as an ongoing, monitored component of a production pipeline, rather than a fixed initial filter, is a mark of a mature streaming-analytics system, and is directly connected to the data-quality and governance challenges examined further in Section 3.8.4.

3.6 Data Aggregation and Real-Time Analysis Techniques

3.6.1 Aggregation Concepts over Streaming Windows

Aggregation is the process of summarizing many individual events into a smaller number of representative statistics, and it is the foundation of nearly every real-time analytics application discussed in this textbook. Applied to a streaming context, aggregation is almost always performed over a window (Sections 2.6.3 and 3.3.4), since an aggregate over an entire, unbounded stream has no meaningful final value



Each key maintains independent window state, so aggregates for Sensor A and Sensor B never interfere with one another.

(Section 3.6.1 — Aggregation Concepts over Streaming Windows)

Figure 3.8: Windowed, keyed aggregation each key (sensor) maintains independent window state and produces its own aggregate.

- **Sum and count:** The most basic aggregates, used for metrics such as total transaction volume or event counts per window.
- **Average, minimum, and maximum:** Commonly used for metrics such as average response time or peak sensor readings within a window.
- **Distinct counts:** Estimating the number of unique entities (such as unique visitors) within a window, which for very large streams is often approximated using specialized probabilistic algorithms rather than

computed exactly, trading a small, bounded error for substantially reduced memory usage.

- **Grouped aggregation:** Computing separate aggregates for each distinct value of a grouping key such as computing a rolling average temperature per sensor, mirroring the keyed, per-entity aggregation pattern established through fields grouping in Storm (Section 2.4.4) and keyBy in Flink (Section 3.3.3).

Table 3.9: Aggregate Function; Typical Application

Aggregate Function	Typical Application
Sum / Count	Total transaction volume; event counts per window
Average / Min / Max	Average response time; peak sensor readings
Approximate distinct count	Unique visitor estimation at large scale
Grouped (keyed) aggregate	Per-sensor, per-customer, or per-region rolling statistics

A further distinction worth drawing out explicitly is between incremental and re-computed aggregation. An incremental aggregate, such as a running sum, can be updated by combining its previous value with only the newly arrived data, without re-examining every event that contributed to it historically this is the approach used internally by both Spark's Structured Streaming stateful aggregations and Flink's keyed state (Section 3.3.5), and it is what makes continuous, large-scale streaming aggregation computationally feasible in the first place. A re-computed aggregate, by contrast, would require re-scanning an entire window's worth of raw events every time a new event arrives, which becomes prohibitively expensive as window sizes or event rates grow. Understanding that both Spark and Flink are specifically engineered to perform aggregation incrementally, rather than by brute-force re-computation, helps explain why these frameworks can sustain the high event rates seen in the case studies of Section 3.9 without requiring proportionally growing computational cost per event.

3.6.2 Real-Time Analytics Techniques

- **Trend detection:** Comparing an aggregate's current value against its recent historical values to identify a meaningful upward or downward trend, rather than reacting to normal, expected fluctuation.
- **Anomaly detection:** Flagging values that deviate significantly from an expected pattern, ranging from simple threshold-based rules (as in the industrial-monitoring scenario of Section 2.4.7) to more sophisticated statistical or machine-learning-based approaches.
- **Join-based enrichment:** Combining a stream of events with reference or dimension data such as joining a stream of clicks with a static table of

product categories to produce a more analytically useful, enriched record, directly extending the enrichment-bolt concept introduced in Section 2.4.3.

- **Stream-to-stream joins:** A more advanced technique, supported by both Spark Structured Streaming and Flink, in which two independent streams (such as ride requests and driver locations in the scenario from Section 2.5) are joined with each other within a bounded time interval, rather than one stream being joined against static reference data.

Illustrative Example: *A stream-to-stream join can be illustrated by revisiting the ride-hailing scenario from Section 2.5, reimplemented using Flink. A stream of ride requests and a stream of driver location updates are both keyed by geographic region, and a windowed join matches each incoming ride request against the most recent driver-location events within the same region and a short time interval (for example, the last thirty seconds), producing a stream of candidate matches for the platform's assignment logic to rank and finalize. Because both streams are unbounded, the join is necessarily bounded by an explicit time interval an unrestricted join between two infinite streams would require unbounded memory to retain every past event from both sides indefinitely.*

Trend and anomaly detection (introduced above) deserve a further practical distinction: a threshold-based anomaly rule, such as the safety-threshold check in the cold-chain scenario of Section 3.6.3, is simple to implement and reason about but can only detect deviations the designer explicitly anticipated when setting the threshold. A trend-based approach, by contrast, can surface a gradual, previously unanticipated drift such as the industrial predictive-maintenance case study's detection of slowly degrading equipment performance in Section 3.9.3 precisely because it compares a metric against its own recent history rather than against a single, fixed value decided in advance. Many production analytics pipelines deliberately combine both approaches: threshold rules for well-understood, hard safety limits, and trend detection for the softer, more gradual patterns that a fixed threshold would miss entirely.

3.6.3 Illustrative Scenario: Aggregating Streaming Sensor Data

Illustrative Scenario: *A cold-chain logistics company monitors the internal temperature of refrigerated shipping containers in transit. Temperature readings from each container stream continuously into a Flink application, keyed by container ID. A five-minute tumbling window computes the average, minimum, and maximum temperature per container, while a separate, longer sliding window (one hour, advancing every ten minutes) computes a smoothed trend used to detect gradual cooling-system degradation that a short window would not reveal. Whenever the average temperature in a five-minute window exceeds a configured*

safety threshold for the specific goods being transported (looked up via a join against a static reference table of shipment types), an alert is emitted to the logistics team. This scenario deliberately combines several concepts introduced across this unit: windowed grouped aggregation (Section 3.6.1), join-based enrichment against reference data (Section 3.6.2), and Flink's native event-time windowing capability (Section 3.3.4), illustrating how these techniques compose naturally in a realistic application rather than being used in isolation.

It is worth noting, as a bridge to the next section, that the numeric aggregates computed in scenarios such as this one are rarely the final product delivered to a human decision-maker; rather, they typically feed directly into the kind of real-time dashboard examined in Section 3.7, which is responsible for presenting these continuously updating aggregates in a form a logistics manager or plant operator can interpret at a glance.

Taken together, Sections 3.5 and 3.6 form the analytical core of this unit: cleaning ensures that a pipeline's inputs are trustworthy, and aggregation converts those trustworthy inputs into the concise, meaningful summaries that any downstream decision whether an automated alert or a human glancing at a dashboard actually depends upon. Neither step is sufficient on its own: an elegantly computed aggregate over dirty data is misleading, while perfectly cleaned data that is never aggregated or summarized provides no practical advantage over the raw event stream studied in Unit 2.

3.7 Real-Time Dashboard Development and Visualization

3.7.1 Principles of Real-Time Dashboard Design

- **Clarity under time pressure:** A real-time dashboard is typically consulted by someone who needs to make a rapid decision, so visual clarity and an immediately interpretable layout matter more than in a static report intended for leisurely reading.
- **Appropriate refresh frequency:** A dashboard's visual refresh rate should be matched to the rate at which the underlying data, and the viewer's ability to act on it, actually changes; refreshing faster than a human can meaningfully process is wasted effort and can even be visually distracting.
- **Highlighting exceptions, not just data:** Effective real-time dashboards draw attention to anomalies and threshold breaches (Section 3.6.2) rather than presenting only raw, undifferentiated numbers, reducing the cognitive burden on the viewer during a high-pressure situation.
- **Consistency with underlying data guarantees:** A dashboard should not visually imply more precision or certainty than the underlying streaming computation actually provides for example, a dashboard drawing on an at-least-once pipeline (Section 2.4.6) should be designed with the

awareness that displayed counts could, in rare cases, be briefly inflated by a duplicate.

- **Progressive disclosure:** A well-designed dashboard presents the most critical, decision-relevant information prominently and immediately, while allowing a viewer to drill down into supporting detail only when needed, rather than overwhelming every viewer with the full depth of available data at all times.

These principles are not merely aesthetic preferences; they directly affect how quickly and accurately a viewer can act on the information presented. A dashboard that buries a critical alert among dozens of routine metrics, or that refreshes so rapidly that a viewer cannot read a value before it changes again, actively works against the goal of enabling a fast, correct decision which is, after all, the entire purpose of building a real-time analytics pipeline in the first place, tracing back to the timeliness-of-insight principle introduced in Section 3.1.2.

A practical checklist consolidating these principles asks: what is the single most urgent decision this dashboard's primary viewer must make, and does the layout make the information needed for that decision immediately visible without scrolling or navigation? Everything else secondary metrics, historical drill-downs, configuration options should be treated as supporting detail, arranged so that it does not compete visually with the primary, decision-critical information identified by this question. Applying this checklist to the four case-study dashboards in Section 3.9 explains their differing designs: a fraud-review dashboard's most urgent decision is which flagged transaction to investigate next, so the alert feed is given visual priority; a ward-monitoring dashboard's most urgent decision is which patient needs immediate attention, so the consolidated alert feed again takes priority over any single patient's detailed trend view.

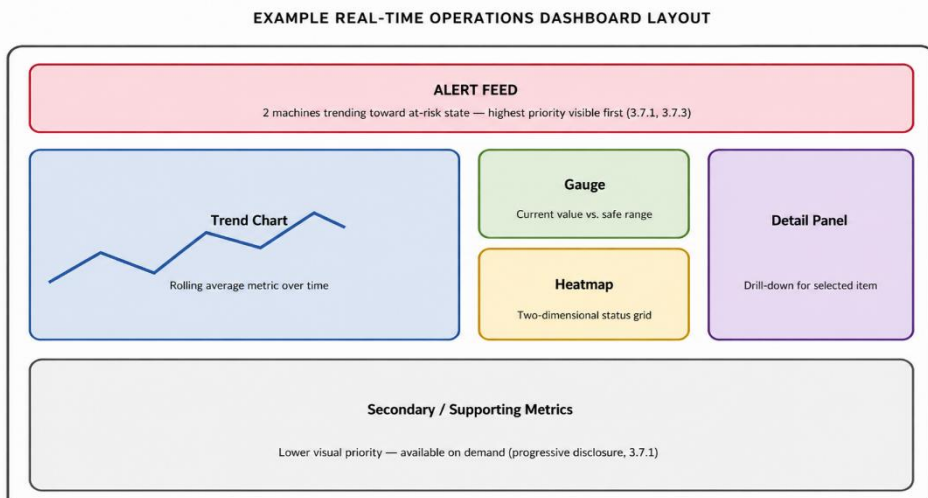


Figure 3.9: Example real-time operations dashboard layout alert feed, trend chart, gauge, heatmap, and detail panel.

3.7.2 Techniques for Developing Real-Time Dashboards

- **Push-based updates:** The dashboard-serving system pushes new data to connected clients as soon as it becomes available (commonly via a persistent connection such as a WebSocket), minimizing the delay between a backend aggregate updating and a viewer seeing that update.
- **Pull-based polling:** The dashboard periodically requests fresh data from the backend at a fixed interval, which is simpler to implement than push-based updates but introduces a small, predictable delay bounded by the polling interval.
- **Pre-aggregated serving layers:** Rather than having the dashboard query raw event data directly, the output of the streaming aggregation pipeline (Section 3.6) is typically written to a dedicated, fast-access serving store optimized for exactly the queries the dashboard needs, avoiding the cost of re-aggregating raw data on every dashboard request.
- **Incremental rendering:** Modern dashboard frameworks update only the specific visual elements affected by new data, rather than redrawing an entire dashboard on every update, both improving performance and avoiding the disorienting effect of a fully refreshing screen.

Illustrative Example: *The industrial predictive-maintenance dashboard introduced in Section 3.9.3 illustrates several of these techniques working together. Rather than having each viewer's browser separately query and re-aggregate raw sensor data, the Flink pipeline writes pre-aggregated, per-machine summary statistics to a fast key-value serving store immediately after each windowed computation completes. The dashboard's backend maintains a push-based connection to each connected client, forwarding updates as soon as the serving store changes. When a single machine's status changes, only that machine's tile on the plant-wide dashboard re-renders, leaving the rest of the screen visually stable an important detail for maintenance engineers who need to notice the one changed tile among dozens of otherwise-unchanging ones.*

3.7.3 Visualization Techniques for Streaming Data

- **Live line and area charts:** Well, suited to displaying a continuously updating trend over time, such as the rolling temperature averages computed in Section 3.6.3, with the newest data appearing at the chart's leading edge.
- **Gauges and single-value indicators:** Appropriate for displaying a single current aggregate value against a known safe range, such as a current system load percentage or the safety-threshold comparison in the cold-chain scenario of Section 3.6.3.
- **Heatmaps:** Useful for displaying an aggregate broken down across two dimensions simultaneously, such as traffic congestion levels across a grid

of city intersections, extending the smart-city case study introduced in Section 2.9.3.

- **Geospatial maps with live markers:** Appropriate for domains involving physical location, such as the ride-hailing and fleet-tracking scenarios introduced in Sections 2.5 and 3.3.7, where a viewer needs to see the current position of many moving entities at once.
- **Alert feeds and notification panels:** A simple but often highly effective visualization for anomaly-driven applications (Section 3.6.2), presenting a chronological list of triggered alerts alongside the more continuous, quantitative visualizations described above.

Illustrative Example: *The healthcare patient-monitoring case study introduced in Section 3.9.4 combines several of these visualization types on a single ward-overview screen: a gauge for each patient's current heart rate and blood-oxygen level against clinically normal ranges, a small trend sparkline beneath each gauge showing the last hour's movement, and a consolidated alert feed at the top of the screen listing any patient currently outside a safe threshold, sorted by urgency. This combination allows nursing staff to absorb the status of an entire ward at a glance from the alert feed alone, while still being able to drill into any individual patient's gauge-and-trend view for more context without needing to navigate to a separate screen.*

3.7.4 Overview of Real-Time Visualization Tools

A wide range of tools exist for building the kind of real-time dashboards described above, differing primarily in their target audience, degree of customizability, and integration approach with the underlying streaming pipeline.

Table 3.10: Category; Typical; Best Suited for Characteristics

Category	Typical Characteristics	Best Suited For
Business-intelligence dashboarding platforms	Low-code, drag-and-drop configuration; broad chart-type support; often near-real-time rather than true real-time	Business stakeholders needing operational visibility without custom development
Time-series-focused visualization tools	Purpose-built for continuously updating line charts and metrics over time; strong integration with monitoring and metrics stores	Engineering and operations teams monitoring system or sensor metrics
Custom web-based dashboards	Built using general-purpose charting libraries within a custom web application; highest flexibility and lowest latency potential	Applications with highly specific visualization needs, such as geospatial live-tracking maps

Selecting among these categories often involves a similar decision process to the framework choices examined in Section 3.4: a business-intelligence platform

minimizes development effort at the cost of some flexibility and true real-time responsiveness; a time-series-focused tool offers a strong middle ground for engineering-oriented monitoring use cases; and a fully custom web dashboard offers maximum control and the lowest achievable latency, at the cost of the greatest development and ongoing maintenance effort. For the case studies examined in Section 3.9, the healthcare and industrial-monitoring dashboards, given their safety-relevant, highly specific visualization needs (ward-wide alert prioritization; per-machine heatmaps), are best served by custom web-based dashboards, while a general business fraud-rate trend, as in the banking case study, may be adequately served by a business-intelligence platform without requiring fully custom development.

Regardless of the specific tool chosen, the underlying architectural pattern remains constant: a streaming pipeline built from the technologies studied in this unit computes and continuously updates a set of aggregates, which are written to a serving layer, from which a dashboard tool retrieves and visually presents them to the end user, closing the loop from raw event to human-interpretable insight that this unit set out to establish in Section 3.1.

3.8 Challenges and Solutions in Data Transformation and Analysis

3.8.1 Latency vs. Accuracy Trade-Offs

As introduced generally in Section 2.1.3, reducing the latency of a computed result frequently comes at the cost of that result's completeness or accuracy, and this trade-off becomes especially concrete in the context of aggregation and analysis.

- **Early, provisional results:** A dashboard may choose to display a window's aggregate as soon as its watermark-defined completion point is reached (Section 3.3.4), accepting that a small number of very late-arriving events might not yet be reflected, in exchange for a timelier result.
- **Approximate algorithms:** As mentioned in Section 3.6.1, approximate techniques for distinct counting and similar aggregates trade a small, bounded statistical error for substantially reduced computation and memory cost, allowing results to be produced faster and at greater scale.
- **Choosing a watermark strategy:** A more aggressive watermark strategy (assuming event time has progressed quickly) produces timelier results but risks prematurely finalizing windows before all relevant data has arrived, directly illustrating the tension introduced conceptually in Section 2.6.2.

3.8.2 Handling Late and Out-of-Order Data

This subsection returns to a challenge first flagged in Section 2.6.2 and 2.6.5, now examined with the concrete mechanisms provided by Spark and Flink.

- **Allowed lateness:** As introduced in Section 3.3.4, Flink permits a window to continue accepting a bounded amount of late data after its watermark-defined completion point, updating previously emitted results if necessary.
- **Side outputs for very late data:** Data arriving after even the allowed-lateness bound can be routed to a separate stream for specialized handling such as manual review or inclusion only in a later batch-based correction rather than being silently dropped or causing an error.
- **Watermark tuning:** Choosing an appropriate watermark delay requires balancing the domain's expected degree of out-of-order arrival (for example, mobile devices reconnecting after a period offline, as in the fitness-tracking example of Section 2.6.2) against the acceptable latency for the application's results.

Both Spark Structured Streaming and Flink provide configurable mechanisms for handling lateness, though Flink's watermark-and-allowed-lateness model, described in Section 3.3.4, is generally regarded as offering more precise, fine-grained control than Spark's comparatively coarser watermarking support.

3.8.3 Scalability Challenges in Analytics Pipelines

- **State growth:** Unlike simple filtering operations, aggregation and windowing (Section 3.6) require maintaining state proportional to the number of distinct keys and open windows, which can grow substantially for applications with a very large number of tracked entities, echoing the state-management trade-offs first introduced in Section 2.6.4.
- **Skewed key distributions:** As discussed for Kafka partitioning in Section 2.7.2, a small number of disproportionately active keys (a small number of very high-traffic sensors, for example) can create hotspots that limit a pipeline's overall scalability even when the majority of keys are evenly distributed.
- **Join costs:** Stream-to-stream and stream-to-reference-data joins (Section 3.6.2) are computationally more expensive than simple filtering or single-stream aggregation, and scaling these operations often requires careful attention to how both sides of the join are partitioned.

Illustrative Example: *The e-commerce recommendation pipeline in Section 3.9.2 illustrates state growth and skew simultaneously. Popular products accumulate view-count state far more rapidly than obscure ones, creating the same kind of hot-key skew discussed for Kafka partitioning in Section 2.7.2, now manifesting as an imbalanced workload across Flink's parallel keyed operator instances. The platform's engineering team addresses this by monitoring per-key state size and, for a small number of extremely popular products, applying a two-stage aggregation pattern first aggregating partial counts across several sub-partitions of the same product, then combining those partial counts into a final total trading*

a small amount of additional complexity for a substantially more even distribution of load across the cluster, a technique conceptually similar to the finer-grained repartitioning solution applied to the ride-hailing scenario's Kafka topic in Section 2.7.2.

3.8.4 Data Quality and Governance Challenges

- **Schema evolution:** As upstream systems change over time, the structure of incoming events may evolve, requiring transformation pipelines to handle new, missing, or renamed fields gracefully rather than failing outright.
- **Auditability:** For regulated domains such as banking (Section 2.8.2) and healthcare (Section 2.8.5), it is often necessary to be able to reconstruct exactly how a given displayed result was computed from its underlying raw events, which requires deliberate attention to lineage and logging throughout the pipeline, beyond what is needed for correctness alone.
- **Data ownership and access control:** As data flows through multiple transformation stages and is exposed through dashboards to a range of stakeholders, ensuring that only appropriately authorized users can see sensitive fields becomes an increasingly important governance concern as a pipeline matures from a prototype into a production system.

These challenges reinforce a theme that has recurred throughout both units of this textbook: building a real-time analytics pipeline is not merely a matter of selecting the fastest available framework, but of carefully balancing latency, correctness, scalability, and governance according to the specific demands of the application domain, as illustrated concretely by the case studies in Section 3.9.

Illustrative Example: *A healthcare organization's patient-monitoring pipeline, extending the scenario introduced in Section 2.8.5, must balance several of these challenges simultaneously. Favoring low latency (Section 3.8.1) is clearly appropriate for triggering an immediate bedside alert, but the same pipeline's monthly compliance reporting requires full auditability (Section 3.8.4) the ability to reconstruct exactly which raw sensor readings contributed to a given historical alert, including any readings that arrived late and were incorporated via Flink's allowed-lateness mechanism (Section 3.8.2). Meeting both requirements from the same underlying pipeline, rather than building two separate systems, requires deliberately retaining sufficient raw-event lineage information alongside the real-time aggregates themselves, illustrating that the challenges introduced in this section are rarely resolved in isolation from one another.*

This section has deliberately revisited, in the specific context of transformation and analysis, several challenges first introduced more generally in Unit 2 the latency-accuracy trade-off from Section 2.1.3, out-of-order data from Section 2.6.2, and scalability from Section 2.7.1 because recognizing a challenge's recurrence

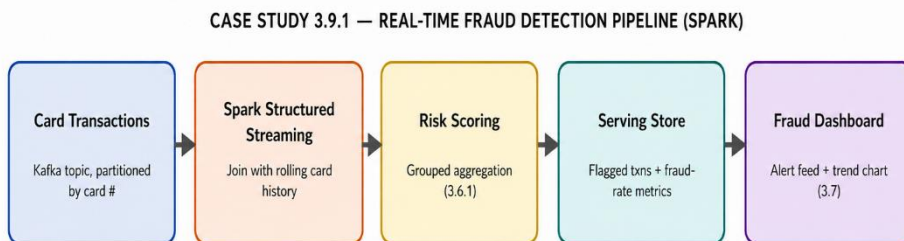
across different layers of the same overall architecture is itself valuable: a solution technique learned for one layer (such as key-based repartitioning to address skew) often transfers directly to another, rather than needing to be reinvented independently for each new framework a student or practitioner encounters.

3.9 Real-World Applications and Case Studies

The four case studies below apply Spark, Flink, and the transformation, aggregation, and dash boarding techniques developed throughout this unit to concrete problems, extending the domain survey conducted for Kafka and Storm in Sections 2.8 and 2.9 into the data transformation and analysis space.

3.9.1 Case Study: Real-Time Fraud Detection Dashboard in Banking (Spark)

Building on the payment-fraud scenario introduced with Storm in Section 2.4.5, a bank implements its fraud-scoring analytics using Spark Structured Streaming, consuming transaction events from Kafka. A series of DataFrame transformations joins each transaction with recent transaction history for the same card (maintained as streaming state), computes a rolling risk score using grouped aggregation (Section 3.6.1), and flags high-risk transactions. Flagged transactions and aggregate fraud-rate metrics are written to a serving database feeding a real-time dashboard (Section 3.7) used by the bank's fraud-review team, who can see both individual flagged transactions and overall fraud-rate trends on a single screen, illustrating the alert-feed-alongside-trend-chart pattern discussed in Section 3.7.3. The bank's data-engineering team specifically chose Spark for this pipeline because the same fraud-scoring logic, expressed as DataFrame transformations, is also reused nightly in a batch mode to retrain the underlying risk model against a full day's transaction history, directly benefiting from the unified batch-and-streaming code reuse discussed in Section 3.2.4.



The same DataFrame-based risk-scoring logic is reused in a nightly batch job to retrain the risk model (Section 3.9.1).

Figure 3.10: End-to-end real-time fraud detection pipeline for the banking case study (Section 3.9.1).

3.9.2 Case Study: Flink-Based Real-Time Recommendation Analytics in E-Commerce

An e-commerce platform uses Flink to power a "customers are viewing" style real-time recommendation feature. Clickstream events, keyed by product ID, feed a Flink application that maintains a sliding-window count of recent views per product (Section 3.6.1), enriched via a join against a product-catalog reference table (Section 3.6.2). Flink's precise event-time windowing (Section 3.3.4) ensures that view counts accurately reflect a genuine five-minute trailing window even when some clickstream events arrive slightly delayed from mobile clients with intermittent connectivity. The resulting trending-product rankings are published to a low-latency serving store queried directly by the platform's website, demonstrating a case where sub-second latency and precise windowing, rather than broad ecosystem integration, were the primary factors favoring Flink over Spark, consistent with the decision framework developed in Section 3.4. The platform's engineering team also applies the deduplication technique described in Section 3.5.2 upstream of the windowed count, since without it, a burst of retried clickstream events during a brief network hiccup could otherwise inflate a product's apparent popularity and distort the recommendation ranking shown to other customers.

3.9.3 Case Study: Industrial Monitoring Dashboard for Predictive Maintenance

Extending the industrial-monitoring scenario first introduced with Storm in Section 2.4.7, a manufacturing plant migrates its equipment-monitoring pipeline to Flink to take advantage of built-in state management (Section 3.3.5) for tracking each machine's rolling vibration and temperature profile. Statistical trend detection (Section 3.6.2) identifies gradual drift in a machine's operating profile that may indicate impending failure, well before a hard threshold would be crossed. Results feed a plant-wide dashboard using the heatmap and gauge visualizations discussed in Section 3.7.3, allowing maintenance engineers to see, at a glance, which machines across the facility are trending toward an at-risk state, enabling proactive maintenance scheduling rather than purely reactive repairs after a hard failure has already occurred. The plant's reliability engineering team specifically credits Flink's exactly-once state guarantees (Section 3.3.6) with eliminating a class of false-negative alerts that had occasionally affected an earlier, Storm-based prototype of the same system, in which an unrecovered task restart had, on rare occasions, silently reset a machine's accumulated trend history.

3.9.4 Case Study: Real-Time Patient Monitoring Analytics in Healthcare

A hospital deploys a Flink-based pipeline to analyze continuous vital-sign data from bedside monitors, building on the healthcare-monitoring domain introduced in Section 2.8.5. The stream is keyed by patient ID, with windowed aggregation computing rolling averages and short-term trend indicators for heart rate, blood oxygen, and blood pressure. Flink's exactly-once state consistency (Section 3.3.6) is considered essential in this domain, since a duplicated or lost reading could directly affect a clinical alerting decision. Data-validation checks (Section 3.5.3) filter out physiologically implausible sensor readings before they reach the alerting logic, and a dashboard presents each patient's current status using the gauge and alert-feed visualizations discussed in Section 3.7.3, giving nursing staff a consolidated, real-time view across an entire ward rather than requiring them to check each bedside monitor individually. Because this pipeline directly affects patient-safety decisions, the hospital's informatics team also maintains a complete audit trail linking every displayed alert back to its originating raw sensor readings, addressing the auditability challenge discussed in Section 3.8.4 as a first-class design requirement rather than an afterthought.

Summary Table of Unit 3 Case Studies

Table 3.11: Case Study; Framework; Key Techniques Illustrated

Case Study	Framework	Key Techniques Illustrated
3.9.1 Banking Fraud Dashboard	Spark	Streaming joins, grouped aggregation, alert + trend dashboard
3.9.2 E-Commerce Recommendations	Flink	Sliding windows, event-time correctness, low-latency serving
3.9.3 Industrial Predictive Maintenance	Flink	Keyed state, trend detection, heatmap/gauge visualization
3.9.4 Healthcare Patient Monitoring	Flink	Exactly-once state, data validation, ward-wide alert dashboard

Common Patterns Across the Unit 3 Case Studies

As with the domain survey concluding Unit 2 (Section 2.8's closing synthesis), it is worth observing what these four case studies share. Each begins with Kafka-based ingestion inherited directly from Unit 2's architecture; each applies at least one data-cleaning step (Section 3.5) before any aggregate is trusted; each computes a keyed, windowed aggregate (Section 3.6) as its core analytical output; and each concludes by feeding a purpose-built real-time dashboard (Section 3.7)

rather than exposing raw numbers directly to end users. The choice between Spark and Flink across the four case studies also tracks the decision checklist developed in Section 3.4 quite closely: the three case studies with the strictest latency, state, or event-time-correctness requirements (3.9.2, 3.9.3, and 3.9.4) all select Flink, while the case study whose requirements more closely resemble a periodic, SQL-friendly analytical workload (3.9.1) selects Spark.

This closing observation is intended to leave students with a transferable design process, not a fixed list of tools: given a new, unfamiliar real-time analytics problem, identify the ingestion requirements (Unit 2), the cleaning and aggregation requirements (Sections 3.5–3.6), the latency and state requirements that determine a Spark-versus-Flink choice (Section 3.4), and the visualization requirements (Section 3.7) and only then select specific technologies to satisfy each identified requirement, rather than starting from a favorite tool and working backward.

As a final synthesis, consider how a genuinely novel application one not among the seven domains surveyed in Section 2.8 or the eight case studies presented across both units might be approached using this process. A university campus wishing to build a real-time occupancy-monitoring system for its libraries and study spaces, for instance, would begin by identifying its ingestion needs (occupancy sensor data, likely via Kafka, partitioned by building or room), its cleaning needs (filtering sensor glitches, as in Section 3.5.2), its latency and state needs (a moderate latency tolerance of perhaps a few seconds, with per-room state suggesting either Spark or Flink would suffice, favoring Spark if the university's existing systems already use it extensively), and its visualization needs (a heatmap of current occupancy across campus, in the style of Section 3.7.3). This kind of systematic, requirements-first reasoning, rather than memorized case-study details, is the lasting skill this unit and its predecessor are intended to build.

3.10 Unit Summary

This unit extended the streaming foundations of Unit 2 into the domain of data transformation and analysis. Apache Spark was examined as a unified batch-and-streaming engine built on the DataFrame abstraction, achieving fault tolerance through RDD lineage and periodic checkpointing, and processing streams primarily through a micro-batch execution model. Apache Flink was examined as a natively streaming engine offering sophisticated, built-in support for event-time windowing, keyed state management, and exactly-once distributed-snapshot checkpointing. Section 3.4 compared the two directly, establishing that the choice between them depends on an application's specific latency, state, and ecosystem

requirements rather than any universal ranking. The unit then developed a practical toolkit for data cleaning, aggregation, and real-time dashboard visualization, before closing with four detailed case studies spanning banking, e-commerce, industrial monitoring, and healthcare demonstrating how ingestion (Unit 2), transformation and analysis (this unit), and visualization combine into complete, real-world streaming analytics systems.

Table 3.12: Concern; Unit 2 (Kafka / Storm) Unit 3 (Spark / Flink)

Concern	Unit 2 (Kafka / Storm)	Unit 3 (Spark / Flink)
Primary role	Ingestion and low-level real-time computation	Structured transformation, aggregation, and analysis
Core abstraction	Topic/partition; topology/spout/bolt	DataFrame/RDD; DataStream/Table API
State handling	Manual/external for anything beyond a tuple	Built-in, higher-level (Flink especially)
Output	Filtered/enriched event streams	Aggregates, dashboards, analytical insight

Taken together, Units 2 and 3 of this textbook trace a complete path from raw, unbounded event generation through to human-interpretable, real-time insight: ingestion and low-level computation (Unit 2) establish a reliable, scalable foundation, while transformation, analysis, and visualization (this unit) convert that foundation into the dashboards and alerts that ultimately justify the entire architecture's engineering cost. Students who have worked through both units should now be equipped to reason about, design, and critically evaluate a real-time data streaming and analysis system for a genuinely novel application domain, well beyond the specific case studies presented in either unit.

3.11 Key Terms

Table 3.13: Term; Definition

Term	Definition
RDD (Resilient Distributed Dataset)	Spark's original immutable, lineage-tracked distributed data abstraction
DataFrame	A structured, table-like data abstraction shared across Spark's batch and streaming APIs
Structured Streaming	Spark's DataFrame-based streaming API using micro-batch or continuous execution
Driver / Executor	Spark's coordinating process and its distributed worker processes

Term	Definition
JobManager / TaskManager	Flink's coordinating process and its distributed worker processes
DataStream	Flink's core abstraction for an unbounded, native stream of events
Watermark	A Flink/Spark mechanism estimating event-time progress to determine window completeness
Allowed lateness	The bounded period after a window's watermark during which late data can still update its result
Checkpointing (Flink)	A distributed, consistent snapshot of application state used for exactly-once recovery
Data deduplication	Removing repeated records, typically using a unique event identifier
Grouped aggregation	Computing separate aggregates for each distinct value of a key
Serving layer	A fast-access store holding pre-aggregated results for dashboard consumption
Lineage	The recorded sequence of transformations used to derive an RDD, enabling recomputation
Broadcast join	Distributing a small reference table to every executor to avoid an expensive shuffle-based join
Trigger interval	The frequency at which a Structured Streaming query processes newly arrived data
Trend detection	Identifying meaningful directional change in a metric relative to its own recent history
Push-based / pull-based update	Dashboard update strategies where the server sends data proactively, or the client polls periodically

3.12 Review Questions

1. Explain why data cleaning is described as a prerequisite to, rather than a step alongside, reliable data analysis.
2. Compare Spark's driver-and-executor architecture with Flink's JobManager-and-TaskManager architecture.
3. Distinguish between an RDD and a DataFrame in Apache Spark.
4. Explain how Spark achieves fault tolerance through lineage, and how this differs from Flink's checkpointing approach.

5. What is a watermark, and why is it necessary for correct windowed computation over event time?
6. Describe the difference between keyed state and operator state in Apache Flink.
7. Under what circumstances would Apache Flink be preferred over Apache Spark for a streaming application, and vice versa?
8. Explain the difference between deduplication and noise filtering as data-cleaning techniques.
9. What design principles distinguish a well-designed real-time dashboard from a static report?
10. Describe the latency-versus-accuracy trade-off in the context of watermark strategy selection.
11. Explain the difference between a checkpoint and a savepoint in Apache Flink, and give a scenario where each would be used.
12. Describe the filter-aggregate-decide pattern as it applies to a data transformation and analysis pipeline, using one of the Section 3.9 case studies as an example.
13. Explain why an unrestricted join between two unbounded streams is not computationally feasible, and describe the technique both Spark and Flink use to make stream-to-stream joins practical.

3.13 Multiple Choice Questions

1. Which Spark abstraction is immutable and tracks its derivation history for fault tolerance?
(a) DataFrame (b) RDD (c) Executor (d) Driver
2. In Apache Flink, which component coordinates task scheduling and checkpointing?
(a) TaskManager (b) Executor (c) JobManager (d) Nimbus
3. Which mechanism allows a Flink window to accept a bounded amount of late-arriving data?
(a) Checkpointing (b) Allowed lateness (c) Lineage (d) Consumer group
4. Which of the following best describes Spark's default streaming execution model?
(a) Native event-at-a-time processing (b) Micro-batch processing (c) Purely batch-only processing (d) Manual polling

5. Which technique is most appropriate for estimating unique visitor counts at very large scale with bounded memory use?
- (a) Exact distinct counting (b) Approximate distinct counting (c) Schema validation (d) Session windowing
6. Which Flink mechanism is manually triggered and intended for planned application upgrades?
- (a) Checkpoint (b) Savepoint (c) Watermark (d) Keyed state
7. Which visualization is best suited to displaying congestion levels across a two-dimensional grid of city intersections?
- (a) Gauge (b) Alert feed (c) Heatmap (d) Line chart
- (Answer key available separately for instructor use.)

3.14 Application/Problem-Solving Questions

1. Design a Flink-based pipeline for a smart-building energy-monitoring system, specifying the keying strategy, window types, and state used, and justify choosing Flink over Spark for this use case.
2. A Spark Structured Streaming dashboard is reporting fraud counts that occasionally spike briefly before correcting downward. Diagnose the likely cause, referring to Sections 2.4.6 and 3.5.2, and propose a fix.
3. For the cold-chain logistics scenario in Section 3.6.3, propose an appropriate watermark and allowed-lateness configuration, and justify your choice.
4. Design a real-time dashboard layout (described textually) for the industrial predictive-maintenance case study in Section 3.9.3, specifying which visualization type you would use for each key metric and why.
5. A Flink application's per-key state has grown large enough to cause noticeable checkpoint delays. Propose two distinct remedies, referring to the performance considerations in Section 3.3 and the scalability challenges in Section 3.8.3, and explain the trade-off of each.
6. For the e-commerce recommendation case study in Section 3.9.2, propose a specific data-cleaning check from Section 3.5 that would be essential to implement before trusting the resulting trending-product rankings, and justify why.

3.15 Practical/Laboratory Activities

1. Given a small sample dataset with missing and duplicate records, manually apply and compare at least two data-cleaning techniques from Section 3.5, and discuss the effect of each on a simple aggregate.
2. Diagram a Flink dataflow graph (sources, keyed operators, windows, sinks) for the e-commerce recommendation case study in Section 3.9.2.
3. Sketch a serving-layer schema suitable for powering the healthcare patient-monitoring dashboard described in Section 3.9.4.
4. Compare, in a short-written analysis, how the same rolling-average computation would be expressed conceptually using Spark's DataFrame API versus Flink's DataStream API, based on the descriptions in Sections 3.2.3 and 3.3.3.
5. Given a short list of raw event records containing at least one duplicate and one out-of-range value, manually apply the deduplication and validation techniques from Section 3.5 and record the cleaned dataset.
6. Design, in descriptive terms, a serving-layer update strategy (push versus pull, and refresh frequency) appropriate for the e-commerce recommendation case study in Section 3.9.2, and justify your choice.

3.16 Further Reading

1. Zaharia, M. et al. "Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing." NSDI.
2. Apache Spark Documentation Structured Streaming Programming Guide.
3. Carbone, P. et al. "Apache Flink: Stream and Batch Processing in a Single Engine." IEEE Data Engineering Bulletin.
4. Apache Flink Documentation DataStream API and State & Fault Tolerance guides.
5. Akidau, T., Chernyak, S., and Lax, R. "Streaming Systems." O'Reilly Media.
6. Zaharia, M. et al. "Spark SQL: Relational Data Processing in Spark." ACM SIGMOD.
7. Carbone, P. et al. "State Management in Apache Flink: Consistent Stateful Distributed Stream Processing." Proceedings of the VLDB Endowment.
8. Armbrust, M. et al. "Structured Streaming: A Declarative API for Real-Time Applications in Apache Spark." ACM SIGMOD.

CHAPTER IV

REAL-TIME DATA PROCESSING

4.1 Introduction to Real-Time Data Processing

Modern digital systems – e-commerce platforms, ride-hailing applications, banking networks, industrial sensors, and social media feeds – generate data continuously and in enormous volumes. Traditionally, organisations collected this data and analysed it hours or days later using batch pipelines. However, many decisions cannot wait: detecting a fraudulent transaction, rerouting a delivery vehicle around traffic, or shutting down an overheating machine must happen within seconds or milliseconds of the triggering event. This need has given rise to real-time data processing (also called stream processing), a computing paradigm in which data is ingested, processed, and acted upon almost immediately after it is produced, rather than being stored first and processed later.

4.1.1 What is Real-Time Data Processing?

Real-time data processing is the continuous acquisition, transformation, and analysis of data as it is generated, with results made available within a bounded and typically very small latency – ranging from a few milliseconds to a few seconds – of the data's arrival. The defining characteristic is not raw speed alone but the processing model: instead of operating on a fixed, finite dataset (a batch), a real-time system operates on an unbounded, continuously arriving sequence of events called a data stream.

A data stream can be thought of as a table that is never “finished” – new rows keep arriving indefinitely. Because the input never ends, real-time systems must process each event (or small micro-batches of events) as it arrives, maintain only the state they need, and emit results incrementally.

4.1.2 Key Characteristics

- **Unbounded data:** Streams have no defined end; the system must run continuously, 24x7.
- **Low latency:** Results are produced within milliseconds to a few seconds of event arrival, not hours later.

- **Event-driven:** Processing is triggered by the arrival of individual events rather than by a schedule.
- **Incremental computation:** Aggregates and models are updated continuously rather than recomputed from scratch.
- **Ordering and time semantics:** Systems must reason about event time (when the event actually occurred) versus processing time (when the system saw it), and must tolerate out-of-order and late-arriving events.
- **Scalability and fault tolerance:** Because the workload never stops, the system must scale horizontally and recover from failures without losing or duplicating data.

4.1.3 A Typical Real-Time Pipeline

Most real-time architectures share a common shape, regardless of the specific tools used:

Stage	Responsibility	Example Technologies
Data Sources	Generate raw events – sensors, applications, logs, clickstreams	IoT devices, web/mobile apps, databases (CDC)
Ingestion Layer	Reliably collect and buffer high-velocity events	Apache Kafka, Amazon Kinesis, Azure Event Hubs
Processing Layer	Transform, filter, aggregate, join, and enrich events in-flight	Apache Flink, Spark Structured Streaming
Storage / Serving Layer	Persist processed results for querying or further use	Cassandra, Druid, Elasticsearch
Consumption Layer	Present insights or trigger actions	Dashboards, alerting engines, microservices

4.1.4 Why Real-Time Processing Matters

The business and engineering motivations for real-time processing include faster and more relevant decision-making, improved customer experience through instant personalisation, early detection of anomalies and fraud, efficient use of infrastructure, and the ability to power applications that are inherently reactive, such as recommendation engines, autonomous vehicles, and industrial control systems.

Example. A ride-hailing application must compute surge pricing for a locality within seconds of demand spiking, match a rider to the nearest available driver in real time, and detect GPS anomalies that might indicate a safety issue – none of these can wait for a nightly batch job.

4.2 Real-Time Data Processing vs Batch Processing

Batch processing and real-time (stream) processing represent two complementary paradigms for handling data, and most mature data platforms use both together – an architecture often referred to as the Lambda architecture, or its simplification, the Kappa architecture.

4.2.1 Batch Processing

Batch processing collects data over a period of time and processes it as one large, finite job. It is well suited to workloads where latency is not critical and where processing benefits from having the complete dataset available – for example, computing monthly payroll or generating end-of-day financial reports. Frameworks such as Apache Hadoop MapReduce and Apache Spark (batch mode) are classical batch engines.

4.2.2 Stream (Real-Time) Processing

Stream processing treats data as a continuous, unbounded flow and computes results incrementally as each event or micro-batch of events arrives. It trades some computational efficiency for dramatically lower latency and immediacy.

4.2.3 Comparative Summary

Aspect	Batch Processing	Real-Time (Stream) Processing
Data scope	Bounded, finite dataset	Unbounded, continuous stream
Latency	Minutes to hours	Milliseconds to a few seconds
Processing trigger	Scheduled (e.g., nightly cron job)	Event-driven, continuous
Fault tolerance	Re-run the failed job	Checkpointing, exactly-once recovery
Typical use cases	Payroll, historical reporting, ML training	Fraud detection, dashboards, monitoring
Representative tools	Hadoop MapReduce, Spark (batch), Hive	Kafka, Flink, Kafka Streams

4.2.4 Choosing Between Batch and Streaming

The choice is rarely absolute. Engineers evaluate how quickly a decision must be acted upon, whether the computation needs the complete dataset or can be computed incrementally, the acceptable cost of running an always-on cluster, and whether late or out-of-order data can be tolerated. Many production systems adopt

a hybrid approach: a streaming layer handles immediate, approximate results, while a batch layer periodically recomputes exact results to correct any drift.

4.3 Real-Time Data Processing Tools and Frameworks

A real-time data platform is typically assembled from several categories of tools: message brokers/ingestion systems, stream-processing engines, and serving/storage systems.

4.3.1 Message Brokers and Ingestion Systems

- **Apache Kafka:** A distributed, partitioned, replicated commit log used as the de-facto standard for high-throughput event ingestion.
- **Amazon Kinesis Data Streams:** A fully managed AWS service offering Kafka-like semantics.
- **Apache Pulsar:** A distributed pub-sub messaging system with multi-tenancy and tiered storage.
- **Azure Event Hubs / Google Pub/Sub:** Cloud-native managed event ingestion services.

4.3.2 Stream-Processing Engines

Framework	Processing Model	Notable Strengths
Apache Flink	True event-at-a-time streaming	Very low latency, rich event-time semantics
Apache Spark Structured Streaming	Micro-batch	Unified batch + streaming API, mature ecosystem
Kafka Streams	Event-at-a-time, embedded library	Lightweight, tightly integrated with Kafka
Apache Storm	Event-at-a-time topology (spouts & bolts)	One of the earliest stream processors
Apache Samza	Event-at-a-time, Kafka-centric	Strong local state management

4.3.3 Serving, Storage and Visualisation

- **Time-series & OLAP stores:** Apache Druid, Click House, Influx DB.
- **Search & analytics:** Elasticsearch/OpenSearch.
- **Key-value stores:** Apache Cassandra, Redis.
- **Dashboards:** Grafana, Kibana, Apache Superset.

A common production stack ingests events with Apache Kafka, processes and enriches them with Apache Flink, stores aggregated results in Druid or Cassandra, and visualises them in Grafana – with alerts routed through Prometheus Alert manager or Pager Duty.

4.4 Real-Time Data Processing with Apache Kafka

Apache Kafka is a distributed event-streaming platform originally developed at LinkedIn and open-sourced in 2011. It is designed to handle high-throughput, fault-tolerant, real-time data feeds, and today serves as the backbone of most streaming architectures.

4.4.1 Core Concepts

- **Topic:** A named, logical channel to which records are published.
- **Partition:** Each topic is split into ordered, immutable, append-only logs, enabling parallel reads and writes.
- **Broker:** A Kafka server that stores partitions and serves client requests.
- **Producer:** A client application that publishes records to topics.
- **Consumer / Consumer Group:** Clients that read records; consumers in a group divide partitions among themselves.
- **Offset:** A monotonically increasing identifier for each record within a partition.
- **Replication:** Each partition is replicated across brokers for durability.
- **KRaft:** Modern Kafka uses a built-in consensus protocol for cluster metadata, removing the ZooKeeper dependency.

4.4.2 Kafka Architecture

Producers publish records to topic partitions. Within a partition, Kafka guarantees strict ordering; across partitions, no ordering is guaranteed unless the application keys records appropriately. Each partition has one broker acting as leader; followers replicate the data. Consumers pull records at their own pace and commit offsets, allowing independent, scalable consumption and replay of historical events.

4.4.3 Setting Up a Topic

```
kafka-topics.sh --create --topic transactions \
  --bootstrap-server localhost:9092 \
  --partitions 6 --replication-factor 3
```

4.4.4 A Simple Kafka Producer (Java)

```
Properties props = new Properties();
props.put("bootstrap.servers", "localhost:9092");
props.put("key.serializer",
StringSerializer.class.getName());
props.put("value.serializer",
StringSerializer.class.getName());
props.put("acks", "all");

KafkaProducer<String, String> producer = new
KafkaProducer<>(props);
ProducerRecord<String, String> record =
    new ProducerRecord<>("transactions", customerId,
transactionJson);
producer.send(record, (metadata, exception) -> {
    if (exception == null)
        System.out.printf("partition %d offset %d%n",
metadata.partition(), metadata.offset());
});
producer.close();
```

4.4.5 A Simple Kafka Consumer (Java)

```
Properties props = new Properties();
props.put("bootstrap.servers", "localhost:9092");
props.put("group.id", "fraud-detection-service");
props.put("auto.offset.reset", "earliest");

KafkaConsumer<String, String> consumer = new
KafkaConsumer<>(props);
consumer.subscribe(Collections.singletonList("transactions
"));
while (true) {
    ConsumerRecords<String, String> records =
consumer.poll(Duration.ofMillis(100));
    for (ConsumerRecord<String, String> record : records)
        processTransaction(record.key(), record.value());
    consumer.commitSync();
}
```

4.4.6 Delivery Guarantees

Guarantee	Description	Typical Configuration
At most once	A record may be lost but never redelivered	acks=0, no retries
At least once	A record is never lost but may be duplicated	acks=all, retries enabled, manual commit

Guarantee	Description	Typical Configuration
Exactly once	Each record affects final state exactly once	idempotent producer + transactional API

Kafka's design makes it suitable not only as a message queue but as a system of record for streaming data, and it is the standard ingestion layer feeding processing engines such as Apache Flink and Spark Structured Streaming.

4.5 Processing Real-Time Data Streams with Apache Flink

Apache Flink is an open-source, distributed stream-processing framework designed for stateful computations over unbounded and bounded data streams at high throughput and low latency.

4.5.1 Why Flink?

- **True streaming engine:** Processes events one at a time with millisecond-level latency.
- **Rich event-time processing:** First-class support for event time, watermarks, and late data.
- **Exactly-once state consistency:** Distributed snapshots (checkpoints) guarantee consistency after failure.
- **Powerful windowing API:** Tumbling, sliding, session, and global windows.
- **Unified batch and stream API:** Both bounded and unbounded computations expressed uniformly.

4.5.2 Core Architectural Concepts

- **JobManager:** Coordinates job execution, scheduling, and checkpointing.
- **TaskManager:** Worker processes that execute data-processing tasks.
- **Parallelism:** Each operator can be split into parallel subtasks across TaskManagers.
- **State backend:** Where operator state is stored (memory, RocksDB, or external), checkpointed for fault tolerance.
- **Watermarks:** Markers representing Flink's estimate of event-time progress, used to close windows safely.

4.5.3 Event Time, Processing Time, and Windows

Flink distinguishes event time, ingestion time, and processing time. For correctness, most analytical applications should use event time, since network delays can cause out-of-order arrival.

Window Type	Definition	Example
Tumbling window	Fixed-size, non-overlapping, contiguous intervals	Count of orders every 1 minute
Sliding window	Fixed-size, overlapping intervals advancing by a slide	Moving average, last 5 min every 30s
Session window	Groups events separated by gaps of inactivity	Grouping clicks into a session
Global window	One window for the entire stream, custom trigger	Custom count-based triggers

4.5.4 A DataStream API Example (Java)

The following reads transaction events from Kafka, keys them by customer ID, computes the total in 1-minute tumbling event-time windows, and flags customers exceeding a threshold.

```
StreamExecutionEnvironment env =
StreamExecutionEnvironment.getExecutionEnvironment();

KafkaSource<Transaction> source =
KafkaSource.<Transaction>builder()
    .setBootstrapServers("localhost:9092")
    .setTopics("transactions")
    .setGroupId("flink-fraud-consumer")
    .setValueOnlyDeserializer(new
TransactionDeserializer())
    .setStartingOffsets(OffsetsInitializer.latest())
    .build();

DataStream<Transaction> transactions =
env.fromSource(source,

WatermarkStrategy.<Transaction>forBoundedOutOfOrderness(Duration.ofSeconds(5))
    .withTimestampAssigner((tx, ts) ->
tx.getEventTime()),
    "kafka-source");

DataStream<CustomerTotal> windowedTotals = transactions
```

```
.keyBy(Transaction::getCustomerId)
.window(TumblingEventTimeWindows.of(Time.minutes(1)
)))
.aggregate(new SumAmountAggregator());

DataStream<Alert> alerts = windowedTotals
    .filter(total -> total.getAmount() > 100000.0)
    .map(total -> new Alert(total.getCustomerId(),
        "High transaction volume detected"));

alerts.sinkTo(alertKafkaSink);
env.execute("Real-Time Fraud Signal Detection");
```

4.5.5 Fault Tolerance: Checkpoints and Savepoints

Flink achieves exactly-once state consistency using the Chandy-Lamport distributed snapshot algorithm. The Job Manager periodically injects checkpoint barriers into the stream; as each operator receives a barrier, it snapshots its state to a durable state backend. If a Task Manager fails, Flink restarts the job from the most recent checkpoint. Save points are similar, manually triggered snapshots used for planned upgrades or rescaling.

Flink vs Spark Structured Streaming. Spark Structured Streaming processes data in configurable micro-batches, which is simpler to reason about and integrates tightly with the Spark ecosystem. Flink processes truly event-at-a-time, generally achieving lower latency and more precise event-time semantics, which is why it is often preferred for latency-critical applications such as fraud detection.

4.6 Implementing Real-Time Data Processing Use Cases

4.6.1 A General Design Process

1. Identify the event sources and the schema of each event.
2. Choose an ingestion topic design in Kafka – decide the partition key so related events land in the same partition.
3. Define the processing logic in Flink – filtering, enrichment, joins, windowed aggregation, and pattern detection.
4. Decide on state management and fault-tolerance requirements.
5. Select a sink for results.
6. Define SLAs for latency, throughput, and correctness, and design monitoring around those SLAs.

4.6.2 Case Study 1 – Real-Time Fraud Detection in Banking

Every card transaction is published to a Kafka topic ‘transactions’, keyed by account ID. A Flink job maintains a rolling state of recent transaction counts, amounts, and locations per account. Rule-based and statistical checks run continuously – for example, an impossible-travel rule, or an amount that is a large multiple of the account’s rolling average. When a rule fires, the job emits an alert to a ‘fraud-alerts’ topic within milliseconds.

Pipeline Stage	Implementation Choice
Ingestion	Kafka topic ‘transactions’, partitioned by account ID
Processing	Flink keyed state + CEP library for pattern rules
Enrichment	Join with a reference stream of merchant risk scores
Output	Kafka topic ‘fraud-alerts’ consumed by a case-management service
Latency target	< 200 MS end-to-end from swipe to decision

4.6.3 Case Study 2 – Real-Time Recommendation Engine

An e-commerce platform streams clickstream events to Kafka. A Flink job maintains a short-term interest profile per user and joins this with a precomputed item-similarity table refreshed by an offline batch job. The streaming job recomputes a ranked recommendation list and writes it to a low-latency key-value store such as Redis, which the website reads on each request – a classic Lambda-architecture pattern.

4.6.4 Common Implementation Pitfalls

- **Choosing a poor partition key:** Leads to hot partitions or broken ordering guarantees.
- **Ignoring late data:** Without a sensible watermark strategy, windows stay open indefinitely or straggling events are dropped.
- **Unbounded state growth:** Keeping per-key state forever without TTL can exhaust memory.
- **Treating streaming as a disguised batch job:** Computing exact global aggregates that are only feasible with approximation algorithms.
- **No idempotency in downstream sinks:** At-least-once delivery can cause duplicate side effects.

4.7 Real-Time Analytics for Streaming Data

Real-time analytics is the continuous computation of metrics, aggregates, and insights from streaming data so dashboards reflect the current state with only a few seconds of delay.

4.7.1 Common Analytical Patterns

- **Windowed aggregation:** Counts, sums, averages computed over tumbling or sliding windows.
- **Top-K and heavy-hitter detection:** Tracking the most frequent items using algorithms such as Count-Min Sketch.
- **Approximate distinct counting:** Estimating unique visitors using HyperLogLog.
- **Stream joins:** Enriching one stream with reference data from another.
- **Anomaly and trend detection:** Comparing live metrics against historical baselines.

4.7.2 Architecture for Real-Time Dashboards

A typical stack ingests events through Kafka, computes rolling aggregates in Flink, writes results to a fast analytical store such as Apache Druid or ClickHouse, and exposes them through Grafana or Superset, refreshed every few seconds.

Example. A logistics company streams GPS pings and delivery events through Kafka. Flink computes rolling metrics – average delivery delay per city, active vehicles, orders per minute – pushed into Druid and displayed on a Grafana dashboard the operations team uses to respond to disruptions immediately.

4.8 Real-Time Decision Making with Streaming Data

4.8.1 Complex Event Processing (CEP)

Complex Event Processing detects meaningful patterns across a sequence of events – for example, “a temperature reading above threshold followed within 30 seconds by a pressure spike.” Frameworks such as Flink CEP allow developers to declare these patterns declaratively.

```
Pattern<SensorEvent, ?> overheatPattern = Pattern
    .<SensorEvent>begin("highTemp").where(e ->
e.getTemperature() > 90.0)
    .next("pressureSpike").where(e -> e.getPressure() >
150.0)
    .within(Time.seconds(30));
PatternStream<SensorEvent> patternStream =
```

```

CEP.pattern(sensorEventStream.keyBy(SensorEvent::getMachineId),
    overhealPattern);

DataStream<MaintenanceAlert> alerts =
    patternStream.select(match ->
        new
MaintenanceAlert(match.get("highTemp").get(0).getMachineId
    ( ),
        "Overheat + pressure spike detected"));

```

4.8.2 Rule Engines and Streaming ML

Decisions can also be driven by declarative business rule engines, or by machine-learning models scored in real time as each event passes through a stream-processing job. Feature values are often maintained incrementally in a feature store updated by the same pipeline.

4.8.3 Human-in-the-Loop vs Fully Automated Decisions

High-confidence, low-risk decisions are usually automated end-to-end, while high-impact, ambiguous decisions often route to a human reviewer, with the streaming system providing urgency and context but leaving the final action to a person.

4.9 Real-Time Monitoring and Alerting

4.9.1 What to Monitor

Category	Representative Metrics
Ingestion (Kafka)	Consumer lag, under-replicated partitions, request latency
Processing (Flink)	Checkpoint duration/failures, backpressure, watermark lag
Infrastructure	CPU, memory, network I/O, GC pauses
Business / application	Events per second, alert counts, error/reject rates
End-to-end latency	Time from event generation to action being taken

4.9.2 Monitoring Architecture

A standard open-source stack pairs Prometheus (scraping metrics from Kafka, Flink, and application code) with Grafana for visualisation and Alertmanager for routing notifications. Log-based monitoring typically uses the ELK/OpenSearch stack. Distributed tracing tools such as OpenTelemetry or Jaeger help follow a single event's journey across services.

4.9.3 Designing Effective Alerts

- **Alert on symptoms, not just causes:** Alert on sustained consumer lag rather than transient blips.
- **Set meaningful thresholds and durations:** Use “for X minutes above threshold” conditions to avoid alert fatigue.
- **Tier alerts by severity:** Distinguish paging alerts from informational alerts.
- **Include actionable context:** Point to a runbook or dashboard.
- **Monitor the monitors:** Ensure the monitoring pipeline itself is observable.

Example alert rule (Prometheus-style). `kafka_consumergroup_lag > 10000` for 5 minutes – severity warning – annotation: “Consumer group is lagging on topic; check downstream processing capacity.” A second rule at `lag > 100000` for 5 minutes would page the on-call engineer.

4.10 Unit Summary

- Real-time (stream) data processing continuously ingests, processes, and acts on unbounded data with latency of milliseconds to seconds, contrasting with batch processing.
- Batch and streaming are complementary; many production systems use both, following Lambda or Kappa architectures.
- The streaming ecosystem organises into ingestion (Kafka, Kinesis, Pulsar), processing (Flink, Spark Structured Streaming, Kafka Streams), and serving/visualisation layers.
- Apache Kafka provides a distributed, partitioned, replicated log with at-least-once, at-most-once, and exactly-once delivery semantics.
- Apache Flink is a true event-at-a-time processor with rich event-time semantics and exactly-once state consistency.
- Real-world use cases combine ingestion, keyed state, windowed aggregation, enrichment, and CEP-style pattern detection.
- Real-time analytics powers live dashboards using windowed aggregation and approximate algorithms.
- Real-time decision making automates responses using CEP, rule engines, or streaming ML, balanced with human review for high-impact decisions.
- Robust monitoring and alerting is essential to operating a reliable, always-on streaming platform.

4.11 Key Terms

Real-time processing; batch processing; Lambda architecture; Kappa architecture; Apache Kafka; topic; partition; broker; consumer group; offset; Apache Flink; Job Manager; Task Manager; watermark; tumbling window; sliding window; session window; checkpoint; save point; Complex Event Processing (CEP); exactly-once semantics; consumer lag.

4.12 Review Questions

Part A – Short Answer

1. Define real-time data processing and list four characteristics that distinguish it from batch processing.
2. What is the difference between event time and processing time in stream processing?
3. Explain the role of a partition in Apache Kafka. Why is the choice of partition key important?
4. What is a consumer group in Kafka, and how does it enable scalable, parallel consumption?
5. Differentiate between at-most-once, at-least-once, and exactly-once delivery semantics.
6. What is a watermark in Apache Flink, and what problem does it solve?
7. List and briefly describe the four types of windows supported by Flink.
8. What is Complex Event Processing (CEP)? Give one real-world example.
9. Name three metrics that should be monitored in a Kafka-based streaming pipeline.
10. What is the Lambda architecture, and how does the Kappa architecture simplify it?

Part B – Long Answer

11. Compare batch processing and real-time stream processing across data scope, latency, fault tolerance, and typical use cases, with two examples of each.
12. Describe the end-to-end architecture of a real-time data pipeline, naming a representative technology for each stage.
13. Explain the architecture of Apache Kafka, including brokers, topics, partitions, and replication. How does Kafka achieve fault tolerance?
14. With the help of a code outline, explain how Flink's `DataStream` API computes a windowed aggregation over a Kafka source stream.
15. Design a real-time fraud-detection pipeline for a digital payments company.

16. Discuss the role of CEP and streaming ML in automated real-time decision making. When would you prefer a human-in-the-loop design?
17. Explain the key metrics and tools involved in monitoring a production streaming pipeline.

Part C – Practical

18. Design a Flink job with an appropriate watermark strategy to compute average temperature per sensor every 30 seconds, given readings arriving out of order by up to 10 seconds.
19. Write a Flink CEP pattern that detects three consecutive failed logins within 2 minutes followed by a successful login from a different IP address.
20. Design an alerting strategy for a Flink job whose checkpointing has started failing intermittently.

4.13 References

- Kreps, J., Narkhede, N., & Rao, J. – *Apache Kafka: A Distributed Messaging System for Log Processing*. LinkedIn Engineering.
- Carbone, P. et al. – *Apache Flink: Stream and Batch Processing in a Single Engine*. IEEE Data Engineering Bulletin.
- Kleppmann, M. – *Designing Data-Intensive Applications*. O'Reilly Media.
- Akidau, T., Chernyak, S., & Lax, R. – *Streaming Systems*. O'Reilly Media.
- Confluent – *Kafka: The Definitive Guide* (2nd ed.), O'Reilly Media.
- Apache Kafka Documentation – kafka.apache.org/documentation
- Apache Flink Documentation – nightlies.apache.org/flink/flink-docs-stable

CHAPTER V

DATA VISUALIZATION AND ANALYTICS

5.1 Introduction to Data Visualization and Analytics

5.1.1 What Is Data Visualization?

Data visualization is the graphical representation of information and data using visual elements such as charts, graphs, maps, and dashboards. It transforms raw numbers, categories, and relationships into shapes, positions, colors, and sizes that the human visual system can interpret quickly and accurately. Where a spreadsheet of ten thousand rows overwhelms working memory, a well-designed line chart lets a viewer perceive a trend in under a second. Visualization is therefore not decoration; it is a cognitive tool that extends the analytical reach of the human mind by offloading pattern recognition onto the eyes rather than forcing it through slow, sequential reasoning.

Data analytics, by contrast, refers to the broader discipline of examining data sets to draw conclusions, test hypotheses, and support decisions. Analytics encompasses descriptive methods that summarize what has happened, diagnostic methods that explain why it happened, predictive methods that forecast what is likely to happen, and prescriptive methods that recommend what should be done next. Visualization and analytics are complementary: analytics produces insight, and visualization communicates that insight in a form that stakeholders can absorb, question, and act upon. Neither is complete without the other. An analysis buried in a table of coefficients rarely persuades an executive; a beautiful chart built on flawed analysis misleads just as effectively as no chart at all.

5.1.2 Why Data Visualization and Analytics Matter

Organizations today generate data at a scale that was unimaginable a generation ago -- transaction logs, sensor readings, clickstreams, social media activity, and operational telemetry accumulate continuously. The value of this data is not in its volume but in the decisions it enables. Effective visualization and analytics practice delivers several concrete benefits:

- **Faster comprehension:** visual encoding exploits pre-attentive processing, the brain's ability to detect features such as color, length, and position in under 250 milliseconds, well before conscious attention engages.
- **Pattern and anomaly detection:** trends, cycles, clusters, and outliers that are invisible in tabular form often become obvious the moment data is plotted.
- **Shared understanding:** a chart or dashboard becomes a common reference point that aligns technical and non-technical stakeholders around the same facts.
- **Evidence-based decision making:** visual analytics reduces reliance on intuition alone by grounding choices in observed patterns and statistically supported inference.
- **Monitoring and early warning:** dashboards that update continuously allow organizations to detect deviations from expected behavior before they escalate into serious problems.

5.1.3 The Data-to-Decision Pipeline

It is useful to think of visualization and analytics as stages in a pipeline that begins with raw data and ends with a decision or action. Data is first collected from operational systems, sensors, or third-party sources. It is then processed: cleaned of errors and duplicates, transformed into consistent formats, and often aggregated or joined with other sources. Analysis follows, applying statistical or algorithmic methods to summarize, explain, or predict. The results are then encoded visually, choosing chart types and layouts appropriate to the message. Finally, the visualization is delivered to an audience who interprets it and, ideally, acts on it. Figure 5.1 depicts this pipeline.

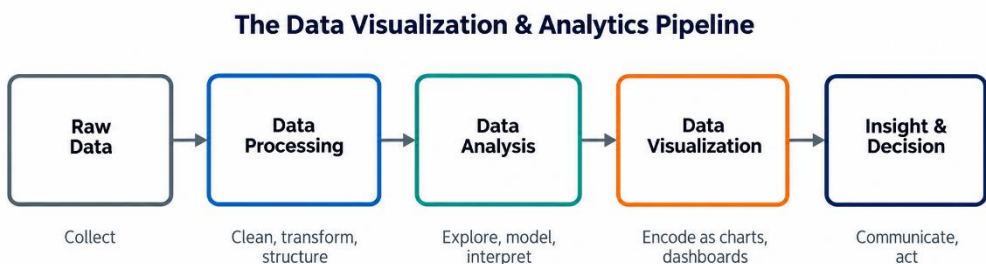


Figure 5.1: The data visualization and analytics pipeline, from raw data collection through to insight and decision.

Each stage introduces its own risks. Poor data collection propagates errors downstream regardless of how elegant the final chart is; a table under "garbage in, garbage out" applies here as strongly as anywhere in computing. Because of this, data visualization and analytics should never be treated purely as a "front-end" or presentation activity performed at the end of a project. The choices made at the collection and processing stages -- what to measure, how often, at what granularity -- directly determine what stories can later be told.

5.1.4 Descriptive, Diagnostic, Predictive, and Prescriptive Analytics

A textbook distinction organizes analytics into four maturity levels, each answering a different question and typically requiring a different visualization style.

Descriptive analytics answers "what happened?" It relies on aggregation and summarization totals, averages, counts and is commonly presented through bar charts, line charts, and summary tables. Diagnostic analytics answers "why did it happen?" It drills into descriptive results using techniques such as correlation analysis, decomposition, and root-cause exploration, often visualized through scatter plots, small multiples, and drill-down dashboards. Predictive analytics answers "what is likely to happen?" using statistical models and machine learning to forecast future values, typically visualized with forecast bands, trend lines, and probability distributions. Prescriptive analytics answers "what should we do?" combining predictive output with optimization or simulation to recommend a course of action, often visualized through scenario comparisons and decision trees.

Most organizations begin with descriptive analytics because it is the least technically demanding and delivers immediate value. As data maturity grows, teams progressively adopt diagnostic, predictive, and finally prescriptive approaches. Real-time streaming analytics, the central theme of later chapters in this text, cuts across all four levels: a streaming dashboard might describe current throughput, diagnose why latency spiked, predict when a queue will overflow, and prescribe an automatic scaling action, all within the same monitoring system.

5.1.5 Types of Data and Their Visual Implications

The visual techniques appropriate for a data set depend heavily on its structure. Quantitative data (continuous or discrete numeric values) is well suited to position- and length-based encodings such as bar charts, histograms, and scatter plots, because position along a common scale is the most accurately perceived visual channel. Categorical or nominal data (unordered labels such as product names or regions) is best shown with distinct hues or grouped bars rather than a continuous color gradient, since there is no inherent order to preserve. Ordinal data (ordered

categories such as satisfaction ratings) benefits from a sequential color scale or ordered axis that visually communicates the ranking. Temporal data (time series) is almost always shown along a horizontal axis, using line charts for continuous trends or heat-map calendars for cyclical patterns. Geospatial data uses maps, choropleths, or symbol maps to exploit the viewer's spatial reasoning. Streaming data, which is the focus of this book, is a special case of temporal data characterized by continuous arrival, unbounded length, and the requirement that analysis and visualization occur with minimal delay after each new observation arrives.

5.1.6 Analytics Maturity and the Rise of Real-Time Visualization

Historically, most business intelligence was built around static, periodic reporting: a nightly batch job populated a data warehouse, and analysts produced weekly or monthly reports. This model remains appropriate for many use cases, such as financial reporting or long-range strategic planning, where decisions do not need to react within seconds. However, an increasing share of business and operational activity now depends on decisions made in near real time -- fraud detection during a payment transaction, dynamic pricing during a demand spike, network monitoring during a security incident, or patient vital-sign monitoring during surgery. These use cases have driven the emergence of streaming data visualization and analytics as a distinct discipline, one that borrows the statistical and visual foundations of traditional analytics but must adapt them to continuously arriving, unbounded data under strict latency constraints. Chapters 3 through 9 of this text build up the concepts, architecture, and techniques required to design, implement, and interpret real-time visual analytics systems.

5.2 Types of Data Visualization Tools

5.2.1 The Business Intelligence and Visualization Tool Landscape

The market for data visualization and business intelligence (BI) tools has matured into several broad categories: full-featured commercial BI platforms, cloud-native analytics services, open-source visualization libraries, and spreadsheet-based tools. Selecting a tool is rarely about finding the single "best" product; it is about matching a tool's strengths to an organization's data sources, technical skill level, budget, and latency requirements. This chapter surveys the major categories and the leading products within each, with particular attention to Tableau and Microsoft Power BI, the two platforms most widely used in enterprise settings, before turning to the tools most relevant to streaming use cases later in the book.

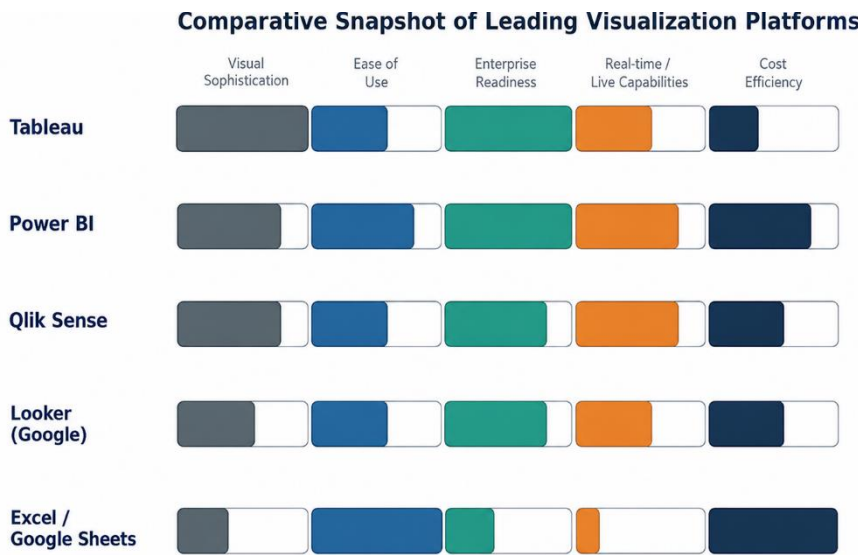


Figure 5.2: Comparative snapshot of leading visualization platforms across five illustrative dimensions.

5.2.2 Tableau

Tableau, originally developed at Stanford University and later acquired by Salesforce, is widely regarded as a pioneer of the modern self-service visualization category. Its core innovation was VizQL, a query language that translates drag-and-drop actions into database queries and renders the results as interactive graphics, allowing analysts without SQL expertise to explore data visually. Tableau's strengths include an unusually deep library of chart types, fine-grained control over visual encoding, strong geospatial mapping, and a large community that shares dashboard templates and extensions. Tableau connects to hundreds of data sources, including relational databases, cloud warehouses, and, through Tableau Server and Tableau Online, live or extracted connections that can be refreshed on a schedule. For streaming-style use cases, Tableau supports "live" connections that query a source directly rather than an in-memory extract, which can approximate near-real-time behavior when paired with a fast-refreshing serving layer, though Tableau itself is not a stream-processing engine.

5.2.3 Microsoft Power BI

Power BI is Microsoft's BI platform, tightly integrated with the Microsoft data and productivity ecosystem, including Excel, Azure, and the Microsoft Fabric analytics platform. Power BI is generally considered to have a gentler learning curve than

Tableau, particularly for users already familiar with Excel formulas, since its DAX (Data Analysis Expressions) language extends familiar spreadsheet logic. Power BI's Power Query engine handles data preparation and transformation, while its data model supports relationships across multiple tables, similar to a lightweight in-memory data warehouse. A distinguishing capability for this book's subject matter is Power BI's native support for streaming datasets and real-time dashboards: through Azure Stream Analytics, Event Hubs, or the Power BI REST API, data can be pushed into a Power BI dataset and displayed on tiles that update automatically without a manual refresh, making it one of the more accessible entry points into real-time visualization for organizations already invested in the Microsoft stack.

5.2.4 Other Prominent Platforms

Several other tools are commonly encountered in professional practice. Qlik Sense uses an associative in-memory engine that allows users to explore relationships across a dataset without pre-defined query paths, which some analysts find more flexible for ad hoc exploration than Tableau's or Power BI's more query-driven models. Looker, now part of Google Cloud, takes a "governed self-service" approach: it defines a semantic modeling layer (LookML) on top of a database, ensuring that metrics such as "revenue" or "active users" are calculated consistently across every report, which is valuable in large organizations where inconsistent metric definitions cause confusion. Google's Looker Studio (formerly Data Studio) offers a free, lightweight alternative aimed at marketing and web analytics use cases, with native connectors to Google Analytics, Google Ads, and Big Query. Amazon Quick Sight is AWS's native BI service, notable for its pay-per-session pricing model and tight integration with AWS data services such as Redshift, Athena, and Kinesis, making it a natural choice for organizations building streaming pipelines on AWS. Domo and Sisense target embedded analytics and operational dashboards for mid-sized organizations, emphasizing rapid deployment and pre-built connectors.

5.2.5 Open-Source and Programmatic Visualization

Beyond commercial BI suites, a large ecosystem of open-source libraries gives data scientists and engineers fine-grained programmatic control over visualization, which is often essential for custom real-time applications that off-the-shelf BI tools cannot fully support. In the Python ecosystem, Matplotlib provides low-level plotting primitives; Seaborn builds statistical graphics on top of Matplotlib; Plotly and Bokeh generate interactive, web-ready charts and support streaming updates through callback mechanisms; and Dash and Streamlit allow developers to

assemble full interactive dashboards, including live-updating ones, using primarily Python code. In the JavaScript ecosystem, D3.js remains the most powerful and flexible library for custom, animated, data-driven visualizations, though it demands considerably more development effort than higher-level charting libraries such as Chart.js, High charts, or ECharts, several of which include built-in support for streaming data series that append new points without a full re-render. Grafana deserves special mention as the de facto standard for visualizing time-series and streaming operational data, particularly metrics collected from systems such as Prometheus, InfluxDB, Elasticsearch, and Kafka-based pipelines; its panel-based dashboards refresh on configurable intervals as short as a few seconds, making it a natural fit for the real-time monitoring use cases discussed in Chapters 4 and 6.

5.2.6 Spreadsheet Tools

Microsoft Excel and Google Sheets remain the most widely used data analysis tools in the world by sheer number of users, and their charting capabilities, while limited compared to dedicated BI platforms, are more than adequate for small, static, or infrequently updated datasets. Google Sheets, in particular, can approximate lightweight real-time dashboards through the Google Apps Script API, which can poll external data sources on a timed trigger and update a chart automatically, though this approach does not scale to the volumes or latency requirements of true streaming architectures.

5.2.7 Choosing a Tool: A Framework

Rather than ranking tools abstractly, practitioners should evaluate candidates against the specific requirements of their project. Table 5.1 summarizes the key selection criteria.

Table 5.1: Criterion; Key Questions; Tools That Typically Excel

Criterion	Key Questions	Tools That Typically Excel
Data source connectivity	Does the tool connect natively to our databases, cloud warehouse, or streaming platform?	Power BI (Azure), Quick Sight (AWS), Tableau (broad connector library)
Latency requirement	Do we need sub-minute refresh, or is daily/weekly reporting sufficient?	Grafana, Power BI streaming datasets, custom D3/Plotly dashboards
Audience technical skill	Will end users self-serve, or will a central team build all reports?	Tableau and Qlik for self-service; Looker for governed semantic layers

Criterion	Key Questions	Tools That Typically Excel
Governance and consistency	Must metric definitions be enforced centrally across teams?	Looker, Power BI with a shared semantic model
Budget and licensing	Is this an enterprise deployment or a small team/individual project?	Looker Studio and open-source stacks for low budget; Tableau/Power BI Premium for enterprise
Customization needs	Do we need bespoke, highly interactive, or animated visuals beyond standard chart types?	D3.js, Plotly, Bokeh

5.2.8 Chapter Summary

This chapter surveyed the principal categories of data visualization tools: enterprise BI platforms led by Tableau and Power BI, competitor platforms such as Qlik Sense, Looker, and Amazon Quick Sight, open-source and programmatic libraries such as Matplotlib, Plotly, D3.js, and Grafana, and spreadsheet-based tools. It proposed a practical framework for tool selection based on data connectivity, latency requirements, audience skill, governance needs, budget, and customization requirements. The next chapter turns to a topic that increasingly determines which of these tools is appropriate: the nature of streaming data itself.

5.3 Understanding the Visualization and Analysis of Data Streaming

5.3.1 Defining Streaming Data

Streaming data is data that is generated continuously, typically by many independent sources, and delivered incrementally as a sequence of events rather than as a fixed, complete dataset. Unlike batch data, which exists as a bounded file or table that analysis can traverse repeatedly from beginning to end, a stream is unbounded: at any moment there is no "last row," because new events may always arrive. Examples include sensor telemetry from industrial equipment, clickstream events from a website, trade ticks from a stock exchange, GPS pings from a fleet of vehicles, and log lines emitted by application servers. Streaming analytics is the practice of computing statistics, detecting patterns, and generating visual output from this continuously arriving data, typically with the expectation that results are available within seconds of the underlying events occurring.

5.3.2 Batch versus Stream Processing

The distinction between batch and stream processing is foundational to everything that follows in this book. Batch processing operates on a bounded dataset: a job

reads a complete file or table, computes results, and terminates. Batch jobs are typically run on a schedule hourly, nightly, or weekly and optimize for throughput and completeness rather than latency. Stream processing operates on an unbounded, continuously arriving sequence of events: a job runs indefinitely, processing each event (or small micro-batch of events) as it arrives, and optimizes for low latency, often at some cost to the completeness or exactness of results, since a stream processor cannot wait indefinitely for "all the data" before producing an answer. Table 5.2 summarizes the principal differences.

Table 5.2: Dimension; Batch Processing; Stream Processing

Dimension	Batch Processing	Stream Processing
Data boundedness	Finite, complete dataset	Unbounded, continuously arriving
Typical latency	Minutes to hours (or longer)	Milliseconds to a few seconds
Processing model	Read all, compute, write all	Process each event or micro-batch as it arrives
Fault tolerance approach	Re-run the whole job	Checkpointing and replay from a durable log
State management	Entire dataset available in memory or on disk	Bounded state (windows, aggregates) must be maintained incrementally
Typical use cases	Financial close, historical reporting, model training	Fraud detection, monitoring, alerting, live dashboards

5.3.3 Core Concepts in Stream Processing

Several concepts recur throughout streaming systems and are essential vocabulary for anyone building or interpreting streaming visualizations.

An event is a single, timestamped record representing something that happened, such as a sensor reading or a page view. A stream is an ordered, unbounded sequence of events. Event time is the timestamp at which an event actually occurred in the real world, while processing time is the timestamp at which the system observes and processes the event; these two clocks can diverge due to network delay, buffering, or out-of-order delivery, and reconciling them correctly is one of the more subtle challenges in stream processing. Windowing is the technique of grouping an unbounded stream into finite chunks for aggregation, since operations such as "compute the average" have no meaning over an infinite stream; common window types include tumbling windows (fixed, non-overlapping intervals, such as "every 1 minute"), sliding windows (fixed-length intervals that advance by a smaller step, producing overlapping windows, such as "the last 5

minutes, updated every 10 seconds"), and session windows (windows defined by periods of activity separated by gaps of inactivity, useful for modeling user sessions). Watermarks are a mechanism for handling out-of-order and late-arriving events: a watermark is a heuristic estimate of "how far behind" the system believes it has received all events up to a given event time, allowing the processor to decide when a window can be safely closed and emitted, while still providing a policy for handling stragglers. Backpressure describes the condition where a downstream component cannot keep pace with the rate of incoming events, and it must be managed -- through buffering, load shedding, or elastic scaling -- to prevent the system from falling progressively further behind or crashing.

5.3.4 Streaming Platforms and Architecture Patterns

Most production streaming systems are built around a small number of architectural building blocks. A message broker or log-based platform, most commonly Apache Kafka, Amazon Kinesis, or Google Cloud Pub/Sub, ingests events from producers and makes them available to multiple independent consumers, decoupling data producers from data consumers and providing durability and replay. A stream processing engine, such as Apache Flink, Apache Spark Structured Streaming, or ksqlDB, subscribes to these topics and performs transformations, aggregations, joins, and windowed computations, emitting results either back to the broker or directly to a serving layer. A serving layer, often a time-series database (such as Influx DB or Timescale DB), a low-latency key-value store, or an in-memory cache, stores the most recent aggregated results in a form optimized for fast reads by dashboards. Finally, a visualization layer, discussed in depth in Chapter 4, queries or subscribes to the serving layer and renders continuously updating charts. Figure 5.2 illustrates a representative end-to-end reference architecture connecting these components.

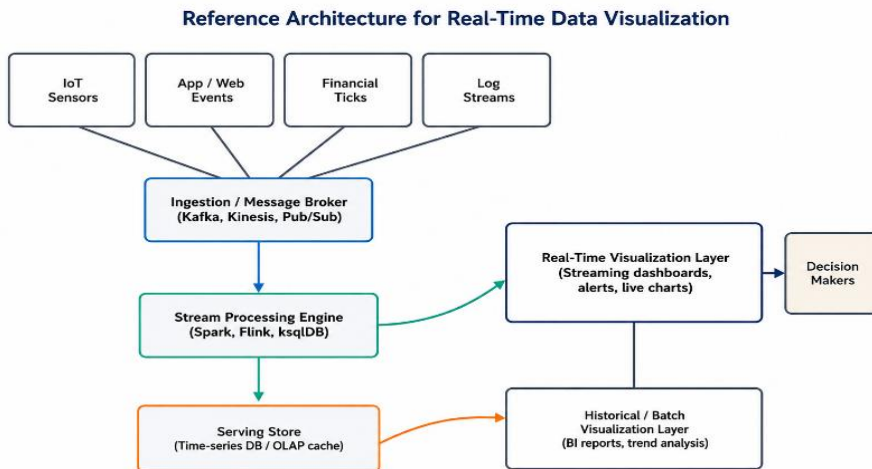


Figure 5.3: Reference architecture for real-time data visualization, from source systems through ingestion, stream processing, and serving, to the visualization layer and end users.

5.3.5 Two Dominant Processing Paradigms: Lambda and Kappa

Two architectural philosophies have shaped how organizations combine batch and stream processing. The Lambda architecture maintains two parallel processing paths: a batch layer that periodically recomputes accurate, comprehensive results over the full historical dataset, and a speed layer that processes only recent data to provide low-latency approximate results, with a serving layer merging both views for query. Lambda architectures offer strong correctness guarantees but at the cost of maintaining two separate codebases and reconciling their outputs. The Kappa architecture, proposed as a simplification, uses a single stream-processing pipeline for both real-time and historical processing, treating batch computation as simply a replay of the stream from an earlier offset; this reduces operational complexity but requires a stream-processing engine and storage layer capable of efficiently replaying large volumes of historical events. Modern managed platforms increasingly favor Kappa-style designs as stream processors have matured in their ability to handle large-scale reprocessing.

5.3.6 Characteristics That Distinguish Streaming Visualization

Several properties of streaming data create requirements that traditional, batch-oriented visualization tools were not originally designed to satisfy. Volume and velocity mean that a dashboard may need to summarize thousands of events per second without redrawing an entire chart on every single event, which would overwhelm both the rendering engine and the viewer's perception. Unboundedness means that summary statistics such as averages or percentiles must be computed incrementally, using windowed or approximate algorithms, rather than by re-scanning a complete dataset. Volatility means that the "current" state of a metric is continuously changing, so a visualization must clearly communicate both the latest value and its recent trajectory, rather than presenting a single frozen snapshot. Out-of-order arrival means that a chart may need to be revised retroactively as late data arrives, which can be visually disorienting if not handled carefully, for example by clearly marking provisional versus finalized values.

5.4 Data Visualization for Real-Time Data Streaming

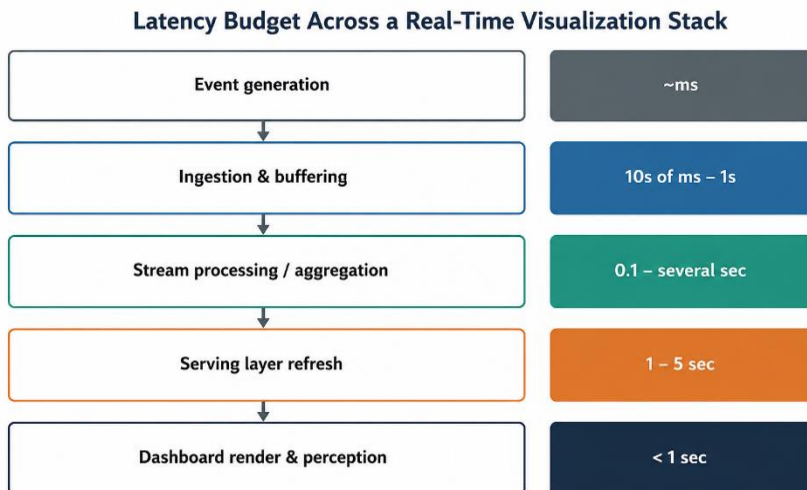
5.4.1 What Makes Real-Time Visualization Different

Traditional dashboards are typically refreshed on a fixed schedule perhaps once an hour or once a day and are designed around the assumption that the underlying data is essentially static during a viewing session. Real-time visualization inverts

this assumption: the underlying data is changing continuously while the viewer is looking at it, and the chart must be redrawn, or incrementally updated, frequently enough that the visualization tracks reality within an acceptable delay. This changes the design problem in several important ways. The visualization must be computationally cheap to update, since it may need to refresh many times per minute without noticeable flicker or performance degradation. It must communicate not just a value but a rate of change and a trend, since the viewer's primary question is often "is this getting better or worse, and how fast?" And it must manage visual stability, since data arriving at high frequency can cause charts to jitter distractingly if not smoothed or throttled appropriately.

5.4.2 Latency Budgets and the Meaning of "Real Time"

The phrase "real time" is used loosely in industry and rarely means true zero latency. It is more useful to think in terms of a latency budget: the maximum acceptable delay between an event occurring and that event being reflected in a visualization, given the decision the visualization supports. A dashboard monitoring a manufacturing line for safety-critical anomalies may need sub-second latency; a dashboard tracking daily sales trends refreshed every few minutes is still reasonably described as "real time" relative to a legacy nightly batch report. Figure 5.3 decomposes the end-to-end latency of a typical real-time visualization stack into its component layers, showing how delay accumulates from event generation through ingestion, processing, serving, and rendering.



Total end-to-end latency = sum of each layer; dashboards labeled "real-time" typically target a few seconds, not zero.

Figure 5.4: Latency budget across a real-time visualization stack, showing how delay accumulates across ingestion, processing, serving, and rendering layers.

Designers should set an explicit target latency for each dashboard based on the decision it supports, then work backward to determine which architectural choices (window size, refresh interval, aggregation strategy) can meet that target without imposing unnecessary infrastructure cost. Striving for lower latency than a use case requires is a common and costly design mistake.

5.4.3 Chart Types Suited to Streaming Data

Not every chart type translates well to a continuously updating context. Line charts remain the workhorse of streaming visualization because they naturally show a value's trajectory over time and can be updated incrementally by simply appending a new point and, typically, dropping the oldest point from a fixed-width scrolling window. Sparklines small, axis-free line charts embedded inline within text or tables are especially effective for dense dashboards summarizing many metrics at a glance. Gauges and single-value "big number" tiles communicate a current reading against a known range or threshold, and work well when paired with a small trend indicator (an arrow or micro-sparkline) showing recent direction. Bullet charts extend the gauge concept by comparing a current value against a target and qualitative performance bands. Heat maps and matrix displays are effective for showing many parallel streams (for example, one row per server or per sensor) where color intensity encodes a value, allowing a viewer to scan dozens of streams simultaneously for outliers. Streaming bar charts and race charts work for comparing discrete categories (such as top products by live sales) but must be careful about excessive reordering, which can be visually distracting; techniques such as capping the reorder frequency or animating transitions smoothly help mitigate this. Control charts, discussed further in Chapter 8, overlay statistical control limits on a streaming metric to flag when it moves outside expected bounds.

5.4.4 Design Techniques for Continuously Updating Displays

Several specific techniques help make continuously updating visualizations both accurate and comfortable to view. Scrolling or rolling windows fix the amount of history shown (for example, "the last 15 minutes") so that the chart's horizontal scale remains stable even as new data arrives, rather than compressing indefinitely as a full history accumulates. Smoothing and aggregation, such as computing a rolling average or binning raw events into small time buckets before plotting, reduce visual noise from high-frequency fluctuations that would otherwise obscure the underlying trend. Throttled rendering updates the visual display on a fixed interval, such as once per second, even if underlying data arrives far more frequently, decoupling the update rate perceived by the human eye (which cannot usefully process changes much faster than a few times per second) from the

ingestion rate of the pipeline. Progressive disclosure of uncertainty marks recently arrived, not-yet-finalized data (for example, a window still waiting on late-arriving events) with a visually distinct style, such as a dashed line or lighter shading, so viewers do not mistake provisional values for confirmed ones. Alerting overlays draw threshold lines, shaded bands, or color changes directly onto the chart to make deviations self-evident without requiring the viewer to mentally compare the current value against a remembered baseline.

5.4.5 Interactivity in Real-Time Dashboards

Interactivity remains valuable in a streaming context but must be designed carefully so that user interactions do not conflict with automatic updates. Common patterns include a pause or freeze control, allowing a viewer to temporarily stop the live update in order to examine a specific moment without the chart shifting underneath their cursor; time-range selection, allowing a viewer to zoom into a historical window while a separate, smaller "live" indicator continues to show the current value; drill-down, allowing a viewer to click an aggregated metric (for example, a spike in error rate) to see the underlying raw events that produced it; and filtering, allowing a viewer to narrow a multi-stream dashboard to a specific subset (for example, a single region or device) without reloading the entire page.

5.4.6 Common Pitfalls in Real-Time Visualization

Several failure modes recur often enough to warrant explicit warning. Over-updating -- redrawing a full chart on every single incoming event -- can cause flicker, excessive CPU and network load, and can actually reduce comprehension by making it harder for the eye to track a stable trend; throttled or batched rendering, described above, is almost always preferable. Misleading axis behavior, such as an auto-scaling y-axis that continuously rescales as new data arrives, can visually exaggerate or minimize the apparent magnitude of change; fixing axis ranges where feasible, or clearly signaling when they change, helps preserve accurate perception. Ignoring late data can cause a dashboard to silently under-report a metric for a window that later receives delayed events, producing a discontinuity when the corrected value eventually replaces the provisional one; the progressive-disclosure technique described above mitigates this. Finally, conflating "real time" with "correct" is a conceptual pitfall: speed and accuracy are somewhat in tension in streaming systems, and dashboard designers must be transparent with viewers about the approximation or provisional nature of very fresh numbers.

5.4.7 Chapter Summary

This chapter examined how visualization techniques must adapt for continuously updating, real-time data. It introduced the concept of a latency budget as a way of grounding design decisions in the actual requirements of a use case, surveyed chart types well suited to streaming contexts, and described concrete design techniques -- rolling windows, smoothing, throttled rendering, and uncertainty disclosure -- that keep real-time dashboards both responsive and trustworthy. It closed with common pitfalls that new practitioners frequently encounter. The next chapter broadens the lens to best practices that apply across both real-time and traditional visualization and analysis.

5.5 Best Practices for Data Visualization and Analysis

5.5.1 Why Best Practices Matter

Data visualization sits at the intersection of statistics, design, and communication. A technically correct chart can still fail if it is misread, ignored, or misinterpreted, and a visually striking chart can still mislead if it distorts the underlying data. Best practices exist to reduce the gap between what the data actually shows and what the viewer perceives, across every stage from initial analysis through final presentation. This chapter organizes best practice around eight core principles, summarized in Figure 5.4, and then discusses each in turn.

5.5.2 Know Your Audience

Every visualization is created for someone, and the right level of detail, terminology, and interactivity depends entirely on who that audience is. An executive audience typically needs a small number of high-level metrics tied directly to business outcomes, presented with minimal jargon and a clear takeaway. An analyst audience can absorb greater density and may want the ability to filter, drill down, or export underlying data. A general public or customer-facing audience needs the simplest possible encoding, since it cannot be assumed to have any statistical background. Before building a single chart, practitioners should be able to answer: who will see this, what decision or action will they take after seeing it, and how much time will they realistically spend looking at it?

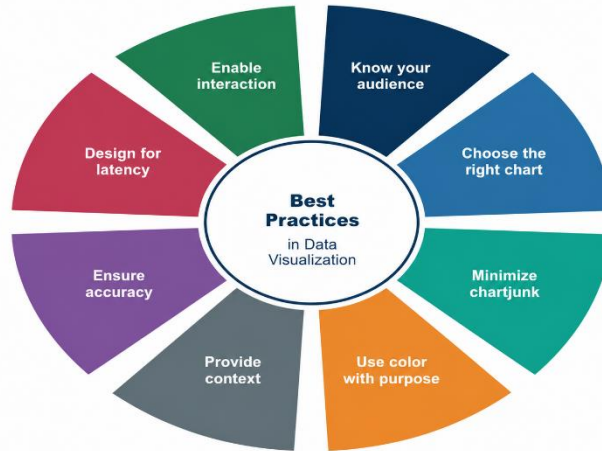


Figure 5.5: Eight core principles of best practice in data visualization and analysis.

5.5.3 Choose the Right Chart for the Data and the Message

Chart selection should be driven by the relationship the data expresses and the message intended, not by aesthetic preference alone. Comparisons among a small number of categories are well served by bar charts; trends over time call for line charts; part-to-whole relationships are best shown with stacked bars or, for very few categories, pie charts (used sparingly, since angular judgments are less accurate than length judgments); distributions are shown with histograms, box plots, or violin plots; relationships between two continuous variables call for scatter plots; and geographic patterns call for maps. A frequent error is selecting an unnecessarily elaborate chart type -- a 3D pie chart or a radial bar chart -- when a simple bar or line chart would communicate the same information more accurately and with less cognitive effort from the viewer.

5.5.4 Minimize Chart junk and Maximize the Data-Ink Ratio

Edward Tufte's influential concept of the data-ink ratio holds that the proportion of ink (or pixels) on a chart devoted to representing actual data, relative to the total ink used, should be as high as reasonably possible. Gridlines, borders, background shading, decorative icons, and heavy drop shadows -- collectively termed "chart junk" -- often add visual clutter without adding information, and can actively compete with the data for the viewer's attention. Best practice favors removing unnecessary borders and gridlines, using subtle rather than bold gridlines when they are needed for reading precise values, avoiding redundant encodings (for example, labeling every single bar with its exact value in addition to the visual bar

length, when only the largest or most relevant few need explicit labels), and using white space deliberately to group related elements and separate unrelated ones.

5.5.5 Use Color with Purpose

Color is one of the most powerful and most frequently misused visual channels. Sequential color scales (light to dark shades of a single hue) should be used for ordered, quantitative data such as a heat map of intensity. Diverging color scales (two hues meeting at a neutral midpoint) should be used when data has a meaningful zero or reference point, such as profit versus loss or above-versus-below target. Categorical color palettes (distinct, non-ordered hues) should be used for unordered categories such as product lines, and should generally be limited to seven or fewer categories, since the eye struggles to reliably distinguish more hues than that in a legend. Color should also be checked against common forms of color vision deficiency; palettes that rely solely on red-green contrast should be avoided or supplemented with a redundant encoding such as position or pattern. Finally, color should be used consistently across an entire dashboard or report -- if red means "underperforming" in one chart, it should not mean something unrelated in the next chart the same viewer will see.

5.5.6 Provide Context

A number is rarely meaningful in isolation. Best practice is to always provide a reference point: a prior period for comparison, a target or benchmark, an industry average, or a control group. Techniques for providing context include reference lines (a target or average drawn across a chart), annotations (brief text callouts marking specific events, such as a product launch or an outage, directly on a time series), and comparative framing (showing "this month vs. last month" or "actual vs. forecast" side by side rather than in isolation). Context is what turns a chart from a simple data display into an interpretable insight.

5.5.7 Ensure Accuracy and Avoid Distortion

Visualizations carry an implicit promise of honesty: the visual magnitude of a difference should correspond to the actual magnitude of the underlying data. Common distortions to avoid include truncated y-axes on bar charts, which can make small differences appear dramatic (a bar chart's y-axis should generally start at zero, whereas a line chart tracking a trend can sometimes reasonably use a non-zero baseline if the range is clearly labeled); inconsistent scales across small multiples, which make cross-panel comparison misleading; area or volume encodings that scale a two- or three-dimensional shape by a one-dimensional data value (a common error, since area scales with the square of a linear dimension,

exaggerating differences); and cherry-picked time windows that create a visually compelling trend by omitting the surrounding context in which the trend reverses.

5.5.8 Design for the Delivery Context

A static report intended to be printed or exported as PDF has different constraints from an interactive dashboard viewed on a monitor, which in turn differs from a mobile-optimized view. Designing for the delivery context means considering screen size and resolution, whether the audience will interact with the chart or only view it passively, whether the artifact needs to remain legible in black-and-white printing, and, for real-time dashboards specifically, the latency and refresh considerations discussed in Chapter 4.

5.5.9 Validate and Iterate

Best practice treats a visualization as a hypothesis about how to communicate a finding, not a finished product on the first attempt. Practitioners should test drafts with representative members of the intended audience, ask what conclusion they draw from the chart without prompting, and revise based on where that conclusion diverges from the intended message. Sanity-checking the underlying analysis -- verifying totals, checking for double-counting, confirming that filters are applied consistently -- should precede any investment in visual polish, since a well-designed chart of incorrect data is more dangerous than an ugly chart of correct data, precisely because polish increases perceived credibility.

5.6 Implementing Real-Time Data Visualization and Analytics Use Cases

5.6.1 From Principles to Practice

The preceding chapters established the conceptual foundation of streaming data, the design principles for real-time visualization, and general best practices in data visualization and analysis. This chapter applies those ideas to concrete implementation scenarios drawn from four domains where real-time visual analytics has become standard practice: industrial equipment monitoring, financial market surveillance, digital application and infrastructure monitoring, and logistics and fleet tracking. Each case study walks through the data sources, the processing pipeline, the visualization design, and the specific decisions the dashboard is intended to support.

5.6.2 Case Study: Real-Time Equipment Monitoring and Predictive Maintenance

Manufacturing and industrial operations increasingly instrument machinery with sensors measuring vibration, temperature, pressure, and rotational speed. The objective is to detect early signs of mechanical degradation before a failure causes unplanned downtime. Figure 5.5 shows a representative architecture for such a system.

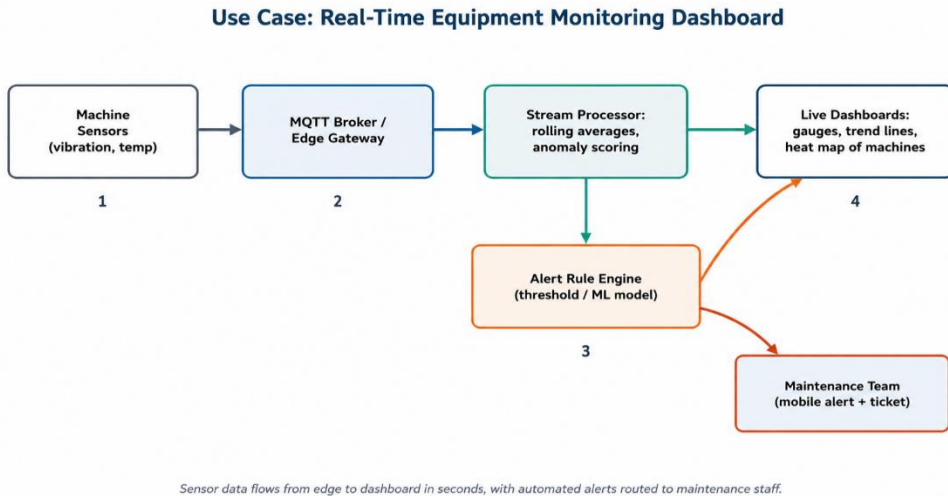


Figure 5.6: Reference architecture for a real-time equipment monitoring dashboard, from sensor to alert.

In this architecture, sensors publish readings to an edge gateway over a lightweight protocol such as MQTT, chosen for its low overhead on constrained devices and unreliable networks. The gateway forwards readings to a stream processor, which computes rolling statistics for example, a one-minute rolling average and standard deviation of vibration amplitude per machine and applies an anomaly-scoring model, which may be as simple as a statistical control chart (Chapter 8) or as sophisticated as a trained machine learning model (Chapter 9). An alert rule engine evaluates the anomaly score against configured thresholds and triggers notifications when a machine's behavior deviates meaningfully from its established baseline. The live dashboard itself typically combines several visualization elements from Chapter 5.4: a facility-wide heat map showing every machine as a colored tile (green for normal, amber for watch, red for alert) so a supervisor can scan dozens of machines at a glance; individual trend charts with control limits for any machine selected for closer inspection; and a running alert log with timestamps

and severity. When an alert fires, it is routed to a maintenance team's mobile devices, closing the loop between detection and action.

Key design decisions in this use case include choosing an appropriate rolling window length (short enough to catch fast-developing faults, long enough to avoid false alarms from normal operational noise), deciding whether anomaly thresholds should be fixed or should adapt to each machine's individual historical baseline, and ensuring the dashboard clearly distinguishes a genuinely new alert from a previously acknowledged one still technically outside normal range.

5.6.3 Case Study: Financial Market Surveillance

Trading desks and risk management functions rely on real-time visualization to monitor market data streams -- price ticks, order book depth, and trade execution reports -- arriving at extremely high frequency, sometimes tens of thousands of events per second for a single actively traded instrument. Latency budgets in this domain are typically the tightest of any use case discussed in this book, often requiring sub-second, and in some trading contexts sub-millisecond, visibility. Because raw tick-level plotting at that frequency would overwhelm both the rendering pipeline and human perception, dashboards nearly always employ some form of the smoothing and aggregation techniques from Chapter 4: candlestick charts aggregating ticks into fixed time buckets (for example, one-second or one-minute bars showing open, high, low, and close), volume-weighted average price overlays, and order-book heat maps showing the depth of buy and sell interest at each price level. Risk dashboards overlay statistical control-style bands (Chapter 8) on portfolio exposure metrics, flagging positions that move outside pre-approved risk limits, with automated alerts routed to risk officers. Because financial visualization directly informs trading and risk decisions with material monetary consequences, particular care is taken with the accuracy and distortion-avoidance principles from Chapter 5, and with clear visual disclosure of any provisional versus finalized values, since correcting a displayed price after a trading decision has already been made based on it is a serious operational risk.

5.6.4 Case Study: Application and Infrastructure Monitoring (Observability)

Software organizations rely on real-time dashboards, most commonly built on Grafana connected to Prometheus, InfluxDB, or Elasticsearch, to monitor the health of applications and the infrastructure that hosts them. Typical metrics include request rate, error rate, and latency (often summarized as the "RED" method: Rate, Errors, Duration), along with resource utilization such as CPU, memory, and disk I/O. These dashboards are consulted continuously by on-call engineering teams

and are central to incident response: when a service degrades, engineers use the dashboard both to detect the problem (often via automated alerting layered on top of the same metrics) and to diagnose it, drilling from a high-level service overview down to individual host or container-level detail. Design considerations specific to this use case include organizing dashboards hierarchically, from an organization-wide status overview down through service-level and host-level detail, so an engineer paged at 3 a.m. can orient quickly; using consistent color and threshold conventions across every team's dashboards so that "red" reliably means the same severity everywhere; and integrating the dashboard tightly with logging and distributed tracing systems so a spike visible on a chart can be drilled into for root-cause diagnosis within the same workflow.

5.6.5 Case Study: Logistics and Fleet Tracking

Logistics operators track delivery vehicles, shipping containers, or last-mile couriers in real time using GPS telemetry, combining geospatial visualization with the time-series techniques discussed elsewhere in this chapter. A live map plots current vehicle positions, often color-coded by status (on schedule, delayed, idle), while supporting panels show aggregate metrics such as on-time delivery percentage and average stop duration, updated continuously as new location and status events arrive. Because GPS pings can arrive out of order or be delayed by connectivity gaps, this use case is a practical illustration of the watermark and late-data handling concepts from Chapter 3: a vehicle's displayed position should be based on the most recent event time available, not simply the most recently processed event, or a vehicle could appear to jump backward on the map when a delayed ping is finally processed.

5.6.6 Cross-Cutting Implementation Lessons

Across all four case studies, several implementation lessons recur. Start with the decision, not the dashboard: a successful implementation begins by identifying the specific action a viewer will take in response to what they see, and works backward to determine what data, aggregation, and visual form supports that action, rather than starting from "let's build a dashboard" in the abstract. Right-size the latency: not every metric on a dashboard needs the same refresh rate, and over-engineering low-priority metrics for sub-second latency wastes infrastructure cost that could be better spent tightening the truly critical ones. Build in alerting from the start, since a dashboard that must be continuously watched by a human to be useful scales poorly; pairing visualization with automated threshold-based or model-based alerting, as in the equipment monitoring case, extends the reach of a monitoring system far beyond what constant human attention could achieve.

Finally, plan for failure of the pipeline itself: a real-time dashboard that silently stops updating because an upstream component failed is more dangerous than no dashboard at all, since viewers may continue trusting a stale display; dashboards should visibly indicate data freshness (for example, a "last updated" timestamp) and ideally alert separately if the pipeline itself has stalled.

5.7 Exploratory Data Analysis for Streaming Data

5.7.1 What Exploratory Data Analysis Means in a Streaming Context

Exploratory data analysis (EDA), as classically formulated by John Tukey, is the practice of examining a dataset using summary statistics and visualization to understand its structure, identify anomalies, and generate hypotheses before committing to a formal modeling approach. Traditional EDA assumes a static, complete dataset that the analyst can examine repeatedly from multiple angles. Streaming EDA must adapt this practice to data that is continuously arriving and unbounded, which means the analyst can never examine "the whole dataset" at once, and any summary must be computed incrementally and re-evaluated as new data arrives. This chapter describes how the core activities of EDA -- summarizing distributions, checking for outliers, and identifying relationships -- translate into a streaming setting.

5.7.2 Windowed Exploration

Because a stream has no final state, streaming EDA is almost always performed over a window: a bounded, recent slice of the stream, as introduced in Chapter 3. An analyst might examine the distribution of a metric over the last hour, compare it with the distribution over the previous hour, and repeat this comparison as each new window closes. Figure 5.6 illustrates this sliding-window approach to exploratory profiling.

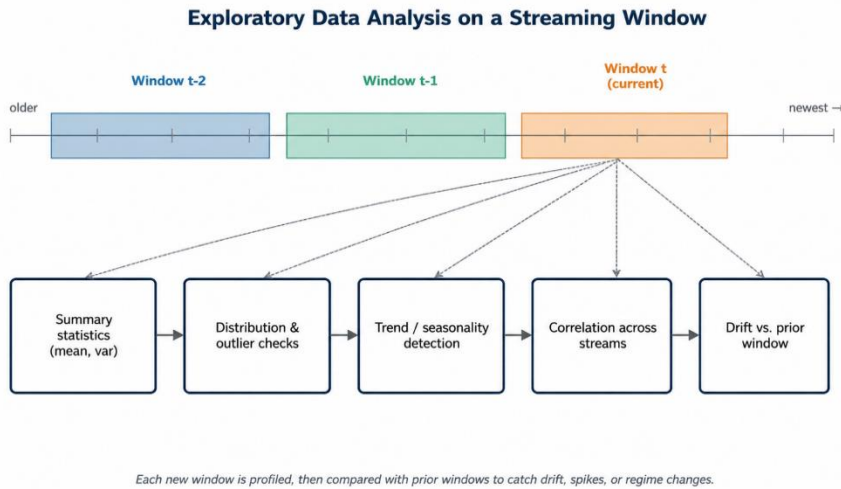


Figure 5.7: Exploratory data analysis on a streaming window, profiling each new window and comparing it against prior windows.

Each window is profiled using the same summary statistics an analyst would compute for a static dataset -- mean, median, variance, minimum, maximum, and quantiles -- but these are computed using algorithms designed for single-pass, memory-bounded computation, since re-scanning the full history for every new window is often infeasible at streaming scale. Common approaches include reservoir sampling, which maintains a fixed-size representative random sample of a stream regardless of how many events have passed, allowing standard statistical summaries to be computed on the sample as an approximation of the full window; and streaming/online algorithms for specific statistics, such as Welford's algorithm for computing a running mean and variance in a single pass with good numerical stability, or the t-digest and HyperLogLog data structures for approximating quantiles and cardinality (distinct counts) respectively within tightly bounded memory.

5.7.3 Distribution and Shape Checks

A central EDA question "what does this variable's distribution look like?" Remains just as important for streaming data, but the tools for answering it must work incrementally. Streaming histograms bin incoming values into a fixed or adaptively-sized set of buckets, updating bucket counts as each event arrives rather than requiring the full dataset in memory. Approximate quantile sketches (such as t-digest, mentioned above, or the GK algorithm) allow an analyst to query, at any moment, an approximate median, 95th percentile, or other quantile of the stream observed so far, within a small and bounded error tolerance. Comparing the shape

of the current window's distribution against a recent historical baseline for example, using a statistical distance measure such as the Kolmogorov-Smirnov statistic or population stability index -- is a standard technique for detecting distributional drift, a topic developed further in Chapter 9 in the context of machine learning model monitoring.

5.7.4 Outlier and Anomaly Detection in EDA

Identifying outliers is one of the most common goals of exploratory analysis, and it takes on particular operational significance in streaming contexts, since an outlier may represent a fraudulent transaction, a failing sensor, or a security intrusion requiring immediate attention. Simple, computationally cheap approaches suitable for high-velocity streams include threshold rules (flagging any value beyond a fixed or configurable bound), z-score methods (flagging values a specified number of standard deviations from a running mean, using the incremental mean and variance algorithms described above), and interquartile-range methods based on streaming quantile estimates.

5.7.5 Correlation and Relationship Discovery Across Streams

Many streaming environments involve multiple related metrics arriving simultaneously -- for example, CPU utilization, request latency, and error rate in an application monitoring context, or temperature, vibration, and rotational speed in an industrial monitoring context. Exploratory analysis of these multivariate streams seeks to identify which metrics move together, which is useful both for root-cause diagnosis (if latency and error rate both spike at the same moment as a deployment event, that co-occurrence is diagnostically meaningful) and for dimensionality reduction (if two metrics are almost always highly correlated, one may be dropped from a dashboard without losing much information). Streaming correlation can be computed incrementally using running covariance and variance estimates, analogous to Welford's algorithm for a single variable, updated as each new joint observation arrives. Cross-correlation with a time lag -- checking whether a change in one stream tends to precede a change in another by a consistent delay -- is particularly valuable in diagnostic settings, since it can suggest a causal ordering (for example, a rise in queue depth consistently preceding a rise in response latency by a few seconds).

5.7.6 Seasonality and Trend Decomposition

Many streaming metrics exhibit regular patterns tied to time of day, day of week, or other cycles -- website traffic typically peaks during business hours and troughs overnight, for instance. Exploratory analysis of streaming data benefits from

decomposing a metric into trend (the long-run direction), seasonal (the repeating cyclical pattern), and residual (what remains after removing trend and seasonality) components, since an anomaly detector or alerting rule that ignores seasonality will generate false alarms every time traffic follows its completely normal daily rhythm. In a streaming context, this decomposition is typically maintained incrementally, comparing each new observation against the expected value for the corresponding point in the seasonal cycle (for example, "is traffic right now unusually low compared with what we'd expect for a Tuesday at 2 p.m., based on recent history?") rather than performing a full batch decomposition on each update.

5.7.7 Visual Techniques for Streaming EDA

The visualizations most useful for streaming EDA differ somewhat from those used in a final, polished dashboard, since EDA is an analyst-facing, investigative activity rather than a broadcast display. Small multiples -- grids of many small charts, one per metric or per entity, sharing a common scale allow an analyst to rapidly scan for anomalies across a large number of streams simultaneously. Overlaid window comparisons, plotting the current window's distribution or trend directly against the previous window's, make drift immediately visible. Rolling correlation heat maps, updated as new windows close, show how relationships between metric pairs evolve over time rather than assuming a fixed correlation structure. Annotated event timelines, marking known external events (deployments, marketing campaigns, holidays) alongside a metric's trend, help an analyst distinguish an anomaly requiring investigation from an expected consequence of a known event.

5.8 Statistical Analysis for Streaming Data

5.8.1 The Role of Statistics in Streaming Analytics

Where exploratory data analysis, discussed in Chapter 7, generates hypotheses and surfaces patterns, formal statistical analysis provides the rigor to test those hypotheses, quantify uncertainty, and set principled thresholds for automated decisions such as alerts. Applying classical statistical methods to streaming data requires the same fundamental adaptation seen throughout this book: methods must operate incrementally, over bounded memory, and must account for the fact that "the sample" is continuously growing and its underlying distribution may itself be changing over time, a phenomenon addressed later in this chapter as non-stationarity.

5.8.2 Statistical Process Control

Statistical process control (SPC), originally developed for manufacturing quality control by Walter Shewhart in the 1920s, remains one of the most widely used and most directly applicable statistical frameworks for monitoring streaming metrics. The central tool of SPC is the control chart, which plots a metric over time alongside a center line (typically the historical mean) and control limits, most commonly set at three standard deviations above and below the mean. Figure 5.7 illustrates a control chart applied to a streaming metric, with several points flagged as out of statistical control.

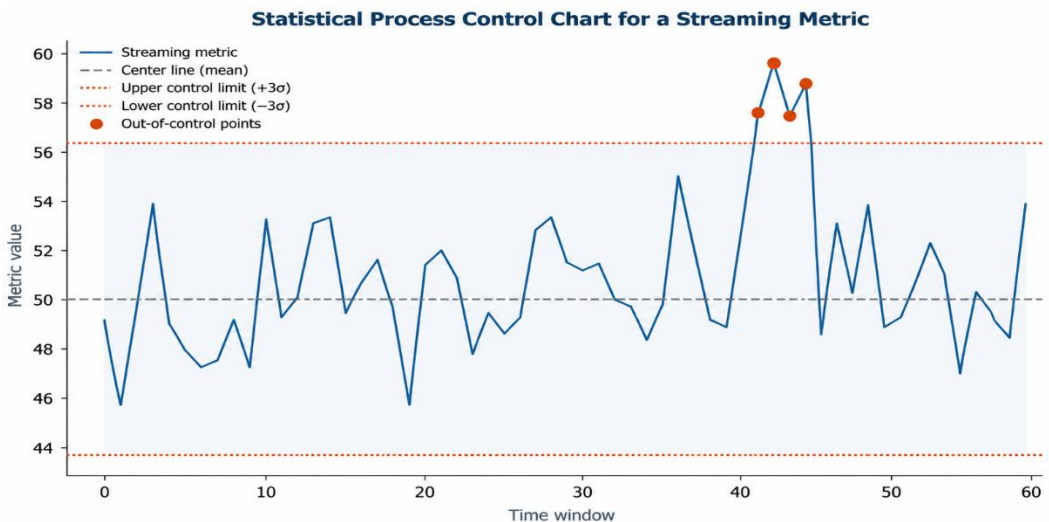


Figure 5.8: A statistical process control chart for a streaming metric, showing the center line, upper and lower control limits, and out-of-control points flagged for investigation.

The statistical logic behind the three-sigma limit is that, for a process in a stable, "in-control" state whose underlying variation follows an approximately normal distribution, roughly 99.7 percent of observations should fall within three standard deviations of the mean; a point outside that range is therefore either a rare chance event or, more likely in practice, evidence that something about the underlying process has genuinely changed. Beyond the basic three-sigma rule, SPC practice includes a set of supplementary rules (commonly called the Western Electric or Nelson rules) that flag more subtle patterns indicating an out-of-control process even when no single point exceeds the control limits, such as eight consecutive points on the same side of the center line, or a run of points steadily trending upward. These rules are directly implementable as streaming computations, since each only requires maintaining a short rolling history of recent points and their relationship to the (incrementally updated) center line.

In a streaming setting, the center line and control limits themselves are usually not fixed permanently but are periodically re-estimated from a rolling baseline window, since many real-world processes drift gradually over legitimate operational reasons (equipment wear, seasonal effects, gradual load growth) that should update the definition of "normal" rather than triggering perpetual false alarms. Balancing the responsiveness of this re-estimation (how quickly the baseline adapts to genuine change) against its stability (how resistant it is to being distorted by the very anomalies it is supposed to detect) is a key design decision in any streaming SPC implementation.

5.8.3 Hypothesis Testing on Streaming Data

Classical hypothesis testing computing a test statistic and comparing it against a reference distribution to accept or reject a null hypothesis can be adapted to streaming data, though with important caveats. Sequential hypothesis testing, most notably Wald's sequential probability ratio test, was specifically designed for situations where data arrives incrementally and a decision (accept, reject, or continue collecting data) should be made as early as possible while controlling the probability of error, making it naturally suited to streaming quality control and A/B testing scenarios where a business wants to detect a meaningful effect as quickly as responsible statistical practice allows. A/B and multivariate testing on streaming data (for example, evaluating two live versions of a product feature) must additionally guard against the well-documented risk of "peeking" repeatedly checking a test's significance as data accumulates and stopping as soon as a result looks significant which inflates the false-positive rate well beyond the nominal significance level unless a sequential testing correction, such as an alpha-spending function, is explicitly applied.

5.8.4 Change Point and Drift Detection

A recurring statistical task in streaming analysis is detecting the moment at which the underlying statistical properties of a stream change, a change point, as distinct from ordinary random fluctuation around a stable mean. The cumulative sum (CUSUM) algorithm is a classical and computationally lightweight method: it accumulates the running sum of deviations from an expected baseline value, resetting or signaling an alarm when the cumulative sum exceeds a threshold, making it more sensitive to small, sustained shifts than a simple control chart, which reacts primarily to individual large deviations. The exponentially weighted moving average (EWMA) is another widely used technique, computing a running average that weights recent observations more heavily than older ones according to a smoothing parameter, providing a tunable balance between responsiveness to

genuine recent change and robustness to transient noise; EWMA control limits can be derived analogously to Shewhart control limits, adapted for the reduced variance of the smoothed statistic. Bayesian online change point detection offers a more formally probabilistic alternative, maintaining a probability distribution over "how long since the last change point" that is updated as each new observation arrives, and is particularly useful when an analyst wants not just a binary alarm but a calibrated estimate of the likelihood that a change has occurred.

5.8.5 Time Series Models for Streaming Forecasting

Beyond detecting whether a stream has changed, statistical analysis is frequently used to forecast a stream's near-future values, supporting proactive rather than purely reactive decision-making (for example, forecasting that a queue will exceed capacity in the next ten minutes, in time to trigger auto-scaling before the queue actually overflows). Classical time series models, including autoregressive integrated moving average (ARIMA) and its seasonal extension (SARIMA), can be fit to a streaming metric's recent history and updated periodically as new data arrives, though full re-fitting on every new observation is often computationally prohibitive at high stream velocity, so practitioners typically re-fit on a coarser schedule (for example, every few minutes) while using the most recently fitted model to score each new observation in between. Exponential smoothing methods, including Holt-Winters smoothing, which explicitly models level, trend, and seasonal components, are popular in streaming contexts precisely because they are cheap to update incrementally and degrade gracefully as new data arrives, in contrast to the heavier re-fitting burden of ARIMA-family models. State-space models and the associated Kalman filter provide a principled, computationally efficient framework for maintaining and updating an estimate of a system's underlying (unobserved) state as noisy observations arrive one at a time, which is precisely the streaming setting, and Kalman filters remain a standard tool for streaming sensor fusion and smoothing in domains such as navigation, robotics, and industrial monitoring.

5.8.6 Non-Stationarity: The Central Statistical Challenge of Streaming Data

Many classical statistical methods assume the data is stationary that its statistical properties (mean, variance, autocorrelation structure) do not change over time. Streaming data is very frequently non-stationary in practice: user behavior shifts over the course of a day, equipment degrades gradually, market regimes change, and external events (a marketing campaign, a competitor's product launch, a public holiday) shift the baseline abruptly. A recurring theme across the methods in this

chapter -- rolling re-estimation of control limits, exponentially weighted statistics that discount old data, and explicit change point detection -- reflects the practical necessity of continuously re-grounding statistical analysis in a moving definition of "normal," rather than relying on a single, fixed statistical model estimated once from historical data and assumed to remain valid indefinitely.

5.9 Machine Learning for Streaming Data

5.9.1 Why Streaming Data Requires a Distinct Machine Learning Approach

Classical machine learning workflows assume a fixed training dataset: a model is trained once (or periodically retrained) on a batch of historical data, validated, and then deployed to make predictions on new inputs. This "train once, predict many times" paradigm runs into two difficulties in a streaming context. First, streaming data is unbounded, so there is no fixed "training set" to converge on new, potentially informative data is always arriving, and a model trained only on historical data may become progressively less accurate as the underlying data-generating process evolves. Second, many streaming applications require predictions with very low latency, often incompatible with the batch retraining cycles (hourly, daily, or longer) common in traditional machine learning operations. These constraints have given rise to a distinct subfield, commonly called online machine learning or incremental learning, and to specific engineering practices for deploying and monitoring models in streaming pipelines.

5.9.2 Online (Incremental) Learning Algorithms

Online learning algorithms are designed to update a model's parameters incrementally as each new labeled example arrives, rather than requiring the full dataset to be available for a single batch training pass. Stochastic gradient descent (SGD) and its variants, originally developed for training on large batch datasets in small random subsets, extend naturally to a true streaming setting: each new labeled example (or small mini-batch) produces a gradient update that nudges the model's parameters slightly, without needing to revisit any prior example. This makes SGD-based online learning a natural fit for streaming linear models, logistic regression, and the training of neural networks on continuously arriving data. Hoeffding trees (also known as Very Fast Decision Trees) are a streaming-native alternative to classical batch decision trees; rather than examining an entire dataset to decide the best split at each node, a Hoeffding tree uses a statistical bound (the Hoeffding inequality) to determine, from only a modest number of streamed examples, that a given split is very likely to be the same one that would have been

chosen with the full dataset, allowing the tree to grow incrementally with strong theoretical guarantees on its eventual accuracy. Online variants of ensemble methods, such as streaming random forests and adaptive boosting variants, combine multiple incrementally updated learners, often incorporating explicit mechanisms to detect and adapt to concept drift, described in Section 9.4 below.

5.9.3 Streaming Anomaly Detection with Machine Learning

Building on the simpler statistical outlier-detection methods introduced in Chapters 5.7 and 5.8, machine learning models offer more sophisticated anomaly detection for streaming data, particularly when normal behavior is multivariate or does not follow a simple, well-characterized statistical distribution. Isolation forests build an ensemble of random partitioning trees and flag points as anomalous if they can be isolated from the rest of the data with unusually few partitioning splits, an approach that adapts naturally to streaming data through incremental variants that periodically rebuild or update the forest on a rolling window of recent data. Autoencoders, a type of neural network trained to reconstruct their own input through a compressed intermediate representation, can be trained on data believed to represent "normal" behavior; when a new streaming observation produces an unusually large reconstruction error, it is flagged as potentially anomalous, an approach well suited to high-dimensional streaming data such as multivariate sensor readings, where simple threshold or z-score rules would need to be applied separately to each dimension and might miss anomalies that only appear in the joint relationship among dimensions. Clustering-based approaches maintain an evolving set of clusters representing normal behavior (using streaming clustering algorithms such as streaming k-means or DBSCAN variants) and flag new points that do not fit well into any existing cluster.

5.9.4 Concept Drift: Definition and Detection

Concept drift refers to a change over time in the statistical relationship between a model's input features and its target variable -- in other words, the very pattern the model was trained to learn is itself changing. This differs subtly from the data drift discussed in Chapter 7, which refers to a change in the distribution of input features alone; concept drift specifically means the mapping from inputs to outputs has changed, even if the inputs themselves look statistically similar to before. Concept drift is typically categorized by its temporal pattern: sudden drift, where the underlying concept changes abruptly at a specific point in time (for example, a fraud detection model facing a fundamentally new fraud technique introduced overnight); gradual drift, where the old and new concepts coexist for a transitional period with the new concept becoming progressively more dominant; incremental

drift, where the concept changes through a continuous sequence of small intermediate steps rather than a sharp transition; and recurring drift, where previously seen concepts reappear periodically (for example, a retail demand model where "holiday shopping behavior" recurs each year). Detecting concept drift typically relies on monitoring a model's predictive performance metrics (accuracy, error rate, or a proxy such as prediction confidence when true labels are delayed or unavailable) over a rolling window, and applying the same statistical change-point techniques described in Chapter 8 -- CUSUM, EWMA-based control charts, or dedicated drift-detection algorithms such as the Drift Detection Method (DDM) or Adaptive Windowing (ADWIN) -- to the resulting performance time series, flagging a drift event when performance degrades beyond statistically expected fluctuation.

5.9.5 The Online Machine Learning Pipeline

Figure 5.7 depicts a representative pipeline for deploying and maintaining a machine learning model against streaming data, integrating the online learning, anomaly detection, and drift detection concepts discussed above into a single operational loop.

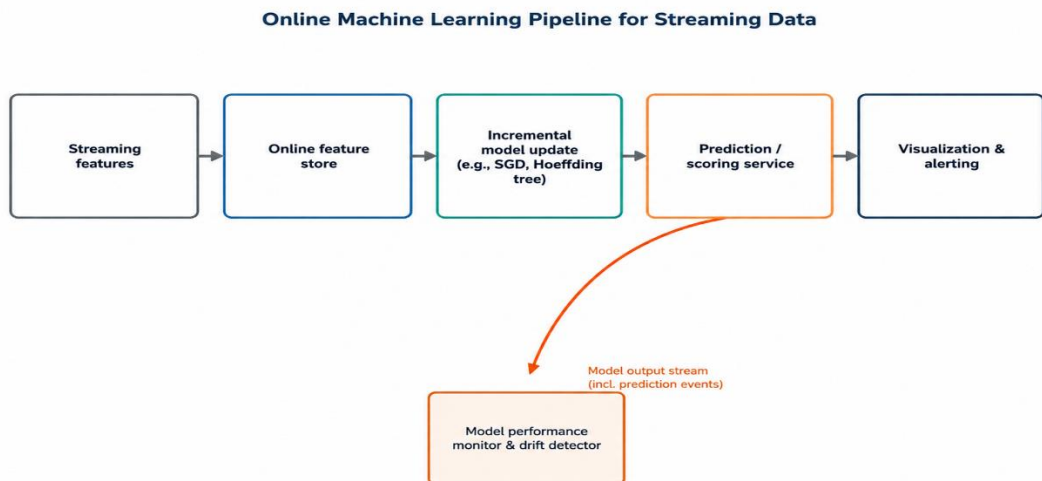


Figure 5.9: An online machine learning pipeline for streaming data, showing incremental model updates, prediction serving, and a feedback loop for drift detection and retraining.

Streaming features are computed from raw events (often reusing the windowed aggregation techniques from Chapters 5.3 and 5.7) and written to an online feature store, ensuring that the exact same feature computation logic used during training is used at prediction time, a consistency requirement often called "training-serving

skew" avoidance. The model itself is updated incrementally as new labeled data arrives, using the online learning algorithms, and serves predictions through a low-latency scoring service integrated with the visualization and alerting layer discussed in Chapters 5.4 and 5.6. Critically, a performance monitor tracks the model's ongoing accuracy against ground-truth labels as they become available (which may themselves be delayed -- for example, a fraud label may only be confirmed days after a transaction), applying drift detection methods to trigger a retraining or model-replacement workflow when performance degrades meaningfully, closing the loop between deployment and continuous improvement.

5.9.6 Visualizing Machine Learning Output in a Streaming Context

Machine learning predictions and their associated uncertainty must be visualized with the same care described in Chapters 4 and 5. Prediction confidence or probability, not just a binary classification, should typically be surfaced visually, for example through color intensity or a numeric score alongside an anomaly flag, so viewers can distinguish a borderline case from a highly confident detection. Model performance dashboards, tracking metrics such as rolling accuracy, precision, and recall alongside a visual indicator of detected drift events, should be maintained as a distinct, ongoing artifact separate from the operational dashboards the model feeds, since the audiences (data scientists and ML engineers monitoring model health, versus operational staff acting on predictions) and the appropriate level of technical detail differ meaningfully between the two. Explainability overlays, such as highlighting which input features contributed most to a specific flagged prediction, help build trust with operational users who must decide whether to act on a model's output, particularly in early deployment stages before a model has an established track record.

5.9.7 Practical Considerations and Limitations

Online machine learning introduces engineering and statistical trade-offs that practitioners should weigh carefully. Online learners are generally more sensitive to noisy individual updates than batch-trained models, which benefit from averaging over a full dataset; techniques such as mini-batching (updating on small groups of examples rather than one at a time) and careful learning-rate scheduling help stabilize online training. Labeled data for supervised online learning is often delayed or partially unavailable in streaming settings (as in the fraud detection example above), which constrains how quickly a model can actually adapt to genuine drift regardless of the underlying algorithm's theoretical capability. Finally, simpler statistical methods from Chapter 8 remain entirely appropriate, and often preferable, for many streaming monitoring tasks; machine learning

should be reserved for cases where the relationship between inputs and the outcome of interest is genuinely too complex for simpler statistical rules to capture adequately, since the operational complexity, computational cost, and reduced interpretability of a machine learning pipeline are real costs that should be justified by a correspondingly real gain in accuracy or coverage.

5.10 Data Storytelling with Real-Time Data Visualizations

5.10.1 What Is Data Storytelling?

Data storytelling is the practice of combining data, visualization, and narrative to communicate insight in a way that is not only accurate but also memorable and persuasive enough to drive action. It sits deliberately between two extremes: raw data exploration, which is thorough but often inaccessible to a non-expert audience, and pure narrative, which is engaging but risks distorting or oversimplifying the underlying evidence. Effective data storytelling preserves the rigor developed throughout this book -- accurate encoding (Chapter 5), sound statistical grounding (Chapter 8), and honest handling of streaming data's provisional nature (Chapter 4) -- while structuring the presentation of that evidence the way a skilled communicator structures any narrative: with context, tension, and resolution.

5.10.2 Why Storytelling Matters More, Not Less, for Real-Time Data

It might seem that storytelling, a technique associated with retrospective narrative, sits uneasily with real-time dashboards, which show a continuously changing present rather than a settled past. In practice, the opposite is true: real-time data is often harder to interpret without narrative framing, precisely because a viewer glancing at a live dashboard has no innate sense of whether the current value is normal, concerning, or a false alarm, unless the display actively supplies that context. A static year-end report can rely on a reader's patience to work through supporting detail; a live operations dashboard glanced at for a few seconds during a busy shift cannot. Effective real-time storytelling therefore front-loads interpretation -- clearly signaling "this is normal," "this is worth watching," or "this requires immediate action" -- rather than presenting raw, unframed numbers and leaving interpretation entirely to the viewer.

5.10.3 The Narrative Arc Applied to Streaming Data

Classical narrative structure -- establishing context, introducing rising tension, reaching a climax, and resolving toward a new stable state -- maps naturally onto how many streaming metrics actually behave during a notable event, such as an

operational incident, a demand spike, or a market shock. Figure 5.8 illustrates this narrative arc as applied to real-time data storytelling.

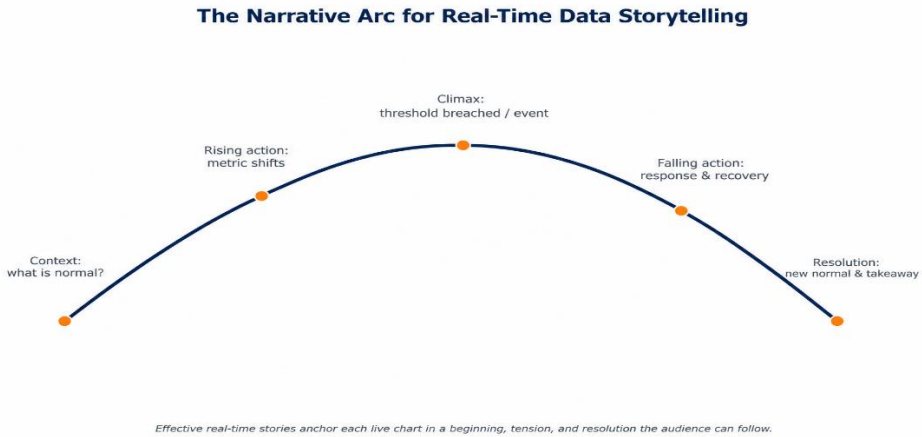


Figure 5.10: The narrative arc for real-time data storytelling, mapping classical narrative structure onto the lifecycle of a streaming event.

A well-told data story begins by establishing what "normal" looks like for the metric in question, whether through a visible historical baseline, a control band as described in Chapter 8, or simply a labeled reference line. It then makes the emerging tension visible as early as possible -- a metric beginning to diverge from its baseline -- ideally before the divergence becomes severe, since the entire purpose of real-time monitoring is to enable intervention before a full-blown incident. When a threshold is breached (the climax), the story should make unmistakably clear that a meaningful event, not ordinary noise, has occurred, often through the alerting overlays discussed in Chapter 4. The falling action and resolution stages -- showing the metric returning toward baseline, and any corrective actions taken -- are frequently underemphasized in dashboard design but are important both for closing the loop on the story and for building an organizational memory of how incidents were actually resolved, which is valuable for future incident response.

5.10.4 Techniques for Real-Time Data Storytelling

Several concrete techniques help embed narrative into real-time visualizations without sacrificing the accuracy principles from Chapter 5. Annotation is the most direct technique: marking a chart with a brief, specific label at the moment a significant event occurred (a deployment, an outage, a marketing launch) turns an unexplained spike into an understood one. Layered detail, presenting a simple, high-level summary as the default view with the ability to progressively drill into

supporting detail, respects a busy viewer's limited attention while still making full rigor available to anyone who needs it, echoing the audience-awareness principle from Chapter 5. Comparative framing situates a live number against a meaningful reference -- "23% above the same time last week" communicates far more than the raw number alone, and is a lightweight form of storytelling that requires no additional narrative text at all. Guided narratives, increasingly common in executive-facing real-time reporting tools, use a sequence of short automatically or manually curated views, each highlighting one specific finding with a brief explanatory caption, rather than presenting a single dense dashboard and leaving the viewer to find the story themselves.

5.10.5 Balancing Narrative and Rigor

The central tension in data storytelling is that narrative techniques -- selecting what to emphasize, choosing which comparison to foreground, deciding when a pattern is "the story" worth telling -- necessarily involve editorial judgment, which creates real risk of bias, whether intentional or not. Practitioners should apply several safeguards. They should show, rather than hide, uncertainty: a real-time metric described as trending in a concerning direction should still visually reflect the confidence interval or provisional status of very recent data, as discussed in Chapter 4, rather than presenting a noisy short-term fluctuation with the same visual confidence as an established trend. They should avoid cherry-picked timeframes, a distortion explicitly warned against in Chapter 5, by defaulting to a time window chosen for its representativeness rather than the one that most dramatically supports a predetermined narrative. They should distinguish correlation from causation explicitly in any narrative text accompanying a chart, particularly when two streaming metrics move together, since the ease of visually juxtaposing two live time series makes an unwarranted causal reading a very easy trap for both the storyteller and the audience to fall into. And they should retain access to the underlying, unedited data alongside any curated narrative view, so a skeptical or diligent viewer can always verify the story against the full evidence.

5.10.6 Storytelling for Different Real-Time Audiences

The techniques described above should be calibrated to audience, echoing Chapter 5's guidance. For operational, on-call audiences, the story is typically compressed into a single glance -- color, a threshold breach, and a one-line annotation -- because the audience needs to act within seconds, not absorb an extended narrative. For executive and strategic audiences, real-time data is often woven into a longer narrative arc spanning multiple related metrics over a business cycle, using comparative framing and periodic guided summaries rather than raw live

dashboards, since this audience typically consumes real-time information episodically rather than continuously. For technical and analytical audiences, storytelling is lighter-touch, since this audience is comfortable interpreting raw charts, statistical bands, and drill-down detail directly; over-narrating for this audience can feel patronizing or can obscure detail the audience specifically wants unfiltered.

5.10.7 Bringing It Together: Storytelling as the Capstone of Visual Analytics

This final chapter closes the arc of the book. Chapter 1 established that visualization and analytics exist to support decisions; Chapters 2 through 4 covered the tools and technical architecture that make real-time visualization possible; Chapter 5 established the design principles that keep visualization honest and effective; Chapter 6 grounded those principles in concrete implementation case studies; and Chapters 7 through 9 built the analytical foundation -- exploratory, statistical, and machine-learning-based -- that generates the insight a dashboard ultimately displays. Data storytelling is the discipline that ties all of these together into a communication that a human being will actually understand, remember, and act on. A technically flawless real-time analytics pipeline that produces a dashboard nobody correctly interprets has failed at its actual purpose just as surely as a pipeline with no data quality controls at all; storytelling, practiced with the rigor described in this chapter, is what closes that final and often underappreciated gap between correct analysis and effective decision-making.

5.11 Unit Summary

- Data visualization turns raw numbers into shapes, positions, colors, and sizes that the human visual system can interpret quickly; data analytics is the broader discipline of examining data to draw conclusions and support decisions.
- Real-time visualization differs from static reporting in its latency budget, its need to handle continuously arriving and revising data, and its emphasis on immediate actionability over exhaustive detail.
- Leading BI platforms (Tableau, Power BI) suit business-facing dashboards, while programmatic tools (D3.js, Plotly.js) suit custom, highly interactive, or public-facing visualizations that off-the-shelf BI tools cannot produce.
- Best practice in visualization design rests on knowing the audience, choosing the right chart, minimizing chartjunk, using color with purpose, and preserving accuracy even under real-time pressure.

- Exploratory data analysis, statistical process control, and streaming machine learning together form the analytical foundation that generates the insight a real-time dashboard ultimately displays.
- Data storytelling ties technical correctness to human understanding, and must balance narrative clarity against the risk of editorial bias in a live, continuously updating context.

5.12 Key Terms

Data visualization; data analytics; descriptive/diagnostic/predictive/prescriptive analytics; latency budget; Lambda architecture; Kappa architecture; chartjunk; data-ink ratio; exploratory data analysis (EDA); statistical process control; streaming machine learning; concept drift; data storytelling; guided narrative.

5.13 Review Questions

1. Distinguish data visualization from data analytics, and explain how they work together in a real-time decision pipeline.
2. What is a “latency budget,” and how does it differ across operational, executive, and technical audiences?
3. Compare Tableau/Power BI against D3.js/Plotly.js as approaches to real-time visualization, and identify one scenario best suited to each.
4. List and briefly explain four best-practice principles for effective data visualization.
5. Explain why streaming exploratory data analysis must be performed over windows rather than the full history of a stream.
6. What is a statistical process control chart, and how is it adapted for a continuously arriving metric?
7. Why must a streaming machine-learning model account for concept drift, and how can drift be detected in production?
8. Describe the narrative arc technique for data storytelling and explain one safeguard against editorial bias in real-time reporting.

Appendix: Laboratory Experiments, Assignments, Case Studies and Mini Projects

A practical companion to Units 1-5

A.0 About This Appendix

The five units of this book explain what streaming systems are, why they are built the way they are, and how their internal machinery works. This appendix puts that knowledge to work. It is organised as a single, continuous practical track built around one running scenario – **City Pulse**, a city-scale real-time operations platform – rather than a set of unrelated exercises. The same four data streams and message schemas run through every experiment, assignment, case study, and mini project that follows, so that a student who works through the appendix in order incrementally builds one realistic streaming platform: ingesting data in Experiment 1, processing it in Experiments 2-5, visualising it in Experiments 6-7, applying analytics in Experiments 8-9, and operating and monitoring it in Experiment 10.

Table App.1: Section; Contents; Suggested Use

Section	Contents	Suggested Use
A	Ten laboratory experiments, one per lab session	Regular lab classes (2 hours each)
B	Ten tool-wise assignments	Home assignments / continuous assessment
C	Ten industry case studies	Seminar / discussion component
D	Eight integrated mini projects	Skill-based activity / project component
E	Report format, rubrics, and viva bank	Evaluation and examination preparation

Every practical item maps to one or more Course Outcomes (CO1: streaming fundamentals; CO2: Kafka; CO3: Flink; CO4: Spark Streaming; CO5: advanced analytics – ML/statistics; CO6: end-to-end system design), so that outcome attainment can be demonstrated during review and students can see why each exercise is being performed.

A.1 The Running Scenario: City Pulse

The Scenario. City Pulse is the real-time operations platform of a mid-sized city of about two million people. It ingests live data from municipal air-quality sensors, the public-transport payment system, a citizen grievance/social-media feed, and the GPS units of registered taxis and buses. The control room needs to know, at any instant, which zones are breaching pollution limits, whether payment activity looks fraudulent, what citizens are complaining about, and where vehicles are congesting – questions that lose their value if answered even an hour late.

Four logical streams flow through the platform, each realised as one Kafka topic, all encoded as JSON so they can be read by Kafka, Storm, Spark, Flink, and JavaScript dashboards without extra serialisation libraries:

Table App.2: Stream; Kafka Topic; Description

Stream	Kafka Topic	Description
S1	sensor-telemetry	Air-quality readings per zone (PM2.5, PM10, temperature, humidity)
S2	txn-events	Public-transport payment transactions (amount, merchant, status)
S3	social-feed	Citizen grievance / social-media posts, tagged by zone
S4	trip-events	GPS pings from registered taxis and buses (location, speed)

No live municipal feed is required: a synthetic generator (`citypulse_gen.py`) produces all four streams at configurable rates and can inject anomalies on demand. Students with internet access may substitute a genuine public API (e.g., OpenAQ for air quality) without changing downstream code, since the schemas are preserved.

A.2 Laboratory Setup

The laboratory stack runs as a set of Docker containers – Kafka, Zookeeper/KRaft, Kafka UI, Storm, Spark, Flink, Prometheus, and Grafana – brought up with a single `docker compose up -d` command, so that no experiment requires a from-scratch cluster installation. Minimum recommended specification is a quad-core machine with 8 GB of RAM (16 GB preferred once Spark and Flink run together) and 20 GB of free disk space. Each student or team should maintain one working repository per the CityPulse scenario, committing code after every experiment so that later assignments and mini projects can reuse earlier pipelines rather than rebuilding them.

Section A – Laboratory Experiments

Each of the following ten experiments is a self-contained two-hour laboratory session. For every experiment, the tools required, its place within the CityPulse platform, the aim, a condensed procedure, the expected outcome, and one representative viva question are given; full starter code and detailed rubrics are provided in the accompanying laboratory manual.

Experiment 1 – Setting Up the Streaming Environment and a First Kafka Producer-Consumer Pipeline *Tools: Apache Kafka, Kafka UI, Python (kafka-*

python) / C01, C02 / 2 hours. Install and verify a single-node Kafka cluster, create a three-partition sensor-telemetry topic, and publish/consume the CityPulse sensor stream. Vary the number of consumers in a group from one to four and observe how partitions are (re)assigned, confirming that adding consumers beyond the partition count creates idle consumers rather than more throughput. Compare the latency of the streaming answer against an equivalent batch job over a saved file of the same events. *Viva*: Why does a fourth consumer joining a three-partition topic's group not increase throughput?

Experiment 2 – Real-Time Stream Processing Topology using Apache Storm

Tools: Apache Storm, Kafka / C01, C02 / 2 hours. Build a Storm topology with a spout reading sensor-telemetry and bolts that filter high-pollution readings and compute a rolling per-zone count. Configure tuple-level acking and observe at-least-once reprocessing after a deliberately failed bolt. *Viva*: Distinguish a spout from a bolt, and explain how Storm's acking mechanism achieves reliability.

Experiment 3 – Real-Time Data Analysis using Apache Spark Structured Streaming

Tools: Apache Spark, Kafka / C01, C04 / 2 hours. Read txn-events as a Structured Streaming DataFrame, compute a five-minute tumbling-window transaction count per zone, and write results to the console and to Parquet. Compare micro-batch trigger intervals of 1, 5, and 10 seconds and measure the latency/throughput trade-off. *Viva*: How does Structured Streaming's micro-batch model differ from Flink's native streaming model?

Experiment 4 – Real-Time Stream Processing and Analytics using Apache Flink

Tools: Apache Flink, Kafka / C01, C03 / 2 hours. Consume sensor-telemetry with the DataStream API, assign event-time watermarks tolerating five seconds of disorder, and compute a one-minute tumbling average PM2.5 per zone. Kill and restart a TaskManager mid-run and confirm that the checkpointed job resumes without losing or duplicating results. *Viva*: What role does a watermark play in deciding when a window may be closed?

Experiment 5 – Data Ingestion and Stream Analytics using Kafka Connect and Kafka Streams

Tools: Kafka Connect, Kafka Streams / C02 / 2 hours. Use a Kafka Connect JDBC source connector to capture changes from a small "parking permits" table into Kafka (change data capture), and write a Kafka Streams application that joins this reference table stream with trip-events to enrich each trip with the vehicle owner's permit status. *Viva*: What distinguishes a KStream from a KTable in the Kafka Streams API?

Experiment 6 – Real-Time Data Visualisation and Dashboarding using Tableau and Power BI

Tools: Tableau or Power BI, a streaming-friendly data source

/ CO5, CO6 / 2 hours. Connect Tableau/Power BI to a live-refreshing table populated by Experiment 3's output and build an operational dashboard showing zone-wise pollution and transaction trends with auto-refresh. *Viva*: What are the practical limits of BI-tool auto-refresh for genuinely sub-second use cases?

Experiment 7 – Interactive Real-Time Web Dashboard using D3.js and Plotly.js *Tools: D3.js or Plotly.js, WebSocket gateway / CO5, CO6 / 2 hours.* Build a browser dashboard that receives alert and metric events over a WebSocket and renders a live-updating zone map and time-series chart, updating within two seconds of a new event. *Viva*: Why might a custom web visualisation be preferred over an off-the-shelf BI tool for a public-facing dashboard?

Experiment 8 – Windowing, Event-Time Semantics and Streaming Statistical Analysis *Tools: Apache Flink / CO3, CO5 / 2 hours.* Implement tumbling, sliding, and session windows over trip-events, and compute a streaming mean/variance and a three-sigma control-chart boundary for vehicle speed per zone, flagging out-of-control points. *Viva*: Why is a session window unsuitable for a metric that must be reported at a fixed cadence?

Experiment 9 – Machine Learning on Streaming Data using Spark MLlib *Tools: Apache Spark MLlib / CO4, CO5 / 2 hours.* Train a streaming logistic-regression or isolation-forest model incrementally on txn-events to flag anomalous transactions, and evaluate it using precision-recall rather than accuracy, since fraud is a rare-event problem. *Viva*: Why is accuracy a misleading metric for a highly imbalanced fraud-detection stream?

Experiment 10 – Real-Time Monitoring, Alerting and Operating a Streaming Platform *Tools: Prometheus, Grafana, Kafka, Flink / CO2, CO3, CO6 / 2 hours.* Instrument the Experiment-4 Flink job and the Kafka cluster with Prometheus exporters, build a Grafana dashboard tracking consumer lag and checkpoint duration, and configure an Alertmanager rule that pages when lag exceeds a threshold for five minutes. This experiment closes the loop by making the whole CityPulse platform observable. *Viva*: Why should an alert threshold include a duration condition rather than firing on a single reading?

Section B – Tool-Wise Assignments

Each assignment is a take-home exercise (6-10 hours effort, 10 marks) extending the corresponding laboratory experiment with a harder, open-ended design problem.

Table App.3: #; Assignment; Primary Tool(s); Task Summary

#	Assignment	Primary Tool(s)	Task Summary
1	Designing an Ingestion Pipeline: Partitioning, Keys, Ordering	Kafka	Add a parking-events stream sustaining 5,000 events/s with guaranteed per-vehicle ordering; justify the key and partition-count design.
2	Building a Fault-Tolerant Storm Topology with Guaranteed Processing	Storm	Extend Experiment 2 with at-least-once acking end-to-end and demonstrate recovery from a killed worker without data loss.
3	Batch Versus Streaming: A Controlled Comparison	Spark	Run the same aggregation as a Spark batch job and a Structured Streaming job; quantify the latency-versus-throughput trade-off.
4	Complex Event Processing for Multi-Stage Fraud Detection	Flink CEP	Detect a multi-step fraud pattern (small test charge followed by a large charge within two minutes) using Flink's Pattern API.
5	Change Data Capture and Real-Time Enrichment	Kafka Connect	Capture changes from a reference table and enrich a live stream with the latest values, handling late-arriving reference updates.
6	An Executive Analytical Dashboard for the Municipal Commissioner	Tableau / Power BI	Design a summary dashboard answering city-wide trend questions for a non-technical executive audience.
7	An Operational Control-Room Dashboard with Live Alerting	D3.js / Plotly.js	Build a dashboard for control-room operators emphasising immediate anomaly visibility over historical trend.
8	A Custom Real-Time Visualisation That No BI Tool Can Produce	D3.js	Design and justify one visualisation (e.g., an animated flow map) that off-the-shelf BI tools cannot render.
9	Unsupervised Anomaly Detection on a Live Stream	Spark MLlib	Apply an unsupervised method (e.g., isolation forest) to detect anomalies without labelled data and report precision-recall.
10	Service Level Objectives and an Alerting Strategy	Prometheus / Grafana	Define SLOs for the CityPulse platform and implement a tiered (paging vs informational) alerting policy.

Each assignment is evaluated on: correctness and justification of the core design decision (30%), a working implementation with reproducible evidence (30%), a

measurement/plot with a one-paragraph interpretation (20%), and a written design note (20%).

Section C – Industry Case Studies

Each case study should be used as the basis of a seminar discussion of roughly thirty minutes, covering the situation the organisation faced, the streaming architecture adopted, and the durable lessons learned.

Case Study 1 – The Birth of the Distributed Log: Apache Kafka at LinkedIn. By around 2010, LinkedIn’s data architecture had become a tangle of point-to-point feeds between a growing number of systems, whose integration cost grows roughly as the square of the system count. Kafka’s insight was to treat this as a logging problem: every event is appended to a single durable, ordered log, and every consumer reads that log independently at its own pace, collapsing the integration cost from quadratic to linear. The lasting lessons are that retention policy (not throughput) is the decision with the widest consequences, that schema governance is load-bearing rather than bureaucratic, and that the partition key is a modelling decision that cannot be changed later without reprocessing.

Case Study 2 – Computing Trends in Real Time: Apache Storm at Twitter. Twitter needed to compute trending topics from a firehose of tweets within seconds, a workload for which periodic batch jobs were structurally too slow. Storm’s topology model (spouts producing tuples, bolts transforming them, tuple-level acking for reliability) allowed continuous, low-latency counting with bounded memory using approximate algorithms. The lesson: exact computation is often unnecessary and unaffordable at this scale – approximate, memory-bounded algorithms are a design requirement, not a shortcut.

Case Study 3 – Unified Batch and Streaming Analytics on a Media Platform. A media company needed both nightly recommendation-model retraining (batch) and live view-count updates (streaming) from the same event log, without maintaining two separate codebases. Adopting Spark allowed the same DataFrame API to express both, reducing code duplication and operational surface area. The lesson: a unified batch/streaming API materially reduces the long-run engineering cost of maintaining two parallel pipelines for what is conceptually one dataset.

Case Study 4 – Marketplace Intelligence at a Ride-Hailing Company. Matching drivers to riders and computing dynamic (surge) pricing requires sub-second visibility into local supply and demand. A Flink-based pipeline computing per-zone, sliding-window supply/demand ratios enabled pricing decisions within the time budget the business required, something a periodic batch recomputation could not

meet. The lesson: some business logic is only viable at all if it can be computed in real time – there is no “slower but correct” fallback.

Case Study 5 – Modernising a Bank’s Core Data Flow with Change Data Capture. A bank needed downstream fraud and reporting systems to reflect core-banking database changes within seconds rather than the next nightly ETL run, without touching the legacy core system’s code. Change Data Capture (via Kafka Connect) turned the database’s write-ahead log into a stream, decoupling downstream consumers from the legacy schema. The lesson: CDC allows real-time capability to be added to a legacy system non-invasively, which is often the only politically and technically feasible path.

Case Study 6 – A Patient Flow Command Centre in a Hospital Network. A hospital network needed a real-time view of bed occupancy, admissions, and discharges across facilities to reduce emergency-department boarding times. Streaming ingestion of admission/discharge/transfer events into a Flink-computed command-centre dashboard shortened the time to detect capacity bottlenecks from hours to minutes. The lesson: in operational (not just analytical) settings, the value of a metric decays extremely fast, making streaming a patient-safety concern rather than a purely technical preference.

Case Study 7 – Airport Operations Under Disruption. During weather disruptions, an airport’s operations team needed to re-plan gate assignments and ground crew scheduling as flight delays cascaded, faster than a periodic scheduling batch job could react. A streaming re-optimisation loop, triggered by flight-status events, reduced cascading delay by re-planning within minutes of a disruption rather than at the next scheduled run. The lesson: streaming systems are often justified less by average-case efficiency than by resilience during exactly the disrupted conditions when batch systems are weakest.

Case Study 8 – Election Night Graphics at a National Newspaper. A newspaper’s live election results page needed to update graphics for millions of concurrent readers within seconds of new counts being reported, with absolute correctness (no re-issued incorrect results). A Kafka-backed pipeline feeding a custom D3.js visualisation, with strict exactly-once processing on the count-aggregation stage, met both the latency and correctness bar. The lesson: for a public-facing, high-stakes number, exactly-once processing is not a performance nicety but a correctness and reputational requirement.

Case Study 9 – Real-Time Card Fraud Detection at a Payments Network. A payments network needed to approve or decline a transaction within the card-authorisation time budget (a few hundred milliseconds), scoring a machine-

learning fraud model in that window. A Flink pipeline maintaining per-card rolling features and scoring a streaming model inline met the latency budget without a separate offline scoring step. The lesson: when a decision must be made before the event completes (here, before authorisation), stream processing is not an optimisation – it is the only architecture that can meet the deadline at all.

Case Study 10 – Making a Platform Observable: Prometheus and Grafana. As a streaming platform’s component count grew, engineers found themselves unable to answer “is the pipeline healthy right now” without manually checking each service. Standardising on Prometheus metrics scraping and Grafana dashboards, with alerting tied to business-meaningful thresholds (such as consumer lag) rather than only infrastructure metrics, cut incident detection time sharply. The lesson: observability must be designed in from the start of a streaming platform, not retrofitted after the first unexplained outage.

Each case study concludes with 4-5 discussion questions and a short classroom activity in the accompanying laboratory manual, in which students redesign a piece of the CityPulse platform using the pattern illustrated by the case.

Section D – Mini Projects

Mini projects are team exercises (2-3 students, 3-4 weeks) integrating multiple units into one working sub-system of CityPulse. Each should be evaluated through weekly milestones, a final report, and a live demonstration with an induced-failure recovery test.

Table App.4: #; Mini Project; Tools Combined; Objective

#	Mini Project	Tools Combined	Objective
1	Air Quality Early-Warning System	Kafka, Flink, Prometheus, Grafana, D3.js	Predict a threshold breach before it happens, using trend extrapolation over a sliding window, not just detect it after.
2	Fraud Detection and Case Management	Kafka, Flink CEP, a case-management microservice	Detect multi-stage fraud patterns and route flagged transactions into a reviewable case queue.
3	Traffic Congestion Nowcasting	Kafka, Spark Structured Streaming, a mapping UI	Estimate current (not historical) congestion per road segment from GPS pings and display it on a live map.

#	Mini Project	Tools Combined	Objective
4	Citizen Grievance Triage and Routing	Kafka, Flink, an NLP classification model	Classify and route incoming grievance-feed posts to the correct municipal department in real time.
5	Transit Revenue Reconciliation	Kafka Connect, Kafka Streams	Reconcile streaming payment events against a batch settlement file and flag discrepancies as they occur.
6	Crisis Detection from a Citizen Feed	Kafka, Spark MLlib, an alerting service	Detect an emerging localised crisis (e.g., a burst of related complaints) from unstructured social posts.
7	Streaming Data Quality Observatory	Kafka, Flink, Prometheus, Grafana	Continuously monitor the CityPulse streams themselves for schema drift, null spikes, and delivery gaps.
8	The CityPulse Unified Control Room (Capstone)	All of the above	Integrate every earlier mini project's output into a single control-room view with unified alerting.

For every mini project, functional requirements should state explicit latency and correctness targets (for example, “recover from an induced Flink TaskManager failure without losing or duplicating an alert, demonstrated live”), and deliverables should include a measured end-to-end latency table (p50/p95/p99) at more than one load level, not only a working demo.

Section E – Report Format, Rubrics and Viva Bank

Standard Report Structure. Every experiment, assignment, and mini-project report submitted for evaluation should follow a common structure: (1) Aim/Objective, (2) Theory (brief, in the student's own words), (3) Design/Architecture, including a labelled diagram of the pipeline, (4) Implementation, with key code excerpts rather than a full listing, (5) Results, including any required measurement table or plot, (6) Discussion/Limitations, and (7) Viva-Voce responses, if conducted in writing.

Table App.5: Criterion; Typical Weight

Criterion	Typical Weight
Correctness and justification of the core design decision	30%
Working implementation with reproducible evidence	30%
Measurement, plot, or demonstration with interpretation	20%
Written report quality and completeness	20%

Consolidated Viva Bank. In addition to the item-specific viva question given with each experiment, examiners may draw on a general bank covering: the distinction between event time and processing time and why it matters; how Kafka's consumer-group mechanism achieves both parallelism and independent multi-consumer access to the same stream; why exactly-once semantics require coordination between the source, the processing engine, and the sink, not the processing engine alone; the difference between a tumbling, sliding, and session window, with a scenario in which each is the correct choice; why consumer lag is a better alerting signal than CPU utilisation for a streaming pipeline; and, for any project a student presents, what would break first if the input rate were increased by a factor of ten, and how the design would need to change to absorb it.

INDEX

A

Analytics 10, 14, 25, 50, 74, 129
Algorithm 10, 91, 100, 107, 127
Aggregation 2, 4, 7, 14, 22, 25
Anomaly Detection 3, 66, 68, 101
API 14, 17, 25, 33, 41, 57, 73, 76
Automation 11, 13
Availability 24, 51, 54, 58,
Apache Spark 3, 4, 14, 16, 17, 75, 77

B

Batch Processing 1, 2, 3, 4, 20, 21
Broker 9, 13, 16, 24, 27, 29, 31, 35,
47
Buffering 7, 142, 143,
Backup 4

C

Cluster 13, 17, 23, 24, 26, 29, 31
Consumer 24, 28, 29, 30, 47,
51, 61
Checkpointing 17, 57, 62, 63, 65,
69
Correlation 136, 157, 158, 168,
Classification 3, 165, 179,
Cybersecurity 11, 18, 66, 69,

D

Data Streaming 1, 2, 4, 13, 65,
66, 114
Data Processing 1, 20, 22, 23, 75,
119
Data Transformation 54, 68,
69, 73
Data Cleaning 73, 97, 100, 114
Data Visualization 173
Data Aggregation 100,
Data Pipeline 14, 83

E

Event Processing 24, 88, 129,
132
Event Time 2, 8, 18, 22, 53, 54,
59
Exploratory Data Analysis 155,
158, 170
ETL 21, 75, 177,

F

Flink 3, 5, 7, 9, 16,
Fault Tolerance 3, 5, 6, 9, 16, 17,
22, 31
Filtering 2, 7, 14, 24, 25, 37, 39
Forecasting 161,
Fraud Detection 10, 12, 21, 32,
43, 65
Framework 14, 20, 21, 24, 32, 47,
48, 52

G

Gateway 152, 174,
Governance 100, 109, 141, 176
Grouping 38, 39, 40, 43, 55
Gradient Descent 162

H

Hadoop 3, 21, 32, 75, 76, 78
Historical Data 20, 26, 52, 162,
Horizontal Scaling 29, 60, 61, 65
Hypothesis Testing 160,

I

IoT 4, 5, 6, 11, 18, 21, 24, 25, 50,
65
Ingestion 2, 6, 24, 25, 40
Information 1, 11, 36, 56, 104
Integration 16, 17, 78, 87, 94
Incremental Learning 162
Indexing 32

Infrastructure 13, 17, 40, 41, 78

J

Java 33, 124, 126

JSON 7, 172,

JVM 45

JDBC 173

K

Kafka 3, 21, 23, 24, 25

Kafka Cluster 29, 49, 173

Kafka Consumer 51, 124,

Kafka Producer 124, 172

Kafka Topic 36, 37, 40, 47

Kubernetes 77, 78, 87

L

Latency 3, 5, 14, 107, 145

Load Balancing 28, 38, 61

Lambda Architecture 121, 132,
144

Log Processing 72, 133

Live Data 171

M

Machine Learning 75, 76, 162,
163

Micro-Batching 44,

Monitoring 45, 50, 51

Message Broker 9, 16, 143,

Metadata 2, 7, 26, 50, 123

Model Training 9, 142,

Metrics 44, 51, 56, 83, 89, 100,
129

N

Node 22, 23, 34, 35, 41, 45, 50,
162

Network 6, 11, 22, 23, 28, 41, 54

Neural Network 163

Notification 31, 64, 81, 106

Non-Stationarity 161

O

Online Learning 162, 164, 165

Outlier Detection 98

Optimization 34, 43, 84, 136

Observability 153

Orchestration 78

P

Predictive Analytics 136

Pipeline 144

Partitioning 163, 175,

Producer 28, 30, 124

Processing Time 8, 18, 22, 53, 54

Power BI 137

Predictive Maintenance 10, 111,
112, 153

Q

Quality Control 10, 159, 160

Query Engine 139

R

Real-Time Processing 5, 14, 18,
120

Real-Time Analytics 14, 50, 74,
101, 104

Real-Time Dashboard 74, 103,
112, 114

Reliability 23, 31, 37, 40

Resilience 117

S

Stream Processing 3, 5, 25, 33

Spark 3, 7, 9, 14,

Storm 14, 15, 17, 18, 20, 21, 25

Streaming Analytics 1, 10, 18, 76

Scalability 93, 108, 109, 120

State Management 9, 18, 22, 39,
53

Statistical Analysis 158, 174

Sensor Data 65, 102, 105, 113

T

Transformation 2, 7, 9, 14, 18,
24
Tableau 137, 138
Topic 27
Throughput 3, 14, 17, 22, 26

U

Unbounded Data 2, 4, 5, 18, 79

V

Visualization 103, 105, 106, 107,
112
Validation 39, 64, 97, 99, 112
Velocity 6, 23, 65, 144
Volume 3, 6, 22, 42
Visual Analytics 135, 137, 151,
169

W

Windowing 2, 3, 4, 6, 7, 14, 18,
44
Web Analytics 139
Workflow 6, 154, 168
Watermark 18, 54, 85, 91
Workload 23, 28, 39, 60, 76
WebSocket 105, 174,

Y

YARN 77, 78, 87,
Y-Axis 147,

Z

ZooKeeper 23, 24, 26, 34,
Zero Latency 71, 145,
Z-Score 157

ABOUT THE BOOK

This textbook presents the design and implementation of real-time data streaming and analytics systems, covering concepts from fundamentals to production-grade tools. Unit 1 introduces data streaming and analysis, while Unit 2 explores Apache Kafka and Apache Storm. Unit 3 focuses on data transformation and analysis using Apache Spark and Apache Flink. Unit 4 covers real-time processing patterns, applications, analytics, decision-making, and monitoring. Unit 5 addresses data visualization and streaming analytics, followed by a practical appendix featuring laboratory experiments, assignments, case studies, and mini projects.

ABOUT THE AUTHORS

Dr. P. Renukadevi is an Associate Professor in the Department of Computer Science and Engineering (AIML) at Jain (Deemed-to-be University), Karnataka. She holds a Ph.D. from Anna University, Chennai, and has over 20 years of teaching experience. Her research interests include Data Analytics, with around 20 publications in reputed journals and conferences. She has received research funding from ICMR and Jain University and is a member of IEEE and ISTE.

Dr. Chinmaya Kumar Swain is an Assistant Professor of IT Systems and Analytics at IIM Jammu. He holds an MCA from Utkal University, an M.Tech. from IIT Bombay, and a Ph.D. in Cloud Computing from IIT Guwahati. His research interests include Cloud, Fog and Edge Computing, NLP, and Machine Learning. He has authored over 30 research papers in reputed journals and conferences.

Dr. Archana Sasi is an Assistant Professor in the Department of Computer Science and Engineering at Jain (Deemed-to-be University), Bengaluru. She holds a Ph.D. from Presidency University and has over 9 years of teaching and academic experience. She has published in SCI, SCIE, and Scopus-indexed journals and conferences and received two Best Paper Awards. Her research interests include Supply Chain Management, Machine Learning, and IoT.

Mr. Shahad P. is an Assistant Professor in the Department of Artificial Intelligence and Machine Learning at Jain (Deemed-to-be University), Bengaluru, with over 17 years of academic experience. He holds an M.Tech. from MEA Engineering College, Kerala, and has submitted his Ph.D. thesis at IIIT Kottayam. His research interests include Streaming Data Analysis, Machine Learning, and Responsible AI. He has published in SCI/Scopus-indexed venues and received two Best Paper Awards.



LURNEXA PUBLICATIONS,
130-187, Ramulavari Gudi Centre,
Gorantla, Guntur – 522034, AP, India

MRP - ₹499

ISBN 978-81-665077-4-6



9 788168 507746